Fleet Management

Software Requirements Specification

Multi-Component Stochastic Degradation MILP

Version 1.0 · October 26, 2026

Prepared with spec-creator · Intended for Claude Code implementation

Contents

1 Project Overview & Goals **3**

2 Index Sets and Notation **4**

3 Input Specification **5**
3.1 Sizes and Horizon . . . 5
3.2 Selectors . . . 6
3.3 Reliability . . . 6
3.4 Repair and Starting Anchors . . . 7
3.5 Initial State . . . 7
3.6 Replacement . . . 8
3.7 Increment Profiles and the Phase Lookup . . . 8
3.8 Model Constants . . . 9
3.9 Costs and Objective . . . 10
3.10 Run Options . . . 11
3.11 Rejected Keys . . . 12
3.12 Consistency Checks . . . 12

4 Decision Variables **15**

5 Objective Function **16**

6 Constraints **17**
6.1 Assignment Constraints . . . 17
6.2 Depot-Day Compatibility . . . 17
6.3 Repair Laws . . . 18
6.4 Encoding of the Dynamics . . . 19
6.4.1 Safety of the one-sided rows . . . 20
6.5 Replacement . . . 23
6.6 Linear Forms of the Reliability Rows . . . 23
6.7 Gaussian Model . . . 25
6.7.1 Reliability row . . . 26
6.8 Inverse Gaussian Model . . . 26
6.8.1 Reliability row . . . 27
6.9 Wiener Model . . . 28
6.9.1 Reliability row . . . 29
6.10 Gamma Model . . . 30
6.10.1 Class of true states . . . 30
6.10.2 The largest tail at a given mean . . . 31
6.10.3 Reliability row . . . 32
6.11 Rainflow Model . . . 34
6.11.1 Day maps . . . 35
6.11.2 Starting and replacement values . . . 35
6.11.3 Rows of the dynamics . . . 36
6.11.4 Reliability rows . . . 36
6.12 Repair Cost Variable . . . 38
6.13 Damage Regularisation . . . 39
6.14 Loop Constraint . . . 39

7 Full Optimization Problem **41**

8 Output Specification **42**
8.1 Result Dictionary . . . 42
8.2 Rules for the Values . . . 43
8.3 Reports . . . 44
8.4 Result Files . . . 45

9 Horizon Sweeps **45**
9.1 Cost Measures and the Ranking Criteria . . . 45
9.2 `sweep_operating_horizons` . . . 46
9.3 Early Stop . . . 47
9.4 Report, Checkpoint and Errors . . . 48
9.5 Parallel Cases . . . 49
9.6 `sweep_horizon_grid` . . . 49

10 Warm Starts **50**
10.1 Interface . . . 50
10.2 Day Mapping . . . 51
10.3 Start, Hints and Completion . . . 51
10.4 Relaxation Start . . . 52
10.5 Warm-Start Reports . . . 52
10.6 Warm Starts in Sweeps . . . 52
10.7 Greedy Heuristic . . . 52

11 File & Folder Structure **54**
11.1 Public API . . . 55
11.2 Cell-Builder Interface . . . 55
11.3 Plotting API and Layout . . . 56
11.3.1 Plot Layout . . . 57

12 Validation Tools **58**
12.1 Structural Validator . . . 58
12.2 Dashboards . . . 59
12.3 Validators of the Gamma Model . . . 60

13 Input File Schema **62**
13.1 Format Rules . . . 62
13.2 HDF5 Layout . . . 63
13.3 Keys by Cell Type . . . 63
13.4 YAML Example (Mixed Models) . . . 64

14 Open Questions **66**

15 Changelog **67**

References **68**

Project Overview & Goals

The Python package `fleet_management` schedules a fleet of F trains over a horizon of T days. Every day, M missions must be flown. Each train flies at most one mission, spends the day in the depot, or stays idle. Each train has L components, and each component degrades stochastically. On a depot day a component may be repaired (imperfect repair) or replaced. The schedule is a mixed-integer program solved with Gurobi.

Goal. Minimise the cost of depot days, repairs and replacements, plus a penalty on the largest train damage (Section 5), under a reliability requirement: for every component and every day, $\Pr(D_{i\ell k} > \tau_{i\ell}) \le \varepsilon_{i\ell}$, with $D_{i\ell k}$ the degradation and $\tau_{i\ell}$ the failure threshold. The horizon has two phases. The transitory phase (H_1 days) starts from the current state of the fleet. The operating phase (H_2 days) is the schedule to repeat. The requirement holds on every day of the horizon and of every later repetition of the operating phase (Section 6.14).

Degradation models. Each cell (component ℓ of train i) has its own model, chosen by the input `model`:

- **Gaussian**: a Gaussian random walk; tracks the mean and the variance, and a repair reduces both (Section 6.7).
- **Inverse Gaussian (IG)**: tracks the mean; reliability by a cap on the mean (Section 6.8).
- **Wiener**: Brownian motion with drift; tracks the mean and the variance. The variance grows by σ^2 per mission day; a repair acts on the mean only, and only a replacement lowers the variance (Section 6.9).
- **Gamma**: Gamma increments with mission-dependent rates; tracks the mean; reliability by a cap that is safe for every mix of rates (Section 6.10).
- **Rainflow**: distribution-free. The increment law is measured by rainflow counting of load histories and enters only through summaries (mean, variance, support width, cumulant generating function). Reliability is certified by one of five tail bounds: Markov, Cantelli, Hoeffding, Bernstein or Chernoff (Section 6.11).

Both the Gaussian and the Wiener model use Gaussian increments. The Gaussian model tracks a variance that a repair reduces. The Wiener model fits components whose uncertainty does not depend on the route and is not reduced by a repair. When in doubt, use the Gaussian model. The rainflow model makes no distributional assumption: its certificate holds for every increment law with the measured summaries, at the price of a looser bound than an exact distribution function.

Repair and replacement. Each cell has a repair law, ARD_∞ (the default) or ARD_1 (Section 6.3); every model accepts both. Each cell may be replaced (Section 6.5). The loop row forces at least one replacement in the operating phase in two cases. This is not an input check; a schedule without the replacement is infeasible.

- A Wiener cell whose train flies at least one mission in the operating phase: its variance falls only by replacement.
- An ARD_1 cell with $\rho_{i\ell} < 1$ that accumulates mean damage in the operating phase: a repair never removes the share $1 - \rho$ of the damage since the last intervention (Proposition 6.18).

Problem class. The problem is a mixed-integer linear program (MILP) when every reliability row is linear and the damage penalty is `inf_norm`. Otherwise it is a mixed-integer quadratically constrained program (MIQCP). It is nonconvex when a quadratic reliability row uses the `exact` form (Section 6.6); it is convex when the only quadratic row is the `quadratic` damage penalty (Section 6.13).

Scope of the package. The package solves one schedule (`solve`), sweeps the horizon lengths (Section 9), starts a solve from a previous solution or a heuristic (Section 10), plots a schedule

(Section 11.3), and checks a result against its input (Section 12). It reads inputs in YAML, JSON or HDF5 (Section 13) and writes the outputs of Section 8.

Authority. This specification is authoritative for the formulation and for the tools. The code follows it: every change is written here first.

Non-goals.

- Routing decisions are not optimised: the missions are inputs.
- Continuous-time formulations are out of scope: all dynamics are discrete-time, with a step of one day.

Index Sets and Notation

Symbol	Meaning	Defined in
Indices and horizon		
$i \in \{1, \dots, F\}$	train	
$\ell \in \{1, \dots, L\}$	component of a train	
(i, ℓ)	a *cell*: component ℓ of train i	
$j \in \{0, 1, \dots, M\}$	activity: $j = 0$ is a depot day, $j \geq 1$ a mission	
$k \in \{1, \dots, T\}$	day; index $k = 0$ holds the initial state	
$H = [H_1, H_2]$, $T = H_1 + H_2$	horizon input; a scalar H means $H_1 = H_2 = H$	3.1
days $1, \dots, H_1$	transitory phase	
days $H_1 + 1, \dots, T$	operating phase, repeated without end in the endless schedule	6.14
$p \in \{1, \dots, P\}$	segment of the `pwl` form, $P =$ `pwl_points`	6.6
Parameters, increments and states		
$\tau_{i\ell}$, $\varepsilon_{i\ell}$, $\rho_{i\ell}$	failure threshold, reliability level, repair efficiency of a cell; a model section may drop the subscript $i\ell$ after saying so	3
C_M; $C_{R,i\ell}$, $C_{\mathrm{rep},i\ell}$; C_D	costs: depot day; repair, replacement; damage penalty	5
$\mu^{\mathrm{inc}}_{i\ell jk}$, $v^{\mathrm{inc}}_{i\ell jk}$, $b^{\mathrm{inc}}_{i\ell jk}$, $K^{\mathrm{inc}}_{i\ell jk}$, $\beta_{i\ell jk}$	mean, variance, support width and cumulant generating function (CGF) of the increment of cell (i, ℓ) when train i flies mission j on day k, after the phase lookup; Gamma rate	3.7
$\delta\mu_{i\ell k} = \sum_{j=1}^{M} x_{ijk}\, \mu^{\mathrm{inc}}_{i\ell jk}$	day increment of the mean; δv, δR (from $(b^{\mathrm{inc}})^2$) and δK likewise; 0 on depot and idle days	
$\mu_{i\ell k}$, $v_{i\ell k}$, $R_{i\ell k}$, $K_{i\ell k}$	tracked mean, variance, Hoeffding budget, Chernoff quantity	4
$g^{\mu}_{i\ell k}$, $g^{v}_{i\ell k}$, $g^{R}_{i\ell k}$, $g^{K}_{i\ell k}$	anchors (law ARD_1)	6.3
$\mu_{0,i\ell}$, $v_{0,i\ell}$, $R_{0,i\ell}$, $K_{0,i\ell}$	initial values	3
$\mu^{\mathrm{new}}_{i\ell}$, $v^{\mathrm{new}}_{i\ell}$, $R^{\mathrm{new}}_{i\ell}$, $K^{\mathrm{new}}_{i\ell}$	replacement values	6.5
$b^{0}_{i\ell}$, $b^{\mathrm{new}}_{i\ell}$	support widths of the initial and replacement damage	6.11
ARD_∞, ARD_1; a; ψ	repair laws; repair factor; repair map	6.3
ξ_k, ζ_k, φ_k, Φ, $\mathcal{C}_{i\ell}$	full tracked state, reference trajectory, day map, map of the operating phase, certificate set	6.4, 6.14
Derived constants		
M_s	natural range of a tracked quantity s (big-M constant)	6.4
$\kappa_G = \Phi^{-1}(1 - \varepsilon)$	standard normal quantile	6.7
$V_{\max} = \tau^2/\kappa_G^2$	variance range, Gaussian and Wiener	6.7
$\eta_{i\ell}$; $Z \sim \mathrm{IG}(1, 1/\eta)$; q; $\bar{\mu} = \tau/\mathbb{E}[Z \mid Z > q]$	inverse Gaussian dispersion; unit law; its $(1 - \varepsilon)$-quantile; mean cap	6.8
$\beta^{*}_{i\ell}$; $t = \beta^{*}\tau$; $A_1(t, \varepsilon)$; $Q(a, x)$	smallest Gamma rate among the terms of the cell with a positive mean; scaled threshold; shape cap; regularised upper incomplete gamma function	6.10
$\kappa_C = \sqrt{(1 - \varepsilon)/\varepsilon}$	Cantelli coefficient	6.11

Symbol	Meaning	Defined in
$\kappa = \ln(1/\varepsilon)$	exponent constant (Hoeffding, Bernstein)	6.11
$b_{i\ell}$, $c_{i\ell} = \kappa\, b_{i\ell}/3$	Bernstein support constant and margin	6.11
$s_{i\ell}$	Chernoff parameter (input `s_chernoff`)	6.11
V^{C}, V^{B}	variance ranges, rainflow Cantelli and Bernstein	6.11
$\mu_r = \texttt{tangent_ref} \cdot \tau$; c_2, c_1	tangent point; constants of a quadratic row	6.6
E	evaluation horizon (input `evaluation_horizon`)	5

Input Specification

An input is one mapping from key names to values. This section lists every key, in groups, with its shape, its default and the cells that read it. Section 13 gives the file formats and the keys that each cell type needs.

Cells and abbreviations. A *cell* is a pair (i, ℓ) of a train and a component. Each cell has its own degradation model, repair law, reliability bound, linear form and encoding. The column *Applies to* names the cells that read a key. It uses these abbreviations: G (Gaussian), IG (inverse Gaussian), W (Wiener), Ga (Gamma), RF (rainflow). For a rainflow cell it names the tail bound: Mk (Markov), Ca (Cantelli), Ho (Hoeffding), Be (Bernstein), Ch (Chernoff). The word *all* means every cell.

Shapes. A *cell value* is a scalar, a list of L numbers, or an $F \times L$ array. A scalar is copied to every cell. A list of L numbers holds one value per component and is copied to every train. A *selector* has the same three shapes, with strings in place of numbers. A *profile* holds one value per cell, mission and day; its shapes are given in Section 3.7. A *flag* is `true` or `false`. No other broadcasting is accepted.

Absent entries. At a cell where a key does not apply, the entry of an array may be `null`, and the key may be omitted. At a cell where a key applies, `null` is an error, also when the key has a default. A broadcast value (a scalar, a list, or a profile without the axis F) is read only where the key applies; it gives no warning. A non-`null` entry of an array that has one entry per cell (an $F \times L$ array, or a profile with the axes F and L), at a cell where the key does not apply, is ignored with a `UserWarning`.

Keys. The names in the tables are the only keys that an input may hold. Names are case sensitive. Any other key raises an error (Section 3.11).

Sizes and Horizon

Key	Shape	Default	Applies to	Meaning
`F`	integer	required	all	Number of trains. Must exceed M
`M`	integer	required	all	Number of missions. Every mission is flown by one train on every day
`L`	integer	inferred	all	Number of components per train. When `model` is a list or an array, L is the length of its component axis, and a given `L` must equal it. Required when `model` is one string and $L > 1$
`H`	integer or `[H1, H2]`	required	all	Horizon (below). A scalar H means $H_1 = H_2 = H$

The horizon has $T = H_1 + H_2$ days. Days $1, \ldots, H_1$ form the *transitory phase*. Days $H_1+1, \ldots, T$ form the *operating phase*. Both H_1 and H_2 are positive integers. The operating phase is meant to repeat without end; the loop constraint of Section 6.14 makes every repetition safe. The profiles of Section 3.7 have H_2 entries for the operating phase and, optionally, H_1 entries for the transitory phase. Index $k = 0$ holds the initial state.

Selectors

Key	Shape	Default	Applies to	Meaning
`model`	selector	required	all	Degradation model of the cell: `gaussian`, `inverse_gaussian`, `wiener`, `gamma` or `rainflow`
`repair_model`	selector	`ardinf`	all	Repair law of the cell (Section 6.3): `ardinf` (the repair reduces the whole level) or `ard1` (it reduces the level accumulated since the last intervention). Every model accepts both
`bound_method`	selector	`cantelli`	RF	Tail bound of the reliability row (Section 6.11): `markov`, `cantelli`, `hoeffding`, `bernstein` or `chernoff`
`reliability_impl`	selector	`exact`	G, W, RF (Ca, Be, Ho)	Form of the reliability row (Section 6.6): `exact`, `tangent` or `pwl`. A cell whose row is linear (IG, Ga, Mk, Ch) is built in the form `exact`, and the value is ignored
`encoding`	selector	`bigm`	all	Encoding of the dynamics rows (Section 6.4): `bigm` or `indicator`
`component_names`	list of L strings	`Component` n for component n	all	Name of each component, shown by the plots. A name is not empty after blanks are removed

Reliability

Key	Shape	Default	Applies to	Meaning
`tau`	cell value	required	all	Degradation threshold $\tau_{i\ell} > 0$
`epsilon`	cell value	required	all	Reliability level $\varepsilon_{i\ell} \in (0, 1)$: the probability that the degradation exceeds $\tau_{i\ell}$ must not exceed it on any day. At G and W cells $\varepsilon_{i\ell} < 0.5$
`tangent_ref`	cell value	0.5	cells with form `tangent`	Relative tangent point in $[0, 1)$: the tangent is taken at the mean $\mu_r =$ `tangent_ref` $\cdot\, \tau$
`pwl_points`	integer	8	cells with form `pwl`	Number of segments of the piecewise linear form

Remark (range of ε). The Gaussian and Wiener rows contain $\big(\Phi^{-1}(1-\varepsilon)\big)^2$. This number is positive only for $\varepsilon < 0.5$. The variance range $V_{\max} = \tau^2/\big(\Phi^{-1}(1-\varepsilon)\big)^2$ is the largest variance that the row admits. It is the natural range of the variance, and the big-M constant of the variance rows (Section 6.4). The ratio $V_{\max}/\tau^2$ is 0.18 at $\varepsilon = 0.01$, 0.61 at 0.1, 3.6 at 0.3 and about 1600 at 0.49. The constant is therefore moderate unless ε is close to 0.5. The other models and bounds have no limit below $\varepsilon = 1$: for example, the variance cap $\tau^2\varepsilon/(1-\varepsilon)$ of the Cantelli row is finite for every $\varepsilon < 1$.

Repair and Starting Anchors

Key	Shape	Default	Applies to	Meaning
`rho`	cell value	required	all	Repair efficiency $\rho_{i\ell} \in (0, 1]$
`anchor_mu_0`	cell value	μ_0	`ard1` cells	Starting anchor $g^{\mu}_{i\ell 0}$ of the mean
`anchor_v_0`	cell value	v_0	`ard1` cells at G, RF (Ca, Be)	Starting anchor $g^{v}_{i\ell 0}$ of the variance
`anchor_R_0`	cell value	R_0	`ard1` cells at RF (Ho)	Starting anchor $g^{R}_{i\ell 0}$ of the Hoeffding budget
`anchor_K_0`	cell value	K_0	`ard1` cells at RF (Ch)	Starting anchor $g^{K}_{i\ell 0}$ of the Chernoff quantity

Under the law `ard1`, a repair removes the fraction ρ of the level accumulated since the last intervention (Section 6.3). Before the first intervention, the reference level is the starting anchor of the state. The default of each starting anchor is the initial value of the same state: μ_0, v_0, R_0 and K_0 (Section 3.5). The largest allowed value is this default, because damage only grows between interventions (Section 3.12). A larger anchor gives a weaker repair and a higher state, so the default never overstates what a repair achieves. For example, let $\mu_0 = 0.5$ and $\rho = 0.5$, and let a repair take place on day 1 before any mission. With the starting anchor 0 the mean falls to 0.25; with the default anchor 0.5 it stays at 0.5. The input is exact when the maintenance history of the component is known.

Initial State

Key	Shape	Default	Applies to	Meaning
`mu_0`	cell value	required	all	Initial mean degradation $\mu_{0,i\ell}$, with $0 \le \mu_{0,i\ell} < \tau_{i\ell}$
`v_0`	cell value	0	G, W, RF (Ca, Be)	Initial variance $v_{0,i\ell} \ge 0$. The value 0 means a known initial damage
`support_0`	cell value	0	RF (Be)	Width $b^0_{i\ell}$ of the range of the initial damage. The value 0 means a known initial damage
`R_0`	cell value	0	RF (Ho)	Initial value $R_{0,i\ell}$ of the Hoeffding budget: the square of the width of the range of the initial damage

Key	Shape	Default	Applies to	Meaning
`cgf_0`	cell value	$s\,\mu_0$	RF (Ch)	Initial value $K_{0,i\ell}$ of the Chernoff quantity: the cumulant generating function of the initial damage at $s_{i\ell} =$ `s_chernoff`. It is at least $s\,\mu_0$

The defaults describe a known initial damage: the mean μ_0 with no spread, so that $v_0 = b^0 = R_0 = 0$ and $K_0 = s\,\mu_0$. The optional keys describe a random initial damage. Section 6.11 states why these values keep the certificate valid.

Replacement

Key	Shape	Default	Applies to	Meaning
`replacement_mu`	cell value	0	all	Mean degradation $\mu_{i\ell}^{\text{new}}$ after a replacement
`replacement_v`	cell value	0	G, W, RF (Ca, Be)	Variance $v_{i\ell}^{\text{new}}$ after a replacement
`support_new`	cell value	0	RF (Be)	Width $b_{i\ell}^{\text{new}}$ of the range of the damage after a replacement
`R_new`	cell value	0	RF (Ho)	Hoeffding budget $R_{i\ell}^{\text{new}}$ after a replacement
`cgf_new`	cell value	$s\,\mu^{\text{new}}$	RF (Ch)	Chernoff quantity $K_{i\ell}^{\text{new}}$ after a replacement; at least $s\,\mu^{\text{new}}$
`replacement_as_new`	flag	`false`	all	If `true`, a replacement gives a component in the zero state, and every replacement input of this table must be 0
`allow_replacement`	flag	`true`	all	If `false`, no replacement is possible: the variables x^r and their rows are not built

The state after a replacement comes from these keys for every model. The defaults give a part in the zero state. Section 6.5 states the replacement rows; Section 6.11 states the reset of R and K. For Gamma, the rate of the state after a replacement is `gamma_beta_new` (Section 3.8).

Increment Profiles and the Phase Lookup

A *profile* gives the parameters of the damage increment when train i flies mission j on a day, for component ℓ. Its canonical shape is (F, L, M, H_*): train, component, mission, day of the phase. The last axis has H_2 entries for an operating profile and H_1 entries for a transitory profile. The parser accepts these shapes and expands each of them to (F, L, M, H_*) before it builds any row (*eager broadcasting*):

$$\text{scalar}, \quad (M), \quad (L, M), \quad (L, M, H_*), \quad (F, L, M), \quad (F, L, M, H_*).$$

A missing axis is repeated along that axis. A three-dimensional array whose shape fits both (L, M, H_*) and (F, L, M) is read as (L, M, H_*); this happens only when $F = L = M = H_*$. The parser cannot detect an array whose mission axis and component axis are swapped when $M = L$. Missions have index $j \in \{1, \ldots, M\}$ in a profile; a depot day or an idle day adds no increment.

Phase lookup. Let p be an operating profile and p^{tr} the transitory profile of the same key, if it is given. After the phase lookup, the value used on day $k \in \{1, \dots, T\}$ is

$$p^{\mathrm{inc}}_{i\ell jk} = \begin{cases} p^{\mathrm{tr}}_{i\ell jk} & k \le H_1 \text{ and } p^{\mathrm{tr}} \text{ is given,} \\ p_{i\ell j,\,((k-1) \bmod H_2)+1} & k \le H_1 \text{ and } p^{\mathrm{tr}} \text{ is not given,} \\ p_{i\ell j,\,((k-H_1-1) \bmod H_2)+1} & k > H_1. \end{cases} \tag{1}$$

Every profile key is looked up on its own. A transitory profile may be given for some keys of a cell and not for others. For a scalar H without transitory profile, both phases read the same H entries, one after the other. For $H = [2, 3]$ without transitory profile, days $1, \dots, 5$ read the operating entries $1, 2, 1, 2, 3$. With a transitory profile, days $1, 2$ read its entries $1, 2$ and days $3, 4, 5$ read the operating entries $1, 2, 3$. The symbols $\mu^{\mathrm{inc}}_{i\ell jk}$, $v^{\mathrm{inc}}_{i\ell jk}$, $b^{\mathrm{inc}}_{i\ell jk}$ and $K^{\mathrm{inc}}_{i\ell jk}$ of the formulation denote the values after this lookup.

Key	Shape	Default	Applies to	Meaning
`mu`	profile	required	all	Mean $\mu^{\mathrm{inc}}_{i\ell jk} \ge 0$ of the increment. For Ga the increment has shape $\mu\,\beta$ and rate β, so that its mean is μ
`v`	profile	required	G, RF (Ca, Be)	Variance $v^{\mathrm{inc}}_{i\ell jk} \ge 0$ of the increment
`support`	profile	required	RF (Ho, Be)	Width $b^{\mathrm{inc}}_{i\ell jk} \ge 0$ of the range of the increment: the increment lies in an interval of this length
`cgf`	profile	required	RF (Ch)	Cumulant generating function $K^{\mathrm{inc}}_{i\ell jk} = \ln \mathbb{E}\, e^{sX}$ of the increment X at $s = s_{i\ell}$
`s_chernoff`	cell value	required	RF (Ch)	Chernoff parameter $s_{i\ell}$, with $s_{i\ell} > \ln(1/\varepsilon_{i\ell})/\tau_{i\ell}$
`gamma_beta`	scalar, (L), (F, L), (F, L, M) or (F, L, M, H_2)	required	Ga	Rate $\beta_{i\ell jk} > 0$ of the Gamma increment. A shape without a day axis is repeated along the days
`mu_trans`, `v_trans`, `support_trans`, `cgf_trans`	transitory profile	none	as the key without the suffix	Increment parameters of the transitory phase. A transitory profile requires its operating profile
`gamma_beta_trans`	scalar, (L), (F, L), (F, L, M) or (F, L, M, H_1)	none	Ga	Rates of the transitory phase

The key `support` keeps one meaning in every bound: the width of the range of the increment. A width of 0 means that the increment is known.

Model Constants

Key	Shape	Default	Applies to	Meaning
`sigma`	cell value	required	W	Diffusion constant $\sigma_{i\ell} > 0$: every mission adds $\sigma^2_{i\ell}$ to the variance

Key	Shape	Default	Applies to	Meaning
`eta`	cell value	required	IG	Ratio $\eta_{i\ell} = \mu/\lambda > 0$ of the mean to the shape parameter of the inverse Gaussian law of the state (Section 6.8)
`v_max_user`	cell value	none	G, W, RF (Ca, Be)	Upper bound $V^{\text{user}}_{\max,i\ell} > 0$ on the variance. If it is given, the row $v_{i\ell k} \le V^{\text{user}}_{\max,i\ell}$ holds on every day. If it is not given, only the reliability row bounds the variance
`gamma_beta_0`	cell value	none (known damage)	Ga	Rate of the initial damage, which is a Gamma variable with mean μ_0. If it is omitted, the initial damage is the known constant μ_0, and it does not enter the smallest rate
`gamma_beta_new`	cell value	none (known damage)	Ga	Rate of the damage after a replacement, which is a Gamma variable with mean μ^{new}. If it is omitted, the damage after a replacement is the known constant μ^{new}, and it does not enter the smallest rate

For Gamma, the formulation computes the smallest rate of each cell from the rates of the terms with a positive mean: the entries of `gamma_beta` and `gamma_beta_trans`, and the rates `gamma_beta_0` and `gamma_beta_new` where they are given (Section 6.10). No input gives this rate.

Costs and Objective

Key	Shape	Default	Applies to	Meaning
`C_M`	scalar	required	all	Cost $C_M \ge 0$ of one depot day of one train, whatever the number of jobs done on that day
`C_R`	cell value	required	all	Repair cost $C_{R,i\ell} \ge 0$ per unit of degradation removed. At a cell with $C_{R,i\ell} = 0$ the variable z and its rows are not built
`C_rep`	cell value	required	all	Replacement cost $C_{\text{rep},i\ell} \ge 0$
`C_D`	scalar	required	all	Damage regularisation coefficient $C_D \ge 0$. If $C_D = 0$, the variable u and its rows are not built, and `penalty_type` is ignored
`penalty_type`	string	`inf_norm`	all	Form of the damage regularisation (Section 6.13): `inf_norm` or `quadratic`. The value `quadratic` requires the form `exact` at every cell

All costs are finite. The objective and its three modes (`objective_mode`, Section 3.10) are stated in Section 5.

Run Options

Run options are keys of the input. They do not change the mathematical problem, except `objective_mode` and `evaluation_horizon`, which choose the objective.

Key	Shape	Default	Meaning
`verbose`	integer	1	Gurobi output flag: 0 for no log, 1 for the log
`mip_gap`	number ≥ 0	0.1	Relative gap between the best solution and the best bound at which Gurobi stops (the MIP gap; MIP means mixed-integer program). With the default, the status `optimal` means that the best solution is at most 10 % above the best bound. The gap reached is reported as `mip_gap` in the result
`time_limit`	number > 0	3600	Time limit of Gurobi, in seconds
`fast`	flag	`false`	If `true`, a preset of Gurobi parameters that favours good solutions found early: `MIPFocus` $= 1$, `Heuristics` $= 0.5$, `ImproveStartGap` $= 0.5$ and `NoRelHeurTime` $= \max(2, 0.15\,$`time_limit`$)$
`gurobi_params`	mapping	none	Gurobi parameters, by name. They are set after every other option, and so override it
`objective_mode`	string	`total`	`total`: the cost of all T days; `operating_average`: the cost per day of the operating phase; `evaluation_total`: the projected cost over `evaluation_horizon` days. Each mode adds the damage penalty $C_D\,u$ once (Section 5)
`evaluation_horizon`	integer	none	Evaluation horizon E. Required, and used, only if `objective_mode` is `evaluation_total`; $E > H_1$. If it is given in another mode, the result reports the projected cost for it (Section 8)
`progress_interval_seconds`	number > 0	5.0	Minimum time in seconds between two records of the progress report (Section 8). A record is also made whenever the number of solutions found changes
`relaxation_warm_start`	flag	`false`	If `true`, and no other warm start is applied, a relaxation start is computed before the solve (Section 10)
`relaxation_time_limit`	number > 0	60.0	Time limit in seconds of the relaxation start

No input key selects a warm start from a previous result; Section 10 defines it. A horizon sweep is not an input option; the sweep functions of Section 9 read the same input and solve it for several horizons.

Rejected Keys

Each key of the table below names a quantity that has another name, or another meaning, in this specification. A key of the table raises an error. The message names the canonical key and, where the meaning differs, states the meaning of the canonical key. Any other key that is not in the tables of this section raises the error "unknown key", with the list of the canonical keys. The check on keys runs before any array is read.

Rejected key	Message: canonical key and meaning
`maintenance_type`	Use `repair_model`. Its values are `ardinf` (the repair reduces the whole level) and `ard1` (it reduces the level accumulated since the last intervention). In a file written with `maintenance_type`, the value `ARD1` corresponds to `ardinf`, and the value `ARA1` to `ard1`
`tail_bound`, `method`	Use `bound_method`; its values are `markov`, `cantelli`, `hoeffding`, `bernstein` and `chernoff`
`formulation`	Use `reliability_impl`. The value `exact` keeps its meaning. The value `lp` corresponds to `tangent` with `tangent_ref` equal to 0
`beta`	Use `gamma_beta`. It is a rate, and `beta` is a scale: `gamma_beta` equals 1/`beta`
`alpha_inc`	The Gamma increments are given by the mean `mu` and the rate `gamma_beta`. The shape of an increment is `mu` times `gamma_beta`
`gamma_beta_bound`	Remove it. The formulation computes the smallest rate of each cell from the rates of the cell (Section 6.10)
`gamma_calibration_method`	Remove it. The formulation has no calibration step (Section 6.10)
`b_inc`	Use `support`: the width of the range of the increment
`b_0`	Use `support_0`: the width of the range of the initial damage
`b_new`	Use `support_new`: the width of the range of the damage after a replacement
`n_workers`, `warm_start`	Remove them. Parallel runs and warm starts between horizons are options of the sweep functions (Sections 9 and 10)
`depot_capacity`	Remove it. The availability of the depot is not limited at fleet level (Section 6.2)
`C_P`	Remove it. The loop constraint is a hard constraint and has no penalty (Section 6.14)
`C_S`	Use `C_D`, with the same meaning
`alpha`	Use `tau`, with the same meaning
`xi`, `repair_rho`	Use `rho`, with the same meaning
`mu_new`	Use `replacement_mu`, with the same meaning
`v_new`	Use `replacement_v`, with the same meaning

Consistency Checks

The solver must validate all of the following before it builds the model. A violation raises a `ValueError`. Its message names the key and, for a cell value, the cell (i, ℓ). A check of one model runs only at the cells of that model, on the entries where the key applies. Each item gives the rule and, after the dash, the reason for it.

Keys, shapes and entries.

- *Keys.* Every key is in the tables of Section 3. The keys `F`, `M`, `H`, `model`, `tau`, `epsilon`, `rho`, `mu_0`, `mu`, `C_M`, `C_R`, `C_rep` and `C_D` are present, and so are the keys that each cell type requires (Section 13). — A misspelled or missing key must not change the problem without a message.
- *Shapes.* Each value has one of the shapes of its table row. The last axis of an operating profile has H_2 entries, and that of a transitory profile has H_1 entries. `component_names` has L entries. — Silent broadcasting by the array library could read a value for another cell.

- *Entries.* At a cell where a key applies, every entry is a finite number, or a string for a selector; `null` is an error. — A missing value at a cell must not take a default silently.

Sizes and selectors.

- F, M and L are positive integers, and $F > M$. — Every mission needs a train on every day; with $F = M$ no train could be in the depot, and no repair could take place.
- H is an integer ≥ 1, or a pair of integers $H_1, H_2 \geq 1$. — Each phase has at least one day.
- `model` entries are `gaussian`, `inverse_gaussian`, `wiener`, `gamma` or `rainflow`. `repair_model` entries are `ard1` or `ardinf`, for every model. — Another value names no model or no repair law.
- `bound_method` entries at rainflow cells are `markov`, `cantelli`, `hoeffding`, `bernstein` or `chernoff`. — A rainflow cell without one of these bounds has no reliability row.
- `reliability_impl` entries are `exact`, `tangent` or `pwl` at every cell; at a cell with a linear row (IG, Ga, Mk, Ch) a valid value is ignored, and the form `exact` is built. `encoding` entries are `bigm` or `indicator`. — An unknown value must raise an error at every cell; reading it as `exact` would turn a linear model into a nonconvex one without a message.
- `penalty_type` is `inf_norm` or `quadratic`. If $C_D > 0$ and `penalty_type` is `quadratic`, every cell is built in the form `exact`; a cell with a linear row always is. — The quadratic penalty is a quadratic row, and the linear forms exist to give a linear model.

Reliability, repair and initial state.

- $\tau_{i\ell} > 0$, and finite. — The threshold is a positive degradation level.
- $\varepsilon_{i\ell} \in (0, 1)$, and $\varepsilon_{i\ell} < 0.5$ at G and W cells. — A probability lies in $(0, 1)$; the squared normal quantile of the Gaussian and Wiener rows is positive only below 0.5 (Section 3.3).
- $\rho_{i\ell} \in (0, 1]$. — A repair removes a positive fraction of the level, at most all of it.
- $0 \leq \mu_{0,i\ell} < \tau_{i\ell}$. — A component at its threshold has failed already.
- v_0, `replacement_mu`, `replacement_v`, `support_0`, `support_new`, R_0 and R^{new} are ≥ 0. — They describe a damage, a variance, a width or a squared width.
- `tangent_ref` $\in [0, 1)$ at cells with form `tangent`, and `pwl_points` is an integer ≥ 1. — At `tangent_ref` $= 1$ the tangent row reduces to $Q \leq 0$, and a value outside $[0, 1]$ has no tangent point in $[0, \tau]$.
- Starting anchors: $0 \leq$ `anchor_mu_0` $\leq \mu_0$, $0 \leq$ `anchor_v_0` $\leq v_0$, $0 \leq$ `anchor_R_0` $\leq R_0$ and $0 \leq$ `anchor_K_0` $\leq K_0$, at the cells that read them. — Damage only grows between interventions, so the level after the last intervention cannot exceed the initial level.
- If `replacement_as_new` is `true`, then `replacement_mu`, `replacement_v`, `support_new`, `R_new` and `cgf_new` are 0 at every cell that reads them. The check does not run when the flag is `false`. — The shortcut means a new part, and a non-zero value would be discarded without a message.

Natural ranges and replacement states.

- Every initial value (μ_0, v_0, R_0, K_0) and every replacement value (μ^{new}, v^{new}, R^{new}, K^{new}, checked only if `allow_replacement` is `true`) lies in $[0, M_s]$, where M_s is the natural range of its state: τ for a mean; $V_{\max} = \tau^2/\big(\Phi^{-1}(1-\varepsilon)\big)^2$ for the variance at G and W cells; $\varepsilon\tau^2/(1-\varepsilon)$ for the variance at Ca cells; $\tau(\tau - 2c)/(2\kappa)$ at Be cells, with $c = \kappa b/3$ and $\kappa = \ln(1/\varepsilon)$; $2\tau^2/\kappa$ for R at Ho cells; $s\tau - \kappa$ for K at Ch cells (Sections 6.7 to 6.11). A starting anchor lies

between 0 and the initial value of its state, so it lies in the range as well. — The big-M rows take the natural range as their constant, so a value outside it violates a row at once.

- If `allow_replacement` is `true`, the replacement state of every cell satisfies the reliability row of the cell, with $(\mu^{\text{new}}, v^{\text{new}}, R^{\text{new}}, K^{\text{new}})$ in place of the tracked states (Sections 6.7 to 6.11). — The row holds on the day of a replacement; otherwise a replacement is never feasible.

Increments.

- $0 \le \mu^{\text{inc}}_{i\ell jk} < \tau_{i\ell}$ for every mission j and every day $k \in \{1, \dots, T\}$. — A mission with mean τ or more fails the component in one day; the mean 0 is allowed, so that a mission that does not wear a component can be written as 0.
- Where $\mu^{\text{inc}}_{i\ell jk} = 0$, the values $v^{\text{inc}}_{i\ell jk}$, $b^{\text{inc}}_{i\ell jk}$ and $K^{\text{inc}}_{i\ell jk}$ are 0. — A non-negative increment with mean 0 is 0 almost surely, so it has no variance, no range and a zero cumulant generating function.
- The values $v^{\text{inc}}_{i\ell jk}$, $b^{\text{inc}}_{i\ell jk}$ and $K^{\text{inc}}_{i\ell jk}$ are ≥ 0. A transitory profile has its operating profile. — A variance and a width are not negative, and the lookup (1) needs the operating entries after the transitory phase.

Gaussian cells.

- $\mu_0 \ge 3\sqrt{v_0}$ and $\mu^{\text{inc}}_{i\ell jk} \ge 3\sqrt{v^{\text{inc}}_{i\ell jk}}$ for every mission j and day k. — A Gaussian increment is then negative with probability at most about 0.13 %, so the damage stays non-negative in practice.

Inverse Gaussian cells.

- $\eta_{i\ell} > 0$. — The dispersion of the inverse Gaussian law must be positive.

Wiener cells.

- $\sigma_{i\ell} > 0$ and $v_{0,i\ell} \ge 0$. — The diffusion constant is positive; a known initial damage has variance 0 and is a valid input.

Gamma cells.

- Every entry of `gamma_beta` and of `gamma_beta_trans` is finite and > 0. — The rate of a Gamma law is positive, and the smallest rate sets the cap (Section 6.10).
- `gamma_beta_0` and `gamma_beta_new`, where they are given, are finite and > 0. — A rate is positive; an omitted rate means a known damage, which has no Gamma law and no rate.

Cells with a variance cap.

- If `v_max_user` is given at a G, W or RF (Ca, Be) cell, then $V^{\text{user}}_{\max,i\ell} > 0$ and $V^{\text{user}}_{\max,i\ell} \ge v_{0,i\ell}$. — A cap below the initial variance makes the problem infeasible.

Rainflow cells.

- Bernstein: $v^{\text{inc}}_{i\ell jk} \le (b^{\text{inc}}_{i\ell jk})^2/4$ for every mission j and day k, $v_0 \le (b^0)^2/4$ and $v^{\text{new}} \le (b^{\text{new}})^2/4$. — Popoviciu's inequality (below): a law on a range of width w has variance at most $w^2/4$, so a larger variance contradicts the width.
- Bernstein: $\tau_{i\ell} > 2\kappa\, b_{i\ell}/3$, with $\kappa = \ln(1/\varepsilon_{i\ell})$ and $b_{i\ell}$ the support constant of Section 6.11. — Otherwise the certificate admits no positive variance: it is infeasible at zero damage, and at equality only $\mu = v = 0$ survives, so that the big-M rows break.
- Bernstein with the form `tangent`: `tangent_ref`$\cdot\tau_{i\ell} < \tau_{i\ell} - c_{i\ell}$, with $c_{i\ell} = \kappa\, b_{i\ell}/3$. — Otherwise no point with $v \ge 0$ satisfies the tangent row, so the row is empty (Section 6.11).

- Chernoff: `s_chernoff` is given and $s_{i\ell} > \ln(1/\varepsilon_{i\ell})/\tau_{i\ell}$. — The quantity K is not negative, so the row $K - s\tau \le \ln \varepsilon$ needs $s\tau \ge \ln(1/\varepsilon)$; with a smaller s every schedule is infeasible, and at equality only a component without damage is feasible.
- Chernoff: $K^{\text{inc}}_{i\ell jk} \ge s_{i\ell}\, \mu^{\text{inc}}_{i\ell jk}$ for every mission j and day k, $K_0 \ge s\,\mu_0$ and $K^{\text{new}} \ge s\,\mu^{\text{new}}$. — Jensen's inequality: the cumulant generating function of a random variable at $s > 0$ is at least s times its mean.

Run options.

- `objective_mode` is `total`, `operating_average` or `evaluation_total`. In the last mode, `evaluation_horizon` is an integer $E > H_1$. — The projected cost needs at least one operating day beyond the transitory phase.
- `mip_gap` ≥ 0; `time_limit`, `progress_interval_seconds` and `relaxation_time_limit` are > 0; `gurobi_params` is a mapping; `fast`, `relaxation_warm_start`, `allow_replacement` and `replacement_as_new` are flags. — A value outside its range must raise an error and must not be changed silently.

Remark (Popoviciu's inequality). Let X take values in an interval of width w, with midpoint c. Then $\operatorname{Var} X \le \mathbb{E}\,(X - c)^2 \le (w/2)^2$. The width is the only information on the range, and the range may start at any point $a \ge 0$. The inequality $\operatorname{Var} X \le w^2/4$ holds for every such position, so it is a necessary condition for the pair (variance, width). For example, increments in an interval of width 0.2 have variance at most 0.01; a two-point law on $\{0, 0.2\}$ with probability $1/2$ each has mean 0.1 and reaches 0.01. A condition of the form $v \le \mu\,(b - \mu)$ assumes that the range starts at 0, so it is not necessary for a width.

Decision Variables

A variable, and the rows that define it, are built only where a row or an objective term with a non-zero coefficient uses them. Example: u and its rows are not built when $C_D = 0$; $z_{i\ell k}$ and its rows are not built at a cell with $C_{R,i\ell} = 0$. The day index runs over $k = 1, \dots, T$; index $k = 0$ holds constants (initial values and starting anchors), not variables.

Variable	Shape, type	Built when	Meaning
x_{ijk}	$F \times (M{+}1) \times T$, binary	always	1 if train i flies mission j on day k; $j = 0$: depot day
$x^m_{i\ell k}$	$F \times L \times T$, binary	always	1 if component ℓ of train i is repaired on day k
$x^r_{i\ell k}$	$F \times L \times T$, binary	`allow_replacement = true`	1 if component ℓ of train i is replaced on day k
$n_{i\ell k}$	binary	cells with `encoding = "indicator"`	1 on a day without intervention, Eq. (16)
$\mu_{i\ell k}$	continuous, $[0, \tau_{i\ell}]$	every cell	tracked mean degradation
$v_{i\ell k}$	continuous, $[0, M_v]$	cells whose model or bound tracks a variance	tracked variance
$R_{i\ell k}$	continuous, $[0, M_R]$	rainflow cells with the Hoeffding bound	tracked Hoeffding budget
$K_{i\ell k}$	continuous, $[0, M_K]$	rainflow cells with the Chernoff bound	tracked Chernoff quantity
$g^{\mu}_{i\ell k}$, $g^{v}_{i\ell k}$, $g^{R}_{i\ell k}$, $g^{K}_{i\ell k}$	continuous, range of the quantity	ARD_1 cells, one per tracked quantity except the Wiener variance	anchors, Eq. (11)
$w_{i\ell kp}$	binary, $p = 1, \dots, P$	cells with a quadratic row in the `pwl` form, $P \ge 2$	segment of the mean, Eq. (21)

Variable	Shape, type	Built when	Meaning
$z_{i\ell k}$	continuous, ≥ 0	cells with $C_{R,i\ell} > 0$, on days whose cost enters the objective	degradation removed by a repair, Section 6.12
u	scalar, ≥ 0	$C_D > 0$	damage regularisation, Section 6.13

The ranges M_v, M_R, M_K are the natural ranges of Section 6.4; each model section states which quantities it tracks. The output reports the values of the tracked quantities; they are upper bounds on the reference values (Section 6.4).

Objective Function

The cost of day k is

$$c_k = C_M \sum_{i=1}^{F} x_{i0k} + \sum_{i=1}^{F} \sum_{\ell=1}^{L} C_{R,i\ell}\, z_{i\ell k} + \sum_{i=1}^{F} \sum_{\ell=1}^{L} C_{\mathrm{rep},i\ell}\, x^{r}_{i\ell k}, \tag{2}$$

where $z_{i\ell k}$ reads 0 where it is not built, and the last sum is absent when `allow_replacement = false`. The costs of the two phases are

$$J_{\mathrm{init}} = \sum_{k=1}^{H_1} c_k, \qquad J_{\mathrm{op}} = \sum_{k=H_1+1}^{T} c_k. \tag{3}$$

The input `objective_mode` (default `total`) selects the objective:

$$\min \begin{cases} J_{\mathrm{init}} + J_{\mathrm{op}} + C_D\, u & \texttt{total}, \\ J_{\mathrm{op}}/H_2 + C_D\, u & \texttt{operating_average}, \\ J_{\mathrm{init}} + \dfrac{E - H_1}{H_2}\, J_{\mathrm{op}} + C_D\, u & \texttt{evaluation_total}, \end{cases} \tag{4}$$

where E is the input `evaluation_horizon`, required in the mode `evaluation_total`, with $E > H_1$. The term $C_D\, u$ is present only when $C_D > 0$ (Section 6.13).

Cost terms.

- $C_M \geq 0$, a scalar, is paid once per train per depot day ($x_{i0k} = 1$), whatever the number of components repaired or replaced on that day. It prices the visit to the depot, for example the train out of service and the set-up. An idle day (Section 6.1) costs nothing, so the number of depot days is a costed decision.
- $C_{R,i\ell} \geq 0$ is paid per unit of mean degradation removed by a repair (Section 6.12).
- $C_{\mathrm{rep},i\ell} \geq 0$ is paid per replacement. The input is required; it has no default.
- $C_D \geq 0$ weights the damage penalty u (Section 6.13). The term is added once in every mode, because u is the maximum over the whole horizon.

The shapes of the cost inputs are given in Section 3.

Meaning of the modes.

- `total`: the cost of the T modelled days.
- `operating_average`: the cost per day of the operating phase. It is the long-run cost per day of the endless schedule (Section 6.14), in which the cost of the transitory phase spreads over infinitely many days. It compares solves with different H_2 (Section 9). The transitory costs do not enter the objective in this mode: the solver chooses the transitory schedule for feasibility only.

- `evaluation_total`: the projected cost over E days: the transitory phase once, then $(E - H_1)/H_2$ repetitions of the operating phase, a fraction of a repetition counted pro rata. It also prices the transitory phase, so it compares solves with different H_1.

Example: $H_1 = 5$, $H_2 = 10$, $E = 365$. Then `evaluation_total` weights the operating cost by 36, and `operating_average` by 1/10.

The objective value is computed from the tracked values. It is therefore an upper bound on the cost of the schedule computed from the reference values (Section 6.4).

Constraints

The constraints have three groups. The fleet rows (assignment and depot days) do not depend on the degradation models. The common rules of the dynamics (repair laws, encoding, replacement, linear forms of the reliability rows) hold for every model; the five model sections apply them and add each model's tracked quantities, day maps and reliability row. The repair cost, the damage regularisation and the loop constraint close the problem. All rows hold for every day $k = 1, \dots, T$ unless a section says otherwise.

Assignment Constraints

Each train performs at most one activity per day:

$$\sum_{j=0}^{M} x_{ijk} \le 1 \qquad \forall\, i, k. \tag{5}$$

Each mission is flown by exactly one train every day:

$$\sum_{i=1}^{F} x_{ijk} = 1 \qquad \forall\, j \in \{1, \dots, M\},\ k. \tag{6}$$

Together these rows put exactly M trains on missions every day. The other $F - M$ trains are in the depot or idle. No other fleet-availability row is needed.

Idle days. A train is idle on day k when its assignment row is all zeros: $x_{ijk} = 0$ for every $j \in \{0, 1, \dots, M\}$. An idle train flies no mission and is not in the depot. Idle days and depot days add no increment, because every day aggregate $\delta\mu_{i\ell k}$, $\delta v_{i\ell k}$, $\delta R_{i\ell k}$, $\delta K_{i\ell k}$ sums over missions $j \ge 1$ only. A depot day must be chosen explicitly ($x_{i0k} = 1$), and repairs and replacements need one (Section 6.2). Because Eq. (5) is an inequality, the choice between a depot day and an idle day is free, and the term $C_M \sum x_{i0k}$ of the objective prices it.

Depot-Day Compatibility

A component may be repaired or replaced only on a depot day of its train, and at most one of the two on the same day:

$$x^m_{i\ell k} + x^r_{i\ell k} \le x_{i0k} \qquad \forall\, i, \ell, k. \tag{7}$$

With `allow_replacement = false` the row reads $x^m_{i\ell k} \le x_{i0k}$. Several components of one train may receive interventions on the same depot day. A depot day without an intervention on a component leaves that component unchanged.

Remark (one row instead of three). Eq. (7) has the same binary solutions as the three rows $x^m \le x_{i0k}$, $x^r \le x_{i0k}$, $x^m + x^r \le 1$. Its linear relaxation is tighter: the point $x_{i0k} = x^m = x^r = 0.5$ satisfies the three rows but not Eq. (7). With $0 \le x^m, x^r$ and $x_{i0k} \le 1$, Eq. (7) describes the convex hull of the four binary solutions $(x_{i0k}, x^m, x^r) \in \{(0,0,0), (1,0,0), (1,1,0), (1,0,1)\}$.

Repair Laws

An imperfect repair (in short, a repair) of a component on a depot day removes part of its degradation. The repair efficiency $\rho_{i\ell} \in (0, 1]$ is the fraction removed. Let D^- be the degradation level before the repair (at the end of day $k-1$; a depot day adds no increment) and D^+ the level after it (day k). The input `repair_model` selects one of two laws per cell; the default is `ardinf`.

Definition 6.1 (Repair laws)**.** Let $\rho = \rho_{i\ell}$. The two repair laws are:

- ARD_∞ (input value `ardinf`) removes the fraction ρ of the whole level:

$$D^+ = (1-\rho)\, D^-. \tag{8}$$

- ARD_1 (input value `ard1`) removes the fraction ρ of the degradation accumulated since the last intervention (repair or replacement):

$$D^+ = D^- - \rho\,\big(D^- - D^{\mathrm{last}}\big), \tag{9}$$

  where D^{last} is the level just after the last intervention. Before the first intervention of the horizon, D^{last} is the level just after the last intervention before the horizon.

Names and sources. ARD_1, "arithmetic reduction of deterioration of order 1", is the law of [2, §3.1, eq. (4)]. That source measures the removed degradation from the level just after the last maintenance, as Eq. (9) does. The name ARD_∞ is an analogy, not a citation. Doyen and Gaudoin define the classes ARA_∞ [1, eq. (20), p. 11] and ARA_1 [1, eq. (22), p. 12], which reduce a virtual age, not a degradation level. ARA_∞ reduces the whole virtual age; ARA_1 reduces only the age accumulated since the previous repair. The subscript is the memory of the class: the largest number of previous failure times that enter the failure intensity [1, p. 8]. ARD_∞ applies the rule of ARA_∞ to the whole degradation level. No law of this specification acts on a virtual age.

Repair maps of the tracked quantities. A model tracks moments or other summaries of D, not D itself. For a tracked quantity s with anchor g, a repair on day k applies the repair map

$$\psi(s_{k-1}, g_{k-1}) = \begin{cases} a\, s_{k-1} & \mathrm{ARD}_\infty, \\ a\, s_{k-1} + (1-a)\, g_{k-1} & \mathrm{ARD}_1, \end{cases} \tag{10}$$

with a repair factor $a \in [0, 1]$. For the mean, $a = 1 - \rho_{i\ell}$, because expectation is linear: Eq. (9) gives $\mathbb{E}[D^+] = (1-\rho)\,\mathbb{E}[D^-] + \rho\,\mathbb{E}[D^{\mathrm{last}}]$. For a variance and for R the factor is $(1-\rho_{i\ell})^2$, and for K it is $1-\rho_{i\ell}$; the model sections derive each factor. The Wiener variance has the factor $a = 1$ under both laws: a repair acts on the Wiener mean only and leaves the variance unchanged.

Anchors. Under ARD_1 each tracked quantity has an anchor g: the value of the quantity just after the last intervention. The Wiener variance is the exception: with $a = 1$ its map does not use an anchor, and none is built. The anchor recursion is

$$g_0 = \text{starting anchor}, \qquad g_k = \begin{cases} s_k & \text{on a repair or replacement day}, \\ g_{k-1} & \text{otherwise}, \end{cases} \tag{11}$$

built by the rows Eq. (15) or Eq. (18). The starting anchors $g^{\mu}_{i\ell 0}$, $g^{v}_{i\ell 0}$, $g^{R}_{i\ell 0}$, $g^{K}_{i\ell 0}$ come from the optional inputs `anchor_mu_0`, `anchor_v_0`, `anchor_R_0`, `anchor_K_0`. The default of each is the initial value of its quantity. Each must satisfy $0 \le g_0 \le s_0$ (Section 3.12). Example: $\mu_{k-1} = 0.6$, $g_{k-1} = 0.2$, $\rho = 0.5$. A repair gives $\mu_k = 0.6 - 0.5\,(0.6 - 0.2) = 0.4$ and $g_k = 0.4$. On the next day the gap $\mu - g$ is 0, and it grows again with each mission.

Remark (default of the starting anchor). Damage only grows between interventions. So the true level after the last intervention before the horizon lies between the level of a new part and the

initial level. A larger anchor gives a weaker repair and a higher state, because ψ is non-decreasing in g. The default $g_0 = s_0$ therefore never overstates what a repair achieves; the input gives the exact value when the maintenance history is known. Example: $\mu_0 = 0.5$, $\rho = 0.5$, and a repair on day 1 before any mission. With the starting anchor 0 the mean drops to 0.25; with the default μ_0 it stays at 0.5, and the repair removes only damage accumulated within the horizon.

Laws per model. Every model accepts both laws. Table 14 gives the reason for each model.

Table 14: Repair laws accepted by each model.

Model	ARD_∞	ARD_1	Reason or note
Gaussian	yes	yes	both laws keep the state Gaussian, with exact moments
Inverse Gaussian	yes	yes	the cap holds for any repair weights and any dependence between the terms (Section 6.8)
Wiener	yes	yes	on the mean only; the variance changes only by replacement
Gamma	yes	yes	Section 6.10
Rainflow, every bound	yes	yes	Chernoff under ARD_1 uses a conservative recursion for K (Section 6.11)

Encoding of the Dynamics

Each day, every tracked quantity of a cell follows one rule: a mission adds an increment, a repair applies the repair map of Section 6.3, and a replacement resets the quantity (Section 6.5). This subsection states these rules once, for every model and both repair laws. The model sections give, for each tracked quantity, its increment, its repair factor and its natural range.

One-sided rows. Every dynamics row is an inequality ($\geq$), not an equality. The solver may choose a value above the result of the rule, never a value below it. So a tracked quantity is an upper bound on the value that the rule gives. Example: with $\mu_{i\ell,k-1} = 0.3$ and a mission increment of 0.2, the rule gives 0.5; the rows accept $\mu_{i\ell k} = 0.5$ and also $\mu_{i\ell k} = 0.6$. Replacement rows and anchor updates are equalities.

Generic rows. Let s be one tracked quantity of a cell (a mean, a variance, R or K), with day increment δs_k, natural range M_s, replacement value s^{new}, anchor g (law ARD_1 only) and repair map ψ (Section 6.3). Write x^m_k, x^r_k for the binaries of the cell and $e_k = x^m_k + x^r_k$. The input `encoding` selects one of two forms per cell. Both forms are built for $k = 1, \ldots, T$; index $k = 0$ holds the initial value and the starting anchor.

Big-M form (`encoding = "bigm"`, the default):

$$\text{no intervention:} \quad s_k \geq s_{k-1} + \delta s_k - M_s\,(x^m_k + x^r_k), \tag{12}$$

$$\text{repair:} \quad s_k \geq \psi(s_{k-1}, g_{k-1}) - M_s\,(1 - x^m_k), \tag{13}$$

$$\text{replacement:} \quad |s_k - s^{\text{new}}| \leq M_s\,(1 - x^r_k), \tag{14}$$

$$\text{anchor } (\text{ARD}_1): \quad |g_k - s_k| \leq M_s\,(1 - e_k), \qquad |g_k - g_{k-1}| \leq M_s\,e_k. \tag{15}$$

Each row with an absolute value is built as two linear rows.

Indicator form (`encoding = "indicator"`). One extra binary $n_{i\ell k}$ per cell and day marks a day without intervention:

$$n_{i\ell k} + x^m_{i\ell k} + x^r_{i\ell k} = 1. \tag{16}$$

The rows are Gurobi indicator constraints:

$$n_k = 1 \;\Rightarrow\; s_k \geq s_{k-1} + \delta s_k, \qquad x^m_k = 1 \;\Rightarrow\; s_k \geq \psi(s_{k-1}, g_{k-1}), \qquad x^r_k = 1 \;\Rightarrow\; s_k = s^{\text{new}}, \tag{17}$$

$$n_k = 1 \;\Rightarrow\; g_k = g_{k-1}, \qquad x^m_k = 1 \;\Rightarrow\; g_k = s_k, \qquad x^r_k = 1 \;\Rightarrow\; g_k = s_k. \tag{18}$$

The binary $n_{i\ell k}$ is built only at cells in this form.

Common rules for both forms.

- The anchor rows are built only under ARD_1, one anchor per tracked quantity, except the Wiener variance.
- The Wiener variance takes the repair map $\psi = s_{k-1}$ (factor $a = 1$) under both laws, and has no anchor. A repair day adds no increment, so in the big-M form its no-intervention and repair rows together equal the single row $s_k \geq s_{k-1} + \delta s_k - M_s\, x^r_k$ (Eq. (30)), which may replace them. The variance is never left free below on a repair day.
- With `allow_replacement = false` the binaries x^r are not built: read $x^r_k = 0$ in every row, and drop the replacement rows.
- At binary points the two forms have the same feasible set. Every statement below holds for both.

Natural ranges. The big-M constant M_s is the natural range of s: an upper bound of s at every point that satisfies the reliability row of the cell. It is $\tau_{i\ell}$ for a mean and for its anchor, $V_{\max}$ for a Gaussian or Wiener variance, V^{C} or V^{B} for a rainflow variance (Cantelli or Bernstein), $2\tau^2/\kappa$ for R and $s_{i\ell}\tau - \kappa$ for K (Section 6.11). An anchor has the range of its quantity.

Remark (validity of the big-M constants). Every tracked quantity and every anchor lies in $[0, M_s]$. The initial values, the starting anchors and the replacement values lie in $[0, M_s]$ too, by the input checks of Section 3.12. An intervention day is a depot day (Section 6.2), so $\delta s_k = 0$ on it. A relaxed row then cuts no point: Eq. (12) with $e_k = 1$ reads $s_k \geq s_{k-1} - M_s$, whose right-hand side is at most 0; Eq. (13) with $x^m_k = 0$ has a right-hand side $\psi - M_s \leq 0$, because ψ is a convex combination of values in $[0, M_s]$; Eq. (14) with $x^r_k = 0$ allows $s_k \in [s^{\mathrm{new}} - M_s,\ s^{\mathrm{new}} + M_s] \supseteq [0, M_s]$; the relaxed anchor rows allow every difference in $[-M_s, M_s]$.

Remark (choice of the default form). Gurobi accepts a binary as integral within its integrality tolerance `IntFeasTol` (default 10^{-5}). A big-M row can therefore leak by at most $10^{-5} M_s$. Because M_s is the quantity's own range, this is a relative error of 10^{-5}. Indicator constraints have no such leak, and Gurobi usually needs more time to solve them. Which form solves this model faster has not been measured.

Safety of the one-sided rows

Fix a schedule, that is, values of every binary, and one cell. The following objects are used here and in Section 6.14.

- The *full tracked state* ξ_k collects the values on day k of every tracked quantity of the cell and of every anchor. ξ_0 holds the initial values and the starting anchors.
- The *day map* φ_k gives, for each coordinate, the value that the rule of day k assigns: $s + \delta s_k$ and g on a day without intervention; $\psi(s, g)$ for both s and g on a repair day; s^{new} for both on a replacement day.
- The *reference trajectory* is $\zeta_0 = \xi_0$, $\zeta_k = \varphi_k(\zeta_{k-1})$: the rows with every inequality replaced by an equality.
- The *true statistics* θ_k are the true values on day k of the tracked quantities (the mean, the variance, R or the CGF of the damage $D_{i\ell k}$) and of their anchors (their values just after the last intervention).
- The *certificate set* $\mathcal{C}_{i\ell}$ is the set of values of the tracked quantities at which the reliability row of the cell holds in its exact form, side rows included. Anchors do not enter it.

Every model section states three facts, labelled (M1)–(M3) there:

(M1) Each repair factor a lies in $[0, 1]$ and each increment δs_k is non-negative.

(M2) On day 0 the true statistics are at most the initial values and starting anchors, $\theta_0 \le \xi_0$. On every later day the true statistic is at most the day map applied to the true statistics of the day before: $\theta_k \le \varphi_k(\theta_{k-1})$. Every map holds with equality except the map of the Chernoff quantity K, which is an upper bound (Section 6.11).

(M3) If values $s \in \mathcal{C}_{i\ell}$ are at least the true statistics of day k, then $\Pr(D_{i\ell k} > \tau_{i\ell}) \le \varepsilon_{i\ell}$.

The set $\mathcal{C}_{i\ell}$ is closed, and it is a down-set: if $s \in \mathcal{C}_{i\ell}$ and $0 \le s' \le s$ componentwise, then $s' \in \mathcal{C}_{i\ell}$. The reason is that every reliability row, written in chance-bound form, reads "a continuous function of the tracked quantities, non-decreasing in each of them, is at most a constant":

Model or bound	Row in chance-bound form	Section
Gaussian, Wiener	$\mu + \Phi^{-1}(1-\varepsilon)\sqrt{v} \le \tau$	6.7, 6.9
Inverse Gaussian; Gamma	$\mu \le \bar{\mu}$; $\mu \le A_1(\beta^*\tau, \varepsilon)/\beta^*$	6.8, 6.10
Rainflow, Markov	$\mu \le \varepsilon\,\tau$	6.11
Rainflow, Cantelli	$\mu + \kappa_C\sqrt{v} \le \tau$	6.11
Rainflow, Hoeffding	$\mu + \sqrt{\kappa R/2} \le \tau$	6.11
Rainflow, Bernstein	$\mu + c + \sqrt{c^2 + 2\kappa v} \le \tau$	6.11
Rainflow, Chernoff	$K \le s_{i\ell}\,\tau + \ln\varepsilon$	6.11

Lemma 6.2 (Monotone day maps)**.** *Each day map φ_k is non-decreasing in each coordinate of the full tracked state, anchors included.*

Proof. Let $\xi \le \xi'$ componentwise. Each coordinate of φ_k has one of the forms $s + \delta s_k$, g, s, $a\,s$, $a\,s + (1-a)\,g$ or s^{new}. By (M1), $a \ge 0$ and $1 - a \ge 0$. Therefore each coordinate of $\varphi_k(\xi)$ is at most the same coordinate of $\varphi_k(\xi')$. □

Proposition 6.3 (Domination)**.** *Every feasible solution satisfies $\xi_k \ge \varphi_k(\xi_{k-1})$ and $\theta_k \le \zeta_k \le \xi_k$ for $k = 1, \ldots, T$.*

Proof. At binary points the rows of day k read: $s_k \ge \varphi_k(\xi_{k-1})_s$ for every tracked quantity s, with equality on a replacement day; $g_k = s_k \ge \varphi_k(\xi_{k-1})_g$ on an intervention day; $g_k = g_{k-1} = \varphi_k(\xi_{k-1})_g$ on any other day. So $\xi_k \ge \varphi_k(\xi_{k-1})$. Let $\theta_{k-1} \le \zeta_{k-1} \le \xi_{k-1}$ (true for $k = 1$ by (M2), since $\zeta_0 = \xi_0$). Then, by (M2) and Lemma 6.2,

$$\theta_k \le \varphi_k(\theta_{k-1}) \le \varphi_k(\zeta_{k-1}) = \zeta_k \le \varphi_k(\xi_{k-1}) \le \xi_k.$$

Therefore $\theta_k \le \zeta_k \le \xi_k$ for every k by induction. □

Consequence on the horizon. On every day $k = 1, \ldots, T$ the tracked values satisfy the reliability row in the form chosen for the cell. Every form lies inside $\mathcal{C}_{i\ell}$ (Section 6.6), so $\xi_k \in \mathcal{C}_{i\ell}$. By Proposition 6.3, $\theta_k \le \xi_k$, so (M3) gives $\Pr(D_{i\ell k} > \tau_{i\ell}) \le \varepsilon_{i\ell}$. Theorem 6.17 (Section 6.14) extends this to every day of every later repetition of the operating phase.

What the one-sided rows accept. Call a schedule *safe* for a cell when the reference trajectory of the endless schedule (the transitory phase once, then the operating phase repeated without end, Section 6.14) lies in $\mathcal{C}_{i\ell}$ and in the natural ranges on every day. The one-sided rows relate to this property as follows.

- *They never accept an unsafe schedule* (Theorem 6.17).
- *They accept every schedule that equality rows accept*, and more. Equality rows force $\xi_k = \zeta_k$; Proposition 6.4 (a) shows that such a schedule satisfies the one-sided rows too. The first example below shows a safe schedule that only the one-sided rows accept.
- *They may reject a safe schedule*, as equality rows may, but only when tracked quantities move in opposite directions over the first repetition (Proposition 6.4 (b) and the second example below).

Proposition 6.4 (Acceptance)**.** *Fix a schedule and a cell, use the exact reliability form, and let the schedule be safe for the cell. Let* $\bar{\zeta} = \zeta_{H_1}$ *and let* Φ *be the composition of the day maps of the operating phase (Section 6.14).*

(a) If $\Phi(\bar{\zeta}) \le \bar{\zeta}$*, the rows of the cell, the loop row included, accept the schedule with* $\xi_k = \zeta_k$ *for every* k*.*
(b) If $\Phi(\bar{\zeta}) \ge \bar{\zeta}$*, the cell has no anchor (law* ARD_∞*), and the cell is not replaced on day* H_1*, the rows of the cell accept the schedule.*

Proof. (a) The values $\xi_k = \zeta_k$ satisfy every row of day k with equality at binary points. They lie in $\mathcal{C}_{i\ell}$ because the schedule is safe. The loop row reads $\zeta_T = \Phi(\bar{\zeta}) \le \bar{\zeta} = \zeta_{H_1}$.

(b) Let $y_n = \Phi^n(\bar{\zeta})$, the reference state at the end of the n-th repetition. By Lemma 6.2 and $\Phi(\bar{\zeta}) \ge \bar{\zeta}$, $y_{n+1} = \Phi(y_n) \ge \Phi(y_{n-1}) = y_n$. The sequence is non-decreasing and lies in the natural ranges, so it converges to some y^* with $y^* \ge \bar{\zeta}$. Each day map is affine or constant, so Φ and its prefixes Φ_h are continuous: $\Phi(y^*) = \lim_n y_{n+1} = y^*$ and $\Phi_h(y^*) = \lim_n \Phi_h(y_n)$. Each $\Phi_h(y_n)$ is a reference value of a later day, so it lies in the closed set $\mathcal{C}_{i\ell}$; hence $\Phi_h(y^*) \in \mathcal{C}_{i\ell}$ for $h = 1, \dots, H_2$, and $y^* = \Phi_{H_2}(y^*) \in \mathcal{C}_{i\ell}$. Take $\xi_k = \zeta_k$ for $k < H_1$, $\xi_{H_1} = y^*$ and $\xi_{H_1+h} = \Phi_h(y^*)$. Day H_1 is not a replacement day and the cell has no anchor, so $\xi_{H_1} = y^* \ge \varphi_{H_1}(\xi_{H_1-1})$ satisfies the rows of day H_1. The later days satisfy their rows with equality, and the loop row reads $\xi_T = \Phi(y^*) = y^* = \xi_{H_1}$. □

A scalar state is always comparable with its image. So a cell with one tracked quantity under ARD_∞, not replaced on day H_1, accepts every safe schedule. Under ARD_1 the anchor is a second tracked quantity, and its value on day H_1 is fixed by the anchor rows; part (b) does not apply, and acceptance of every safe schedule is not proven.

Example (one quantity, accepted only by the one-sided rows). One tracked mean, law ARD_∞, $\rho = 0.5$. The operating phase is a mission ($\delta\mu = 0.2$) followed by a repair, so $\Phi(y) = 0.5\,(y + 0.2) = 0.5\,y + 0.1$, and $\Phi(y) \le y$ if and only if $y \ge 0.2$. Let $\bar{\zeta} = 0.1$. Then $\Phi(0.1) = 0.15 > 0.1$, so equality rows reject the schedule. The one-sided rows accept it with $\xi_{H_1} = 0.2$: the tracked cycle is $0.2, 0.4, 0.2$, and the reliability row is checked at 0.4. The reference trajectory is $0.1, 0.3, 0.15, 0.35, 0.175, \dots$; its start-of-cycle values rise to 0.2 and its peaks rise to 0.4. So the schedule is safe whenever $0.4 \in \mathcal{C}_{i\ell}$. The price is in the reported cost: each repair is charged $\rho \cdot 0.4 = 0.2$, while the reference removal rises from 0.15 toward 0.2.

Example (two quantities that move in opposite directions, rejected). Cantelli bound, $\varepsilon = 0.1$ ($\kappa_C^2 = 9$), $\tau = 1$, $\rho = 0.5$, law ARD_∞. The operating phase is a mission, which adds 0.2 to the mean and 0.009 to the variance, followed by a repair. So $\Phi(\mu, v) = (0.5\,\mu + 0.1,\ 0.25\,v + 0.00225)$, with fixed point $(0.2, 0.003)$. Let $\bar{\zeta} = (0.5, 0)$.

- Reference: over the repetitions the mean falls to 0.2 and the variance rises to 0.003. The first peak is $(0.7, 0.009)$, with $9 \cdot 0.009 = 0.081 \le 0.3^2 = 0.09$. Every later peak has a mean of at most 0.55 and a variance below 0.012, with $9 \cdot 0.012 = 0.108 \le 0.45^2 = 0.2025$. The schedule is safe.
- One-sided rows: the loop row needs $\xi_{H_1} = y \ge (0.5, 0)$ with $\Phi(y) \le y$, so $y_v \ge 0.003$. The tracked peak is then at least $(0.7, 0.012)$, where $9 \cdot 0.012 = 0.108 > 0.09$. Rejected.
- Equality rows: $\Phi(\bar{\zeta}) = (0.35, 0.00225)$ has a larger variance than $\bar{\zeta}$. Rejected as well.

Meaning of the reported values. The output reports the tracked values ξ_k, not the reference values. By Proposition 6.3 each reported state and anchor is an upper bound on its reference value, and so on its true statistic. The costs computed from them are upper bounds too: the repair cost (Proposition 6.16) and the damage penalty (Section 6.13).

Remark (slack in the reported values). The minimisation removes slack only where slack costs something: a larger mean raises the damage penalty $C_D\,u$ where u is binding, and raises the repair cost $C_R\,z$ of every later repair. A larger variance, R or K tightens the reliability row and so restricts the schedule. Elsewhere the solver may return any value above the reference value. With $C_D = 0$ the damage penalty is absent (Section 6.13), and the reported states may carry more slack. Slack never affects safety (Theorem 6.17).

Replacement

A replacement installs a new part on a depot day. The input `allow_replacement` (default `true`) makes replacements available. When it is `false`, the binaries $x^r_{i\ell k}$, the replacement rows and the replacement cost are not built.

State after a replacement. On a replacement day every tracked quantity of the cell takes its replacement value, and every anchor takes the same value (Eq. (11)). The values come from the input, for every model:

- mean $\mu^{\text{new}}_{i\ell}$, input `replacement_mu`, default 0;
- variance $v^{\text{new}}_{i\ell}$, input `replacement_v`, default 0, where the model tracks a variance;
- $R^{\text{new}}_{i\ell}$ and $K^{\text{new}}_{i\ell}$ for the rainflow Hoeffding and Chernoff bounds, with their defaults and their derivation in Section 6.11.

A replacement value describes the damage of the new part: a known damage when its variance is 0, a random one otherwise. The rows are the equalities Eq. (14) or the indicator rows Eq. (17). The input checks of Section 3.12 require each replacement value to lie in the natural range of its quantity, and the replacement state to satisfy the reliability row of the cell; otherwise no replacement of that cell would be feasible.

Shortcut for a new part. The input `replacement_as_new` (default `false`) states that a replaced part is new, with no damage: every replacement value of the cell is 0. A non-zero replacement input together with `replacement_as_new = true` is an input error (Section 3.12).

Cost. A replacement costs $C_{\text{rep},i\ell}$ and nothing else: the repair cost $z_{i\ell k}$ is 0 on a replacement day (Section 6.12). A cell may be replaced on any depot day of its train, any number of times. The loop row can require at least one replacement in the operating phase: for every ARD_1 cell with $\rho < 1$ that accumulates damage (Proposition 6.18), and for the Wiener variance (Section 6.9).

Linear Forms of the Reliability Rows

Five reliability rows are quadratic: the Gaussian and Wiener rows, and the rainflow Cantelli, Hoeffding and Bernstein rows. Each has the form

$$\chi_{i\ell k} \;\le\; f(\mu_{i\ell k}), \qquad f(\mu) = c_2\,(\tau-\mu)^2 + c_1\,(\tau-\mu), \qquad \mu_{i\ell k} \le \bar{\mu}_{\text{side}}, \tag{19}$$

with the constants of Table 16. The third relation is the side row; it selects the branch of the parabola on which the row is a valid certificate. Since $c_2 > 0$, f is convex in μ, with $f'(\mu) = -2c_2(\tau-\mu) - c_1$.

Table 16: Constants of the quadratic reliability rows, Eq. (19). The model sections derive each row.

Row	χ	c_2	c_1	$\bar{\mu}_{\text{side}}$
Gaussian, Wiener	v	$1/(\Phi^{-1}(1-\varepsilon))^2$	0	τ
Rainflow, Cantelli	v	$1/\kappa_C^2 = \varepsilon/(1-\varepsilon)$	0	τ
Rainflow, Hoeffding	R	$2/\kappa$	0	τ

Rainflow, Bernstein	v	$1/(2\kappa)$	$-b_{i\ell}/3$	$\tau - c_{i\ell}$

The input `reliability_impl` selects one of three forms per cell. Every form keeps the side row.

- `exact`: Eq. (19) as written. The feasible set $\{(\mu, \chi) : \chi \le f(\mu)\}$ lies below a convex function, so it is not convex: the row is a nonconvex quadratic constraint. Gurobi's parameter `NonConvex` is set to 2 whenever at least one cell has a quadratic row in this form.
- `tangent`: one linear row per day, the tangent of f at the point $\mu_r = \texttt{tangent_ref}_{i\ell} \cdot \tau_{i\ell}$:

$$\chi_{i\ell k} \;\le\; f(\mu_r) + f'(\mu_r)\,(\mu_{i\ell k} - \mu_r). \tag{20}$$

  The input `tangent_ref` is per cell, in $[0, 1)$, default 0.5. At `tangent_ref` $= 0$ the Gaussian row becomes $(\Phi^{-1}(1-\varepsilon))^2\, v + 2\tau\mu \le \tau^2$. The value 1 is excluded: for $c_1 = 0$ the tangent at τ is $\chi \le 0$. At a Bernstein cell the tangent point must also satisfy $\mu_r < \tau - c_{i\ell}$ (Section 3.12); otherwise no point with $v \ge 0$ satisfies the row (Section 6.11).
- `pwl`: a piecewise tangent with $P =$ `pwl_points` segments (default 8). The breakpoints split $[0, \tau]$ into equal segments $[(p-1)\tau/P,\ p\tau/P]$ with midpoints $\hat{\mu}_p = (p - \frac{1}{2})\tau/P$, $p = 1, \ldots, P$. One binary $w_{i\ell kp}$ per cell, day and segment selects the segment that contains the mean:

$$\textstyle\sum_{p=1}^{P} w_{i\ell kp} = 1, \tag{21}$$

$$w_{i\ell kp} = 1 \;\Rightarrow\; (p-1)\,\tau/P \le \mu_{i\ell k} \le p\,\tau/P, \tag{22}$$

$$w_{i\ell kp} = 1 \;\Rightarrow\; \chi_{i\ell k} \le f(\hat{\mu}_p) + f'(\hat{\mu}_p)\,(\mu_{i\ell k} - \hat{\mu}_p). \tag{23}$$

  The implications are Gurobi indicator constraints, whatever the value of `encoding`. With $P = 1$ the form is the single tangent at $\tau/2$, with no binaries.

At a cell whose reliability row is linear (Markov, Chernoff, inverse Gaussian, Gamma) the row is built in its exact form, and the value of `reliability_impl` is ignored. An unknown value of `reliability_impl` is an input error at every cell (Section 3.12).

Proposition 6.5 (Safety of the linear forms)**.** *For every μ_r and every μ, $f(\mu_r) + f'(\mu_r)\,(\mu - \mu_r) = f(\mu) - c_2\,(\mu - \mu_r)^2 \le f(\mu)$. Therefore every point that satisfies the `tangent` or the `pwl` form satisfies the `exact` form, and both linear forms lie inside the certificate set $\mathcal{C}_{i\ell}$.*

Proof. f is a polynomial of degree 2 in μ with leading coefficient c_2, so its Taylor expansion at μ_r is exact: $f(\mu) = f(\mu_r) + f'(\mu_r)(\mu - \mu_r) + c_2(\mu - \mu_r)^2$. Since $c_2 > 0$, the tangent lies below f. Each linear form caps χ by one such tangent and keeps the side row. Therefore it implies Eq. (19). □

Remark (accuracy). The `tangent` form loses $c_2(\mu - \mu_r)^2$ in χ: it is exact at $\mu = \mu_r$ and loosest far from it. A tangent point near the operating mean of the cell is the best choice. In the `pwl` form the loss is $c_2(\mu - \hat{\mu}_p)^2 \le c_2\,\tau^2/(4P^2)$. The midpoint of the segment that contains μ is the nearest tangent point to μ. So the `pwl` form accepts exactly the points with $\chi \le \max_p [f(\hat{\mu}_p) + f'(\hat{\mu}_p)(\mu - \hat{\mu}_p)]$, the largest set that the P tangents allow. The price is P binaries per cell and day.

Example (Cantelli, $\varepsilon = 0.1$, $\kappa_C^2 = 9$, $\tau = 1$, $v = 0.004$). The largest admissible mean is $1 - \sqrt{9 \cdot 0.004} \approx 0.810$ with `exact`; 0.482 with `tangent` at 0 ($9v + 2\mu \le 1$); 0.714 with `tangent` at $0.5\,\tau$ ($9v \le 0.75 - \mu$); and 0.810 with `pwl` and $P = 8$. The tangent at 0 never admits a mean above $\tau/2$, even with zero variance.

Gaussian Model

This section applies to the cells (i, ℓ) with `model` = `gaussian`. Inside it, the subscript $i\ell$ is dropped from the per-cell parameters and from the states; k is the day.

Model. A mission increment is Gaussian, $\mathcal{N}(\mu_{jk}^{\text{inc}}, v_{jk}^{\text{inc}})$ (inputs `mu` and `v`); the initial damage is $\mathcal{N}(\mu_0, v_0)$ and each replacement draw is $\mathcal{N}(\mu^{\text{new}}, v^{\text{new}})$. A known damage has variance 0.
Assumption N1 (independence). The increments, the initial damage and the replacement draws are mutually independent. A repair acts on the damage path by the law of the cell (Section 6.3). The Gaussian law gives negative damage a small probability; the input checks of Section 3.12 ($\mu \geq 3\sqrt{v}$ for the initial damage and for each increment) keep it at most $\Phi(-3) = 0.135\,\%$ for each of these terms.

Tracked states and repair laws. The cell tracks the mean μ_k and the variance v_k, and under ARD_1 their anchors g_k^{μ} and g_k^{v}. Both laws are accepted. The exact day maps are

$$\mu_k = \begin{cases} \mu_{k-1} + \delta\mu_k & \text{no intervention on day } k, \\ (1-\rho)\,\mu_{k-1} & \text{repair, } \text{ARD}_\infty, \\ (1-\rho)\,\mu_{k-1} + \rho\, g_{k-1}^{\mu} & \text{repair, } \text{ARD}_1, \\ \mu^{\text{new}} & \text{replacement}, \end{cases} \tag{24}$$

$$v_k = \begin{cases} v_{k-1} + \delta v_k & \text{no intervention on day } k, \\ (1-\rho)^2\, v_{k-1} & \text{repair, } \text{ARD}_\infty, \\ (1-\rho)^2\, v_{k-1} + \big(1-(1-\rho)^2\big)\, g_{k-1}^{v} & \text{repair, } \text{ARD}_1, \\ v^{\text{new}} & \text{replacement}, \end{cases} \tag{25}$$

with $\delta\mu_k = \sum_{j=1}^{M} x_{ijk}\,\mu_{jk}^{\text{inc}}$ and $\delta v_k = \sum_{j=1}^{M} x_{ijk}\,v_{jk}^{\text{inc}}$.
Proposition 6.6 (The state stays Gaussian)**.** *Under N1, on every day of a schedule $D_k \sim \mathcal{N}(m_k, V_k)$, where (m_k, V_k) follow the maps (24) and (25), with anchors equal to the mean and the variance just after the last intervention.*

Proof. A mission adds an independent Gaussian term, and ARD_∞ scales the damage by $1-\rho$; both keep the law Gaussian and give the first two cases. Under ARD_1, $D^- = G + \Delta$, with G the damage just after the last intervention and Δ the increments added since; G and Δ are independent Gaussian variables by N1. Then $D^+ = G + (1-\rho)\Delta$ is Gaussian, with mean $m_G + (1-\rho)(m^- - m_G)$ and variance

$$V_G + (1-\rho)^2\, V_\Delta = V_G + (1-\rho)^2\,(V^- - V_G) = (1-\rho)^2\, V^- + \big(1-(1-\rho)^2\big)\, V_G.$$

A replacement starts from the replacement draw. Therefore the law is Gaussian on every day, with the moments of the maps. □

So ARD_1 is exact for this model. Its variance map uses the factor $(1-\rho)^2$ and the variance anchor g^v. The rows are the generic rows of Section 6.4, with the data below; $V_{\max} = \tau^2/\kappa_G^2$, with $\kappa_G = \Phi^{-1}(1-\varepsilon)$.

State	Day increment	Factor a	Natural range	Initial value	Replacement value
μ; anchor g^{μ}	$\delta\mu_k$	$1-\rho$	τ	μ_0	μ^{new}
v; anchor g^{v}	δv_k	$(1-\rho)^2$	$V_{\max}$	v_0	v^{new}

Reliability row

Gaussian cells need $\varepsilon < 0.5$, so that $\kappa_G = \Phi^{-1}(1-\varepsilon) > 0$. For $D \sim \mathcal{N}(m, V)$, $\Pr(D > \tau) = 1 - \Phi\big((\tau - m)/\sqrt{V}\big) \le \varepsilon$ exactly when $m + \kappa_G\sqrt{V} \le \tau$. With $\kappa_G > 0$ and $\tau - \mu \ge 0$ this is equivalent to the squared row, imposed on every day $k = 1, \dots, T$:

$$\kappa_G^2\, v_k \le (\tau - \mu_k)^2, \qquad \mu_k \le \tau. \tag{26}$$

Validity. Let $(m_k, V_k) \le (\mu_k, v_k)$ be the true moments (Proposition 6.6 and Section 6.4). If $V_k = 0$, then $D_k = m_k \le \tau$. Otherwise $\mu_k < \tau$, because $\mu_k = \tau$ forces $v_k = 0$, and $\Pr(D_k > \tau) = 1 - \Phi\big((\tau - m_k)/\sqrt{V_k}\big) \le 1 - \Phi\big((\tau - \mu_k)/\sqrt{v_k}\big) \le \varepsilon$, because $(\tau - \mu)/\sqrt{v}$ decreases in μ and in v.

Forms. In the form `exact`, Eq. (26) is a *nonconvex* quadratic constraint: $(\mu, v) = (\tau, 0)$ and $(0, V_{\max})$ satisfy it, and their midpoint $(\tau/2, V_{\max}/2)$ does not, since $\tau^2/2 > \tau^2/4$. The solver handles it through Gurobi's nonconvex quadratic support (parameter `NonConvex` = 2). The row has the form $v_k \le f(\mu_k)$ with the convex function

$$f(\mu) = \frac{(\tau - \mu)^2}{\kappa_G^2}, \qquad \text{side row } \mu \le \tau,$$

so the forms `tangent` and `pwl` of Section 6.6 apply, and both keep the side row.

Remark (the level ε and the range $V_{\max}$). $V_{\max} = f(0) = \tau^2/\kappa_G^2$ is the largest variance that the row admits, at $\mu = 0$, so it is the natural range of v and the big-M constant of its rows; the input checks of Section 3.12 keep v_0, its starting anchor and v^{new} inside it. $V_{\max}/\tau^2$ is 0.18 at $\varepsilon = 0.01$, 0.61 at 0.1, 3.6 at 0.3 and about 1600 at 0.49. It grows without bound as $\varepsilon \to 0.5$, and at $\varepsilon \ge 0.5$ the squared row does not describe the reliability requirement, because $\kappa_G \le 0$. Hence the condition $\varepsilon < 0.5$ (Section 3.12); the range is large only close to 0.5.

Optional row. If `v_max_user` is given for the cell, the row $v_k \le V^{\text{user}}$ is added on every day.

Facts used by the encoding and loop arguments. (M1) The day maps are nondecreasing in (μ, v, g^μ, g^v): their coefficients lie in $[0, 1]$. (M2) The true moments follow the maps exactly (Proposition 6.6). (M3) If the tracked states satisfy Eq. (26) and the true moments are not larger, then $\Pr(D_k > \tau) \le \varepsilon$ (the validity paragraph). The loop row of Section 6.14 covers μ, v and, under ARD_1, g^μ and g^v.

Inverse Gaussian Model

This section applies to the cells (i, ℓ) with `model` = `inverse_gaussian` (IG). Inside it, the subscript $i\ell$ is dropped from the per-cell parameters and from the states; k is the day.

Model. $\text{IG}(\mu, \lambda)$ is the inverse Gaussian law with mean $\mu > 0$ and shape $\lambda > 0$. Its density is

$$f(x; \mu, \lambda) = \sqrt{\frac{\lambda}{2\pi x^3}}\, \exp\Big(-\frac{\lambda\,(x-\mu)^2}{2\mu^2 x}\Big), \qquad x > 0,$$

and its variance is μ^3/λ. For $c > 0$, $c\,\text{IG}(\mu, \lambda) = \text{IG}(c\mu, c\lambda)$. The cell has one dispersion $\eta > 0$ (input `eta`), and a term of mean a has the law $\text{IG}(a, a/\eta) = a\,Z$ with $Z \sim \text{IG}(1, 1/\eta)$. Its density is $\sqrt{a/(2\pi\eta x^3)}\, \exp\big(-(x-a)^2/(2\eta a x)\big)$, and η is its squared coefficient of variation. Every term has this law with its own mean: a mission increment (mean μ_{jk}^{inc}), the initial damage (μ_0) and each replacement draw (μ^{new}). A known damage a is the term $a \cdot 1$. The terms may depend on each other in any way: this model needs no independence assumption.

Tracked state and repair laws. The cell tracks the mean μ_k, and under ARD_1 its anchor g^{μ}_k. Both laws are accepted. The exact day map of the mean is

$$\mu_k = \begin{cases} \mu_{k-1} + \delta\mu_k & \text{no intervention on day } k, \\ (1-\rho)\,\mu_{k-1} & \text{repair, } \mathrm{ARD}_\infty, \\ (1-\rho)\,\mu_{k-1} + \rho\, g^{\mu}_{k-1} & \text{repair, } \mathrm{ARD}_1, \\ \mu^{\mathrm{new}} & \text{replacement}, \end{cases} \tag{27}$$

with $\delta\mu_k = \sum_{j=1}^{M} x_{ijk}\,\mu^{\mathrm{inc}}_{jk}$. The rows are the generic rows of Section 6.4, with the data below.

State	Day increment	Factor a	Natural range	Initial value	Replacement value
μ; anchor g^{μ}	$\delta\mu_k$	$1-\rho$	τ	μ_0	μ^{new}

Reliability row

Between replacements, the damage on day k is $D_k = \sum_t w_t X_t = \sum_t (w_t a_t)\, Z_t$, where $X_t = a_t Z_t$ are the terms added since the last replacement (or day 0), and each Z_t has the law of Z or equals 1. Each weight $w_t \in (0,1]$ is a product of factors $1-\rho$: ARD_∞ multiplies every weight by $1-\rho$, and ARD_1, with $D^+ = G + (1-\rho)\,\Delta$, multiplies only the weights of the terms added since the last intervention. When the starting anchor g^{μ}_0 is below μ_0, the initial damage $\mu_0 Z_0$ is read as the two terms $g^{\mu}_0 Z_0$ and $(\mu_0 - g^{\mu}_0) Z_0$. The mean $\sum_t w_t a_t$ is at most the tracked mean μ_k (Section 6.4).

Proposition 6.7 (Stop-loss order)**.** *Let $D = \sum_t w_t a_t Z_t$ with repair weights $w_t \in (0,1]$, means $a_t \geq 0$, $\sum_t w_t a_t \leq \mu$, and each Z_t either with the law of Z or equal to 1, under any joint law. Then $\mathbb{E}(D-d)_+ \leq \mathbb{E}(\mu Z - d)_+$ for every $d \geq 0$.*

Proof. Let $c_t = w_t a_t$ and $A = \sum_t c_t$. If $A = 0$, then $D = 0$. Otherwise, $x \mapsto (x-d)_+$ is convex and nondecreasing, $Z \geq 0$, and $(A-d)_+ \leq \mathbb{E}(AZ-d)_+$ by Jensen's inequality ($\mathbb{E}Z = 1$), so

$$\mathbb{E}(D-d)_+ = \mathbb{E}\Big(\sum_t \frac{c_t}{A}\,(AZ_t) - d\Big)_+ \leq \sum_t \frac{c_t}{A}\,\mathbb{E}(AZ_t - d)_+ \leq \mathbb{E}(AZ-d)_+ \leq \mathbb{E}(\mu Z - d)_+.$$

□

Let q be the $(1-\varepsilon)$-quantile of Z, $\Pr(Z > q) = \varepsilon$, and let $C = \mathbb{E}[Z \mid Z > q]$, the conditional value at risk (CVaR) of Z at the level $1-\varepsilon$. The cap and the row, on every day $k = 1, \ldots, T$, are

$$\bar{\mu} = \frac{\tau}{C}, \qquad \mu_k \leq \bar{\mu}. \tag{28}$$

Theorem 6.8 (CVaR cap)**.** *Let $D \geq 0$ satisfy $\mathbb{E}(D-d)_+ \leq \mathbb{E}(\mu Z - d)_+$ for every $d \geq 0$, with $0 \leq \mu \leq \bar{\mu}$. Then $\Pr(D > \tau) \leq \varepsilon$.*

Proof. If $\mu = 0$, then $\mathbb{E}D \leq 0$ and $D = 0$. Let $\mu > 0$ and $d = \mu q$. Since $C > q$, $d < \mu C \leq \tau$. For every x, $\mathbf{1}\{x > \tau\} \leq (x-d)_+/(\tau - d)$, and $\mathbb{E}(Z-q)_+ = \varepsilon\,(C-q)$. Then

$$\Pr(D > \tau) \leq \frac{\mathbb{E}(D-d)_+}{\tau - d} \leq \frac{\mu\,\mathbb{E}(Z-q)_+}{\tau - \mu q} = \frac{\varepsilon\,\mu\,(C-q)}{\tau - \mu q} \leq \varepsilon,$$

where the last step is $\mu C \leq \tau$. □

By Proposition 6.7 with $\mu = \mu_k$ and Theorem 6.8, the row (28) gives $\Pr(D_k > \tau) \le \varepsilon$ on every day, under both laws. The row is linear: it is built in its exact form, and a value of `reliability_impl` at the cell is ignored (Section 6.6).

Computation of $\bar{\mu}$ (offline, once per cell, before the model is built). With $r(x) = 1/\sqrt{\eta x}$, the distribution function of Z is

$$F(x) = \Phi\big(r(x)\,(x-1)\big) + e^{2/\eta}\,\Phi\big(-r(x)\,(x+1)\big).$$

The second term overflows for small η; it is computed as $\exp\big(2/\eta + \ln\Phi(-r(x)(x+1))\big)$, with $\ln\Phi$ from a log-space routine (`scipy.special.log_ndtr`).

1. q is the root of $1 - F(q) = \varepsilon$, found by bisection or by `brentq` on $\ln q$. The method `isf` of `scipy.stats.invgauss`, with `mu` $= \eta$ and `scale` $= 1/\eta$, gives the same value.
2. The tail mean has the closed form $\mathbb{E}[Z;\, Z > q] = F(1/q)$, so $\bar{\mu} = \tau\,\varepsilon/F(1/q)$. Derivation: substituting into the density gives $z\,f_Z(z) = z^{-2} f_Z(1/z)$, and the change of variable $u = 1/z$ gives $\int_q^\infty z\,f_Z(z)\,dz = \int_0^{1/q} f_Z(u)\,du = F(1/q)$.

Values of $\bar{\mu}/\tau$:

	$\eta = 0.05$	0.25	1	4	10
$\varepsilon = 0.001$	0.491	0.242	0.101	0.0370	0.0190
$\varepsilon = 0.01$	0.570	0.322	0.156	0.0679	0.0401
$\varepsilon = 0.1$	0.693	0.477	0.299	0.183	0.141

Remark (the largest cap for this order). Let $\mu > \bar{\mu}$. For a tail probability $\varepsilon' > \varepsilon$ close to ε, with $(1-\varepsilon')$-quantile q', still $\mu\,\mathbb{E}[Z \mid Z > q'] > \tau$. The two-point law $D = \mu\,\mathbb{E}\big[Z \mid \mathbf{1}\{Z > q'\}\big]$ is below μZ in the stop-loss order, by Jensen's inequality, and $\Pr(D > \tau) \ge \varepsilon'$. So no larger cap is safe for every law that Proposition 6.7 allows. That class also contains laws that no schedule produces, so the row can reject some safe schedules.

Remark (the quantile cap is not the row). The cap τ/q makes the model law μZ meet the target exactly: $\Pr(\mu Z > \tau) = \varepsilon$ at $\mu = \tau/q$. A sum of terms below μZ in the stop-loss order can have a larger tail. Example: $\eta = 10$, $\varepsilon = 0.1$, so $\tau/q = 0.473\,\tau$; three independent missions of equal mean, with total mean τ/q, give $\Pr(D > \tau) = 0.118$. The CVaR cap is $0.141\,\tau$ there. At $\varepsilon = 0.01$ and η from 0.05 to 10, $\bar{\mu}$ is 0.93 to 0.59 times τ/q.

Facts used by the encoding and loop arguments. (M1) The day map (27) is nondecreasing in (μ, g^μ). (M2) The true mean follows it exactly. (M3) If $\mu_k \le \bar{\mu}$ and the true mean is not larger, then $\Pr(D_k > \tau) \le \varepsilon$ (Proposition 6.7, Theorem 6.8). The loop row of Section 6.14 covers μ and, under ARD_1, g^μ. Each day of each repetition has its own set of terms; the argument above holds for each of them, because it needs neither independence nor a fixed set of terms.

Wiener Model

This section applies to the cells (i, ℓ) with `model` $=$ `wiener`. Inside it, the subscript $i\ell$ is dropped from the per-cell parameters and from the states; k is the day.

Model. The damage is a Wiener process with drift, observed once a day. A mission day adds a Gaussian increment $\mathcal{N}(\mu_{jk}^{\mathrm{inc}}, \sigma^2)$: the mean comes from the input `mu`, and the diffusion constant $\sigma > 0$ (input `sigma`) is the same for every mission, because it describes the material of the component, not the route. The initial damage is $\mathcal{N}(\mu_0, v_0)$ with $v_0 \ge 0$, and each replacement draw is $\mathcal{N}(\mu^{\mathrm{new}}, v^{\mathrm{new}})$. **Assumption W1 (independence).** These terms are mutually independent.

Repairs act on the mean only (a modelling choice). A repair shifts the damage by a known amount: $D^+ = D^- - \rho\, m^-$ under ARD_∞ and $D^+ = D^- - \rho\,(m^- - g^\mu)$ under ARD_1, with $m^- = \mathbb{E}D^-$. The law stays Gaussian, the variance is unchanged, and only a replacement lowers it.

Tracked states and repair laws. The cell tracks μ_k and v_k, and under ARD_1 the mean anchor g^μ_k; the variance has no anchor. Both laws are accepted. The mean follows the map (24) of the Gaussian model. The variance map is

$$v_k = \begin{cases} v_{k-1} + \delta v_k & \text{no intervention on day } k, \\ v_{k-1} & \text{repair, both laws,} \\ v^{\mathrm{new}} & \text{replacement,} \end{cases} \qquad \delta v_k = \sigma^2 \sum_{j=1}^{M} x_{ijk}. \tag{29}$$

Under W1 the damage is $\mathcal{N}(m_k, V_k)$ on every day, with (m_k, V_k) given by these maps: a mission adds an independent Gaussian term, and a repair or a replacement acts as stated above.

The rows are the generic rows of Section 6.4, with the data below. The variance takes the repair map $\psi = v_{k-1}$, that is the factor $a = 1$ under both laws, and no anchor. A depot day adds no increment, so for v the generic no-intervention and repair rows together equal the single row

$$v_k \geq v_{k-1} + \delta v_k - V_{\max}\, x^r_k, \tag{30}$$

which may replace them in the big-M form.

State	Day increment	Factor a	Natural range	Initial value	Replacement value
μ; anchor g^μ	$\delta\mu_k$	$1-\rho$	τ	μ_0	μ^{new}
v; no anchor	δv_k	1	$V_{\max}$	v_0	v^{new}

Remark (a repair of the path). A repair of the damage path, as in the Gaussian model, gives the same mean and the variance $(1-\rho)^2 V$ under ARD_∞, or $(1-\rho)^2 V + (1-(1-\rho)^2)\, V_G$ under ARD_1, with $V_G \leq V$ the variance just after the last intervention. Both are at most V. The reliability row increases in v, so the rows of this section are safe for that reading as well.

Reliability row

The row is the Gaussian row (26), with $\kappa_G = \Phi^{-1}(1-\varepsilon)$ and $\varepsilon < 0.5$, on every day $k = 1, \ldots, T$; its validity argument, its forms (`exact`, `tangent`, `pwl`, with the same function f and side row) and the remark on $V_{\max} = \tau^2/\kappa_G^2$ are those of Section 6.7. If `v_max_user` is given for the cell, the row $v_k \leq V^{\mathrm{user}}$ is added on every day.

Facts used by the encoding and loop arguments. (M1) The day maps are nondecreasing in (μ, v, g^μ). (M2) The true moments follow them exactly. (M3) As for the Gaussian model. The loop row of Section 6.14 covers μ, v and, under ARD_1, g^μ.

Remark (the loop forces a replacement). The variance grows by σ^2 on every mission day and falls only at a replacement. If the train flies n missions in the operating phase and the component is not replaced there, then $v_T \geq v_{H_1} + n\,\sigma^2 > v_{H_1}$, and the loop row $v_T \leq v_{H_1}$ fails. So every feasible schedule includes a replacement of the component in the operating phase whenever its train flies a mission there. No input check enforces this. With `allow_replacement` false, the train of a Wiener cell cannot fly in the operating phase.

Gamma Model

This section applies to the cells (i, ℓ) with `model` = `gamma`. Inside it, the subscript $i\ell$ is dropped from the per-cell parameters and from the states; k is the day.

Model. Gamma(a, β) is the Gamma law with shape $a > 0$ and rate $\beta > 0$; its mean is a/β. Every term that adds damage is a Gamma variable with its own rate:

- a mission increment with mean $\mu_{jk}^{\text{inc}} > 0$ (input `mu`) and rate β_{jk} (inputs `gamma_beta` and `gamma_beta_trans`) is Gamma$(\beta_{jk}\,\mu_{jk}^{\text{inc}},\ \beta_{jk})$;
- the initial damage with mean $\mu_0 > 0$ is Gamma$(\beta_0\,\mu_0,\ \beta_0)$ when its rate β_0 is given (input `gamma_beta_0`); when `gamma_beta_0` is omitted, the initial damage is known: the constant μ_0;
- each replacement draw with mean $\mu^{\text{new}} > 0$ is Gamma$(\beta^{\text{new}}\mu^{\text{new}},\ \beta^{\text{new}})$ when its rate β^{new} is given (input `gamma_beta_new`); when `gamma_beta_new` is omitted, it is the known constant μ^{new}.

Assumption G1 (independence). All these terms are mutually independent.

A repair acts on the damage path by the law of the cell (Section 6.3): ARD_∞ gives $D^+ = (1-\rho)\,D^-$, and ARD_1 gives $D^+ = D^{\text{last}} + (1-\rho)\,(D^- - D^{\text{last}})$. After a repair the damage is, in general, not a Gamma variable: it is a weighted sum of independent Gamma terms and known constants. The reliability row below holds for every such sum.

Tracked state and repair laws. The cell tracks the mean μ_k only, and under ARD_1 its anchor g_k^μ. Both laws are accepted. The shape of the damage is not tracked. The exact day map of the mean is

$$\mu_k = \begin{cases} \mu_{k-1} + \delta\mu_k & \text{no intervention on day } k, \\ (1-\rho)\,\mu_{k-1} & \text{repair, } \text{ARD}_\infty, \\ (1-\rho)\,\mu_{k-1} + \rho\, g_{k-1}^\mu & \text{repair, } \text{ARD}_1, \\ \mu^{\text{new}} & \text{replacement}, \end{cases} \tag{31}$$

with $\delta\mu_k = \sum_{j=1}^{M} x_{ijk}\,\mu_{jk}^{\text{inc}}$. The map is exact for the true mean $\mathbb{E}D_k$, because expectation is linear. The rows are the generic rows of Section 6.4, with the data below.

State	Day increment	Factor a	Natural range	Initial value	Replacement value
μ; anchor g^μ	$\delta\mu_k$	$1-\rho$	τ	μ_0	μ^{new}

Smallest rate. The cell's smallest rate is

$$\beta^* = \min\Big(\{\beta_{jk} : \mu_{jk}^{\text{inc}} > 0\} \cup \{\beta_0 : \mu_0 > 0\} \cup \{\beta^{\text{new}} : \mu^{\text{new}} > 0\}\Big), \tag{32}$$

where j runs over the missions and k over the days of both phases, and β_0, β^{new} enter only when they are given. So β^* is the smallest rate among the terms with a positive mean: a mission with $\mu_{jk}^{\text{inc}} = 0$ adds no term, and an omitted rate declares a known damage, which is a constant and does not enter β^*. If no term has a rate, the damage is a known constant, and the row below is $\mu_k \le \tau$. Write $t = \beta^*\tau$. Units of $1/\beta^*$ turn every law below into a law of rate at least 1.

Class of true states

Definition 6.9 (Class of weighted Gamma sums)**.** Let $m \ge 0$. The class $\mathcal{C}(m)$ holds the laws of the random variables

$$S = \delta + \sum_{i=1}^{n} c_i\, Y_i, \qquad Y_i \sim \text{Gamma}(a_i, 1) \text{ independent}, \quad c_i \in (0, 1], \quad a_i > 0, \quad \delta \ge 0,$$

with $n \geq 0$ and mean $\delta + \sum_i c_i a_i = m$. The term $c\,Y$ with $Y \sim \text{Gamma}(m'/c, 1)$ has mean m'; as $c \to 0$ it tends to the constant m'.

Lemma 6.10 (True states belong to the class)**.** *Let D_k be the damage of a Gamma cell on day k of a schedule, under either law. Then the law of $\beta^* D_k$ belongs to $\mathcal{C}(\beta^* \,\mathbb{E}D_k)$.*

Proof. Since the last replacement (or day 0), the damage is $D_k = \delta + \sum_t w_t X_t$: the X_t are the independent Gamma terms added since then, $\delta \geq 0$ is the weighted sum of the known constants, and each weight $w_t \in (0, 1]$ is a product of factors $1 - \rho$. A mission adds a term with weight 1. ARD_∞ multiplies every weight, constants included, by $1 - \rho$. ARD_1 multiplies by $1 - \rho$ the weights of the terms and constants added since the last intervention, and keeps the others. A replacement starts a new sum with the replacement draw. When the starting anchor g_0^μ is below μ_0, the initial damage is read as two independent terms of rate β_0, with means g_0^μ and $\mu_0 - g_0^\mu$, whose sum has the law $\text{Gamma}(\beta_0\mu_0, \beta_0)$; a known initial damage is read as the two constants g_0^μ and $\mu_0 - g_0^\mu$. Write $X_t = Y_t/\beta_t$ with $Y_t \sim \text{Gamma}(a_t, 1)$ and $\beta_t \geq \beta^*$ by Eq. (32). Then

$$\beta^* D_k = \beta^* \delta + \sum_t \frac{w_t\,\beta^*}{\beta_t}\, Y_t, \qquad \frac{w_t\,\beta^*}{\beta_t} \in (0, 1].$$

The Y_t are independent by Assumption G1. Therefore the law of $\beta^* D_k$ is in $\mathcal{C}(\beta^* \,\mathbb{E}D_k)$. □

The largest tail at a given mean

For $0 < m < t$, let $W(m, t)$ be the supremum of $\Pr(S > t)$ over the random variables S whose law is in $\mathcal{C}(m)$. $Q(a, x) = \Pr(\text{Gamma}(a, 1) > x)$ is the regularised upper incomplete gamma function (`scipy.special.gammaincc(a, x)`). The one-term member $c\,\text{Gamma}(m/c, 1)$ has the tail $Q(m/c,\, t/c)$ at t.

A member of $\mathcal{C}(m)$ is described by its atoms: the pairs (m_i, c_i) of a term mean $m_i = c_i a_i$ and a scale c_i, and the constant δ, read as an atom at scale $c = 0$. Terms with equal scales merge into one term, because a sum of independent Gamma variables of one rate is a Gamma variable.

Lemma 6.11 (First-order rates)**.** *Let S be a member with at least one term, with density f_S. Since $\delta \leq m < t$, f_S is continuous at t. For $c \in (0, 1]$ let*

$$\varphi_S(c) = \frac{1}{c} \int_0^\infty \Pr(t - y < S \leq t)\, \frac{e^{-y/c}}{y}\, dy, \qquad \varphi_S(0) = f_S(t).$$

(a) Adding mean η at scale c (a term $c\,\text{Gamma}(\eta/c, 1)$, or the constant η when $c = 0$) raises $\Pr(S > t)$ at the rate $\varphi_S(c)$ as $\eta \to 0$; removing mean from an atom at scale c lowers it at the same rate. (b) Moving the scale of an atom (m_i, c_i), $c_i \in (0, 1)$, at fixed mean m_i changes $\Pr(S > t)$ at the rate $m_i\, \partial\varphi_S/\partial c\,(c_i)$.

Proof. S is infinitely divisible, with drift δ and Lévy density $\ell(y) = \sum_i (m_i/c_i)\, e^{-y/c_i}/y$ on $y > 0$. A change of the parameters at the rates $(\dot\delta, \dot\ell)$ changes the Laplace transform $\mathbb{E}e^{-\sigma S} = \exp\bigl(-\sigma\delta - \int_0^\infty (1 - e^{-\sigma y})\, \ell(y)\, dy\bigr)$ at the rate $\int_0^\infty \mathbb{E}\bigl[e^{-\sigma(S+y)} - e^{-\sigma S}\bigr]\, \dot\ell(y)\, dy - \sigma\dot\delta\, \mathbb{E}e^{-\sigma S}$. This is the transform of the rate $\int_0^\infty \Pr(t - y < S \leq t)\, \dot\ell(y)\, dy + \dot\delta\, f_S(t)$ of $\Pr(S > t)$. Case (a) is $\dot\ell(y) = c^{-1} e^{-y/c}/y$, or $\dot\delta = 1$ when $c = 0$. Case (b) is $\dot\ell(y) = m_i\, \partial_c\bigl(c^{-1} e^{-y/c}/y\bigr)$ at $c = c_i$. □

Theorem 6.12 (One term is the worst case)**.** *Let $0 < m < t$. Then*

$$W(m, t) = \max_{c \in (0,1]} Q\left(\frac{m}{c},\, \frac{t}{c}\right). \tag{33}$$

Proof. Let $\mathcal{C}_K(m)$ be the members with at most K atoms, the constant included, and let $K \geq 3$.

1. *A maximiser exists.* $\mathcal{C}_K(m)$ is the image of the compact set of atoms (m_i, c_i) with $m_i \geq 0$, $\sum_i m_i = m$, $c_i \in [0,1]$. The law of S depends continuously on the atoms, and only the constant member $S = m$ has an atom, at $m \neq t$. So $\Pr(S > t)$ is continuous and attains its maximum at some S^*. Since $\Pr(m > t) = 0 < Q(m,t)$, S^* has at least one term. Every member has finitely many atoms, so $W(m,t) = \sup_K \max_{\mathcal{C}_K(m)} \Pr(S > t)$.

2. *First-order conditions.* Write $\varphi^* = \varphi_{S^*}$. By Lemma 6.11: (i) moving mean between two atoms of positive mean must not raise the tail, so φ^* takes one value κ^* at every atom of positive mean; (ii) at an atom with $c \in (0,1)$, $\partial\varphi^*/\partial c = 0$; (iii) if S^* has fewer than K atoms of positive mean, a new atom can be opened at any $c \in [0,1]$, so $\varphi^*(c) \leq \kappa^*$ on $[0,1]$.

3. *A single-peak function.* Put $\lambda = 1/c$ and $h(u) = \Pr(t-u < S^* \leq t)/u$. Then $\varphi^*(\lambda) = \lambda \int_0^\infty h(u)\, e^{-\lambda u}\, du$, and $\varphi^*(\infty) = h(0^+) = f_{S^*}(t)$ is the value at the constant atom. S^* is a constant plus a sum of independent Gamma variables, so its density is unimodal [3, Theorem 4, p. 11], proved in [4, proof of Theorem 4, pp. 194–195]; the constant shifts the mode only. Then $g(v) = f_{S^*}(t-v)$ for $v < t$ and $g(v) = 0$ for $v \geq t$ is unimodal on $[0,\infty)$, and so is its running average $h(u) = u^{-1}\int_0^u g$: before the mode of g, $g \geq h$; after it, $u\,(g(u) - h(u)) = u\,g(u) - \int_0^u g$ is nonincreasing. Since $h' = (g-h)/u$, h rises and then falls.

4. *Zero count.* For every constant κ', $h - \kappa'$ changes sign at most twice. The Laplace transform of a function with at most n sign changes has at most n zeros in $\lambda > 0$, counted with multiplicity: at a sign change u_1, multiply the transform by $e^{\lambda u_1}$; its derivative in λ is the transform of a function with one sign change fewer; Rolle's theorem and induction conclude. So $\varphi^* - \kappa' = \lambda \int_0^\infty (h - \kappa')\, e^{-\lambda u}\, du$ has at most two zeros in $\lambda > 0$. Also $\varphi^*(\lambda) \to 0$ as $\lambda \to 0^+$. Hence φ^* cannot rise above a level $\kappa' > 0$, fall below it and rise above it again: with $\varphi^*(0^+) = 0$ that would give three zeros. So φ^* is quasi-concave. It is analytic in $\lambda > 0$ and not constant, so its maximum over $\lambda \in [1,\infty]$ is attained at one point only: two maximisers would make φ^* constant between them.

5. *One term.* By (i) and (ii), an atom with $c \in (0,1)$ is a zero of $\varphi^* - \kappa^*$ of multiplicity at least 2, and an atom at $c = 1$ a zero of multiplicity at least 1; the constant atom is at $\lambda = \infty$. By step 4, S^* has at most two atoms of positive mean. As $K \geq 3$, (iii) applies, so κ^* is the maximum of φ^* over $[1,\infty]$, attained at every atom of positive mean. By step 4 there is only one such atom, and by step 1 it is not the constant. So $S^* = c^*\,\mathrm{Gamma}(m/c^*, 1)$ with $c^* \in (0,1]$.

Therefore $\max_{\mathcal{C}_K(m)} \Pr(S > t) = \max_{c\in(0,1]} Q(m/c, t/c)$ for every $K \geq 3$, and Eq. (33) follows from step 1. □

Lemma 6.13 (Monotone in the mean)**.** *$W(m,t)$ is nondecreasing in $m \in (0,t)$.*

Proof. Let $m < m' < t$ and let the law of S be in $\mathcal{C}(m)$. Then the law of $S + (m' - m)$ is in $\mathcal{C}(m')$, and $\Pr(S + m' - m > t) \geq \Pr(S > t)$. Therefore $W(m,t) \leq W(m',t)$. □

Reliability row

The cap. For $A \in [0,t)$ let $w(A) = \max_{c\in(0,1]} Q(A/c,\, t/c)$; it is continuous and nondecreasing in A, because $Q(a,x)$ increases in a. The *sharp cap* $A_1(t,\varepsilon)$ is the largest $A \in [0,t)$ with $w(A) \leq \varepsilon$, computed as described below. The row, on every day $k = 1,\ldots,T$, is

$$\mu_k \;\leq\; \frac{A_1(\beta^*\tau,\, \varepsilon)}{\beta^*}. \tag{34}$$

The row is linear: it is built in its exact form, and a value of `reliability_impl` at the cell is ignored (Section 6.6).

Corollary 6.14 (The row is safe)**.** *Let the tracked mean satisfy Eq. (34) on day k. Then* $\Pr(D_k > \tau) \le \varepsilon$.

Proof. Let $m = \beta^* \mathbb{E}D_k$ and $A = A_1(\beta^*\tau, \varepsilon)$. The tracked mean bounds the true mean from above (Section 6.4), so $m \le \beta^*\mu_k \le A < t$. If $m = 0$, then $D_k = 0$. Otherwise

$$\Pr(D_k > \tau) = \Pr(\beta^* D_k > t) \le W(m, t) \le W(A, t) = w(A) \le \varepsilon,$$

by Lemma 6.10, Lemma 6.13, Theorem 6.12 and the choice of A. □

Computation of A_1 (offline, once per cell, before the model is built):

1. $t = \beta^*\tau$. $\hat{A}$ is the root of $Q(\hat{A}, t) = \varepsilon$, found by bisection on $\hat{A}$ (Q increases in its first argument; `gammaincinv` solves for the second argument and does not apply).
2. $w(A)$: the function $c \mapsto Q(A/c, t/c)$ tends to 0 as $c \to 0$ and has a single peak on $(0, 1]$ (checked numerically for t from 10^{-3} to 100 and A/t from 0.05 to 0.999). Its maximum is found on a logarithmic grid of at least 400 values of c in $[\min(10^{-4}, t/100), 1]$, then refined by a golden-section search between the neighbours of the best grid point. If the best grid point is the smallest value of c, the grid is extended downwards.
3. If $\hat{A} < t$ and $w(\hat{A}) \le \varepsilon$, then $A_1 = \hat{A}$. Otherwise A_1 is found by bisection on A in $[\varepsilon t, \min(\hat{A}, t))$, to a relative tolerance of 10^{-9}. The lower end satisfies $w(\varepsilon t) \le \varepsilon$, by Markov's inequality $Q(a, x) \le a/x$. The bisection returns the lower end of its last interval, so $w(A_1) \le \varepsilon$ holds at the returned value.

Remark (the sharp cap and the exact one-rate cap). $\hat{A}$ is the exact cap of a single Gamma law of rate β^*, and $A_1 \le \hat{A}$, because $c = 1$ is in the maximum. Let $t_1(\varepsilon)$ be the value of t at which $\partial_c Q(\hat{A}/c, t/c)$ at $c = 1$ changes sign: $t_1 = 0.436$, 0.448 and 0.614 for $\varepsilon = 0.001$, 0.01 and 0.1. For $t \ge t_1$, $A_1 = \hat{A}$: on a grid of t in $[t_1, 1000]$, no $c < 1$ gives a larger tail than $c = 1$. Below t_1, $A_1 < \hat{A}$. For $\varepsilon = 0.01$:

$t = \beta^*\tau$	0.01	0.1	0.3	1	10
$A_1/\hat{A}$	0.142	0.642	0.961	1	1

When every term has the same rate and no repair occurs, the true state is a single Gamma law and $\hat{A}$ is exact for it. With several rates, or with repairs, the true state is a weighted sum, and below t_1 the exact one-rate cap is not safe. Example: $\varepsilon = 0.01$ and $\beta^*\tau = 0.1$, so $\hat{A} = 0.0055012$. A new part flies one mission of mean $\hat{A}/\beta^*$ and rate $4.56\,\beta^*$. Its mean meets the exact one-rate cap, but its tail is $1.565\,\varepsilon$. The sharp cap is $A_1 = 0.642\,\hat{A}$ there.

Remark (below the threshold). With $r = A/t$ and $s = t/c$, $Q(A/c, t/c) = Q(rs, s)$, and $c \in (0, 1]$ means $s \ge t$. For $t \le t_1(\varepsilon)$ the peak of $s \mapsto Q(rs, s)$ at the cap lies at $s \ge t$, so the maximum does not depend on t, and $A_1(t, \varepsilon) = r^*(\varepsilon)\,t$. The mean cap is then $A_1/\beta^* = r^*(\varepsilon)\,\tau$, whatever the rates: $r^* = 0.00355$, 0.0353 and 0.331 for $\varepsilon = 0.001$, 0.01 and 0.1.

Remark (the choice of β^*). A larger t shrinks the range $s \ge t$, so $A_1(t, \varepsilon)/t$ is nondecreasing in t, and the mean cap $A_1(\beta^*\tau, \varepsilon)/\beta^* = \tau\, A_1(t, \varepsilon)/t$ is nondecreasing in β^*. Lemma 6.10 holds for every β^* at most the smallest rate. So the smallest rate gives the largest safe cap of this form.

Remark (conservatism). The class $\mathcal{C}(m)$ also contains laws that no schedule produces. Below t_1 the row can therefore reject some safe schedules. The row keeps each mission's own mean; β^* only bounds how dispersed each term may be.

Facts used by the encoding and loop arguments. (M1) The day map (31) is nondecreasing in (μ, g^μ). (M2) The true mean follows the map (31) exactly. (M3) If the tracked mean satisfies Eq. (34) and the true mean is not larger, then $\Pr(D_k > \tau) \le \varepsilon$ (Corollary 6.14). The loop row of Section 6.14 covers μ and, under ARD_1, g^μ.

Rainflow Model

This section applies to the cells (i, ℓ) with `model` = `rainflow`. Inside it, the subscript $i\ell$ is dropped from the per-cell parameters and from the states; k is the day.

The rainflow model is *distribution-free*. The law of a mission increment is measured from recorded load histories by rainflow counting (Palmgren–Miner damage), and it enters the model only through summary statistics: its mean, and, as the bound requires, its variance, the width of its range, or its cumulant generating function (CGF) at a fixed point s. No parametric family is assumed. The reliability requirement $\Pr(D_k > \tau) \le \varepsilon$ is certified by one of five tail bounds, chosen per cell by `bound_method`: `markov`, `cantelli`, `hoeffding`, `bernstein`, `chernoff`.

Modelling assumptions (normative).

- **A0 (non-negative damage).** Every increment, the initial damage and every replacement draw are non-negative.
- **A1 (independence).** The mission increments are mutually independent, and independent of the initial damage and of every replacement draw. Each replacement installs a new draw, independent of everything else. Every bound except Markov needs A1.
- **A2 (bounded ranges; Hoeffding and Bernstein).** Each mission increment lies in an interval of width b^{inc}_{jk} (input `support`). The initial damage is a known part plus independent terms, each in an interval; R_0 is the sum of their squared widths (input `R_0`) and b^0 the largest width (input `support_0`). The replacement draw is described in the same way by R^{new} and b^{new} (inputs `R_new`, `support_new`). A known damage has $R = b = 0$, the defaults.
- **A3 (repetition).** The row of the cell's bound holds on every day $k = 1, \dots, T$, and after day T the operating phase repeats with period H_2, schedule and increment profile included. This is the setting of the loop argument of Section 6.14.

Width of a range. The input `support` of an increment X is the width b of an interval $[l, l + b]$ that contains X almost surely. Hoeffding's inequality uses the widths. Bernstein's inequality needs an upper bound of the centred term, and the width is one: $X - \mathbb{E}X \le l + b - l = b$, since $\mathbb{E}X \ge l$. A width b is consistent with a variance v only if $v \le b^2/4$ (Popoviciu's inequality: $\operatorname{Var} X \le \mathbb{E}(X - l - b/2)^2 \le (b/2)^2$); the check is in Section 3.12.

Tracked states per bound. Every cell tracks the mean μ. Each bound adds at most one state, and under ARD_1 one anchor per tracked state (g^μ and g^v, g^R or g^K). Both laws are accepted for every bound.

`bound_method`	States	Increment inputs	Initial; replacement inputs	Forms
`markov`	μ	`mu`	–	`exact`
`cantelli`	μ, v	`mu`, `v`	`v_0`; `replacement_v`	all three
`hoeffding`	μ, R	`mu`, `support`	`R_0`; `R_new`	all three
`bernstein`	μ, v	`mu`, `v`, `support`	`v_0`, `support_0`; `replacement_v`, `support_new`	all three
`chernoff`	μ, K	`mu`, `cgf`, `s_chernoff`	`cgf_0`; `cgf_new`	`exact`

The column "Forms" lists the forms in which the row can be built; "all three" means `exact`, `tangent` and `pwl`. A linear row is built in its exact form, and a value of `reliability_impl` at its cell is ignored. R is the *Hoeffding budget*, the sum of the squared widths of the terms of the damage. K is the CGF of the damage at the cell's parameter s = `s_chernoff` > 0. The day increments are $\delta\mu_k = \sum_{j=1}^{M} x_{ijk}\, \mu^{\mathrm{inc}}_{jk}$, $\delta v_k = \sum_j x_{ijk}\, v^{\mathrm{inc}}_{jk}$, $\delta R_k = \sum_j x_{ijk}\, (b^{\mathrm{inc}}_{jk})^2$ and $\delta K_k = \sum_j x_{ijk}\, K^{\mathrm{inc}}_{jk}$, with $K^{\mathrm{inc}}_{jk} = \ln \mathbb{E}\, e^{sX_{jk}}$ the CGF of the increment at s (input `cgf`).

Day maps

Between replacements the damage is $D_k = \sum_t w_t X_t$: the X_t are the known part, the independent terms of the initial damage or of the replacement draw, and the mission increments; each weight $w_t \in (0, 1]$ is a product of factors $1-\rho$. Under ARD_1, write $D^- = G + \Delta$, with G the damage just after the last intervention and Δ the weighted increments added since. G and Δ are independent by A1, and a repair gives $D^+ = G + (1-\rho)\,\Delta$. Before the first intervention, G is the part of the initial damage that the starting anchors describe (Section 6.3).

For a damage $D = \sum_t w_t X_t$, write $m(D) = \mathbb{E}D$, $V(D) = \operatorname{Var} D$, $R(D) = \sum_t w_t^2 r_t$ with r_t the squared width of X_t, and $K(D) = \ln \mathbb{E}\, e^{sD}$.

Proposition 6.15 (Day maps of the statistics)**.** *Let $\theta \in \{m, V, R\}$ with factor $a_\theta = 1-\rho$ for m and $a_\theta = (1-\rho)^2$ for V and R. Under A1:*

1. *a mission day adds the increment's statistic: $\theta(D+X) = \theta(D) + \theta(X)$ and $K(D+X) = K(D) + K(X)$;*
2. *ARD_∞: $\theta\big((1-\rho)D\big) = a_\theta\,\theta(D)$ and $K\big((1-\rho)D\big) \le (1-\rho)\,K(D)$;*
3. *ARD_1: $\theta(D^+) = a_\theta\,\theta(D^-) + (1-a_\theta)\,\theta(G)$ and $K(D^+) \le (1-\rho)\,K(D^-) + \rho\,K(G)$.*

Proof. Item 1: the mean and R are additive by definition, the variance and the CGF by independence. Item 2: m and V scale by $1-\rho$ and $(1-\rho)^2$; every weight is multiplied by $1-\rho$, so R scales by $(1-\rho)^2$. A CGF is convex with value 0 at 0, so $K\big((1-\rho)D\big) = K_D\big((1-\rho)s\big) \le (1-\rho)\,K_D(s) + \rho\,K_D(0) = (1-\rho)\,K(D)$, with K_D the CGF of D as a function of its argument. Item 3: by independence, $\theta(D^-) = \theta(G) + \theta(\Delta)$; for R the weights of the terms of G are unchanged and those of Δ are multiplied by $1-\rho$. Hence

$$\theta(D^+) = \theta(G) + a_\theta\,\theta(\Delta) = \theta(G) + a_\theta\big(\theta(D^-) - \theta(G)\big) = a_\theta\,\theta(D^-) + (1-a_\theta)\,\theta(G),$$

and, with item 2 applied to Δ,

$$K(D^+) = K(G) + K_\Delta\big((1-\rho)s\big) \le K(G) + (1-\rho)\big(K(D^-) - K(G)\big) = (1-\rho)\,K(D^-) + \rho\,K(G).$$

□

So each tracked state follows the generic map of Section 6.4 with the factor $a = 1-\rho$ for μ and K, and $a = (1-\rho)^2$ for v and R. Its anchor is its value just after the last intervention. The maps of μ, v and R are exact; the map of K is an upper bound, which is what the domination argument of Section 6.4 needs.

Starting and replacement values

The initial and replacement values of μ and v come from the inputs (Section 6.5). Those of R and K are:

- R_0 from `R_0` and R^{new} from `R_new`, default 0;
- K_0 from `cgf_0`, default $s\,\mu_0$, and K^{new} from `cgf_new`, default $s\,\mu^{\text{new}}$.

With `replacement_as_new` all replacement values are 0 (Section 6.5).

Why the defaults. By Jensen's inequality, $K(D) = \ln \mathbb{E}\, e^{sD} \ge s\,\mathbb{E}D$, with equality exactly when D is almost surely constant. So $s\mu_0$ is the exact CGF of a known initial damage μ_0, and a lower bound for a random one; a random initial damage needs its own CGF in `cgf_0`. The input checks of Section 3.12 require `cgf` $\ge s\,\mu^{\text{inc}}$ for every increment, $K_0 \ge s\mu_0$ and $K^{\text{new}} \ge s\mu^{\text{new}}$. A known damage has width 0, so $R_0 = 0$ is exact for it, and a random one needs `R_0`. The same holds for the replacement draw. A starting value of K below $s\mu_0$ makes the row optimistic. Example: $\tau = 1$, $\varepsilon = 0.1$, $s = 5$, a known initial damage $\mu_0 = 0.6$, and one mission that adds 0 or 0.5

with probability 1/2 each, so $K^{\text{inc}} = \ln(0.5 + 0.5\,e^{2.5}) = 1.886$. Then $\Pr(D > \tau) = 0.5$. The row with $K_0 = s\mu_0 = 3$ gives $4.886 - 5 > \ln 0.1$ and rejects the plan; a start at $K_0 = 0$ would give $1.886 - 5 = -3.11 \le \ln 0.1 = -2.30$ and accept it.

Rows of the dynamics

The rows are the generic rows of Section 6.4, built for μ and for the one extra state of the bound, with the data below. $V^{\text{C}} = \varepsilon\tau^2/(1-\varepsilon)$ and $V^{\text{B}} = \tau(\tau - 2c)/(2\kappa)$ are the variance ranges of the Cantelli and Bernstein rows, with $\kappa = \ln(1/\varepsilon)$ and c from Eq. (38). Each natural range is the largest value that the bound's exact row admits at $\mu \ge 0$. The big-M rows need every initial value, starting anchor and replacement value inside the natural range of its state; the input checks of Section 3.12 require it.

State	Day increment	Factor a	Natural range	Initial value	Replacement value
μ; anchor g^{μ}	$\delta\mu_k$	$1-\rho$	τ	μ_0	μ^{new}
v; anchor g^{v}	δv_k	$(1-\rho)^2$	V^{C} or V^{B}	v_0	v^{new}
R; anchor g^{R}	δR_k	$(1-\rho)^2$	$2\tau^2/\kappa$	R_0	R^{new}
K; anchor g^{K}	δK_k	$1-\rho$	$s\tau - \kappa$	K_0	K^{new}

The anchors exist only under ARD_1, and the extra state only for its bound: v for Cantelli and Bernstein, R for Hoeffding, K for Chernoff.

Reliability rows

Write $d_k = \tau - \mu_k$. Let $(m, V, R^{\text{t}}, K^{\text{t}})$ be the true statistics of D_k. By Proposition 6.15 and Section 6.4, each is at most the tracked state of the same name. Each row below is imposed on every day $k = 1, \dots, T$.

Markov (`markov`). By A0 and Markov's inequality, $\Pr(D_k > \tau) \le m/\tau \le \mu_k/\tau$. The row is

$$\mu_k \le \varepsilon\,\tau. \tag{35}$$

It uses the mean only and does not need A1 or A2.

Cantelli (`cantelli`). Cantelli's inequality gives $\Pr(D - m \ge a) \le V/(V + a^2)$ for $a > 0$. The row is

$$(1-\varepsilon)\,v_k \le \varepsilon\,(\tau - \mu_k)^2, \qquad \mu_k \le \tau, \tag{36}$$

equivalently $\mu_k + \kappa_C\sqrt{v_k} \le \tau$ with $\kappa_C = \sqrt{(1-\varepsilon)/\varepsilon}$. *Validity.* If $d_k > 0$,

$$\Pr(D_k > \tau) \le \Pr(D_k - m \ge \tau - m) \le \frac{V}{V + (\tau - m)^2} \le \frac{v_k}{v_k + d_k^2} \le \varepsilon,$$

because $v/(v + d^2)$ increases in v and decreases in d, and $\tau - m \ge d_k$. If $d_k = 0$, the row gives $v_k = 0$, so $V = 0$ and $D_k = m \le \tau$.

Hoeffding (`hoeffding`). $D_k - m$ is a sum of independent centred terms $w_t(X_t - \mathbb{E}X_t)$ with ranges of widths $w_t\sqrt{r_t}$, and the sum of their squares is R^{t}. Hoeffding's inequality gives $\Pr(D_k - m \ge a) \le \exp(-2a^2/R^{\text{t}})$, and $\exp(-2d^2/R) \le \varepsilon$ exactly when $d^2 \ge (\kappa/2)\,R$. The row is

$$(\tau - \mu_k)^2 \ge \frac{\kappa}{2}\,R_k, \qquad \mu_k \le \tau. \tag{37}$$

Validity. If $R^{\text{t}} = 0$, every term is constant and $D_k = m \le \tau$; this covers $d_k = 0$, where the row gives $R_k = 0$. Otherwise $d_k > 0$, and $\Pr(D_k > \tau) \le \exp\bigl(-2(\tau - m)^2/R^{\text{t}}\bigr) \le \exp(-2d_k^2/R_k) \le \varepsilon$, because $\exp(-2d^2/R)$ increases in R and decreases in d, and $\tau - m \ge d_k$.

Bernstein (`bernstein`). The *support constant* of the cell is

$$b = \max\Big(b^0,\ b^{\text{new}},\ \max_{j,k} b^{\text{inc}}_{jk}\Big), \qquad c = \frac{\kappa\, b}{3}, \tag{38}$$

over the missions j and the days k of both phases. It is a constant, not a decision variable. Every weighted term satisfies $w_t(X_t - \mathbb{E}X_t) \le w_t\, b_t \le b$, whatever the schedule, because $w_t \le 1$. Bernstein's inequality for independent terms bounded above by b gives $\Pr(D - m \ge a) \le \exp\big(-a^2/(2(V + b\,a/3))\big)$ for $a > 0$; the right side is at most ε exactly when $a^2 - 2c\,a - 2\kappa V \ge 0$. The row is

$$(\tau - c - \mu_k)^2 \ge c^2 + 2\kappa\, v_k, \qquad \mu_k \le \tau - c, \tag{39}$$

equivalently $\mu_k + c + \sqrt{c^2 + 2\kappa v_k} \le \tau$. With $v_k \ge 0$, the two rows give $d_k \ge 2c$. *Validity.* $a^2/(2(V + b\,a/3))$ increases in $a > 0$ and decreases in V, and $\tau - m \ge d_k \ge 2c$; so $\Pr(D_k > \tau) \le \varepsilon$ when $b > 0$. When $b = 0$, every term is constant and $D_k = m \le \tau$.

Remark (the side row). The quadratic row alone also admits the point $d_k = 0$, $v_k = 0$: a known damage equal to τ. That point is safe, but it is cut off from the rest of the feasible set. The side row $\mu_k \le \tau - c$ removes it and keeps the branch $d_k \ge c + \sqrt{c^2 + 2\kappa v_k}$. At $\mu = v = 0$ the row needs $\tau \ge 2c$; a positive variance needs $\tau > 2\kappa b/3$, the input check of Section 3.12.

Chernoff (`chernoff`). For $s > 0$, Markov's inequality applied to e^{sD} gives $\Pr(D_k \ge \tau) \le \exp(K^{\text{t}} - s\tau)$. The row is

$$K_k - s\,\tau \le \ln \varepsilon. \tag{40}$$

Validity. $K^{\text{t}} \le K_k$, so $\Pr(D_k > \tau) \le \exp(K_k - s\tau) \le \varepsilon$. The row bounds K by $s\tau - \kappa$, its natural range; $K \ge 0$, because $K(X) \ge s\,\mathbb{E}X \ge 0$ by Jensen's inequality and A0. A known zero damage satisfies the row only if $s\tau \ge \kappa$ (input check $s > \ln(1/\varepsilon)/\tau$, Section 3.12). A replacement day satisfies it only if $K^{\text{new}} \le s\tau - \kappa$; the input checks require every replacement state to satisfy the cell's row.

Linear forms. Markov and Chernoff rows are linear: they are built in their exact form, and a value of `reliability_impl` at the cell is ignored. The Cantelli, Hoeffding and Bernstein rows have the form $\chi_k \le f(\mu_k)$ with f convex in μ, and accept the forms `exact`, `tangent` and `pwl` of Section 6.6. Every form keeps the side row.

Bound	χ	$f(\mu)$	Side row	$f(0)$
Cantelli	v	$\varepsilon(\tau - \mu)^2/(1 - \varepsilon)$	$\mu \le \tau$	$\varepsilon\tau^2/(1 - \varepsilon)$
Hoeffding	R	$2(\tau - \mu)^2/\kappa$	$\mu \le \tau$	$2\tau^2/\kappa$
Bernstein	v	$\big((\tau - \mu)^2 - 2c(\tau - \mu)\big)/(2\kappa)$	$\mu \le \tau - c$	$\tau(\tau - 2c)/(2\kappa)$

Each f is a parabola in μ with a positive leading coefficient, so it is convex. For Bernstein with $b > 0$, the tangent point must satisfy $\mu_r < \tau - c$, an input check of Section 3.12. At $\mu_r \ge \tau - c$ the tangent has a nonnegative slope and lies below f, so at every $\mu \le \tau - c$ it is at most $f(\tau - c) = -c^2/(2\kappa) < 0$, and no point with $v \ge 0$ satisfies the row.

Optional row. If `v_max_user` is given for a Cantelli or Bernstein cell, the row $v_k \le V^{\text{user}}$ is added on every day.

Facts used by the encoding and loop arguments. (M1) Every day map is nondecreasing in each state and anchor: its coefficients a and $1 - a$ lie in $[0, 1]$. (M2) The true statistics follow the maps with equality for μ, v and R, and with $\le$ for K (Proposition 6.15). (M3) If the tracked states satisfy the row of the cell's bound and the true statistics are not larger, then $\Pr(D_k > \tau) \le \varepsilon$ (the validity paragraphs above). The loop row of Section 6.14 covers μ, the extra state of the bound and, under ARD_1, their anchors.

Repair Cost Variable

The variable $z_{i\ell k} \geq 0$ is the degradation removed by a repair of cell (i, ℓ) on day k, in mean units, for every model and both laws. The objective charges $C_{R,i\ell}\, z_{i\ell k}$ (Section 5). The variable and its rows are built only at cells with $C_{R,i\ell} > 0$, and only on the days whose cost enters the objective: every day $k = 1, \ldots, T$, except the days of the transitory phase in the mode `operating_average`. The rows are

$$z_{i\ell k} \;\geq\; \rho_{i\ell}\left(\mu_{i\ell,k-1} - g^{\mu}_{i\ell,k-1}\right) - \rho_{i\ell}\,\tau_{i\ell}\left(1 - x^{m}_{i\ell k}\right), \qquad z_{i\ell k} \geq 0, \tag{41}$$

with $g^{\mu} \equiv 0$ under ARD_{∞}. On a repair day the first row reads $z_{i\ell k} \geq \rho\,(\mu_{i\ell,k-1} - g^{\mu}_{i\ell,k-1})$, the removed amount of Eq. (10). On any other day its right-hand side is at most 0, because $0 \leq \mu_{i\ell,k-1} - g^{\mu}_{i\ell,k-1} \leq \mu_{i\ell,k-1} \leq \tau$.

Replacement days. A replacement day has $x^{m}_{i\ell k} = 0$, so $z_{i\ell k} = 0$ at the optimum. A replacement costs $C_{\mathrm{rep},i\ell}$ only. The replaced part is discarded, so its damage does not change the price of the new part.

Remark (no upper rows). The variable $z_{i\ell k}$ appears in no row other than Eq. (41), and in the objective with a positive coefficient. The minimisation therefore drives it to the largest of its lower bounds: $z_{i\ell k} = \rho\,(\mu_{i\ell,k-1} - g^{\mu}_{i\ell,k-1})$ on a repair day and $z_{i\ell k} = 0$ on any other day. The upper rows of a McCormick envelope, $z \leq \rho\,(\mu - g^{\mu})$ and $z \leq \rho\,\tau\,x^{m}$, would only remove points that are never optimal, so they are not built. A solution that is not optimal (for example, the best one found at the time limit) may carry a larger z; its reported cost is then larger, never smaller.

Proposition 6.16 (What z measures)**.** *In every feasible solution, on every repair day k of a cell, under both laws,*

$$z_{i\ell k} \;\geq\; \rho\left(\mu_{k-1} - g^{\mu}_{k-1}\right) \;\geq\; \rho\left(\zeta^{\mu}_{k-1} - \zeta^{g}_{k-1}\right),$$

where ζ^{μ} and ζ^{g} are the mean and its anchor on the reference trajectory (Section 6.4). When the starting anchor equals the true mean level after the last intervention before the horizon, the right-hand side is the expected degradation that the repair removes.

Proof. The first inequality is Eq. (41) with $x^{m}_{i\ell k} = 1$. Under ARD_{∞}, $g^{\mu} = \zeta^{g} = 0$, and $\mu_{k-1} \geq \zeta^{\mu}_{k-1}$ by Proposition 6.3. Under ARD_1, let $t < k$ be the last intervention day before k, or $t = 0$ if there is none. Then $g^{\mu}_{k-1} = g^{\mu}_{t}$ and $\zeta^{g}_{k-1} = \zeta^{g}_{t}$.

- If $t = 0$: $g^{\mu}_{0} = \zeta^{g}_{0}$ (the starting anchor), so $g^{\mu}_{k-1} - \zeta^{g}_{k-1} = 0 \leq \mu_{k-1} - \zeta^{\mu}_{k-1}$.
- If $t \geq 1$: $g^{\mu}_{t} = \mu_{t}$ and $\zeta^{g}_{t} = \zeta^{\mu}_{t}$ (Eq. (11)). On the days $j = t+1, \ldots, k-1$ there is no intervention: $\mu_{j} \geq \mu_{j-1} + \delta\mu_{j}$ and $\zeta^{\mu}_{j} = \zeta^{\mu}_{j-1} + \delta\mu_{j}$. So $\mu_{j} - \zeta^{\mu}_{j}$ does not decrease, and $\mu_{k-1} - \zeta^{\mu}_{k-1} \geq \mu_{t} - \zeta^{\mu}_{t} = g^{\mu}_{k-1} - \zeta^{g}_{k-1}$.

Rearranging gives $\mu_{k-1} - g^{\mu}_{k-1} \geq \zeta^{\mu}_{k-1} - \zeta^{g}_{k-1}$. The mean recursion is linear (Section 6.3); with an exact starting anchor, ζ^{μ} and ζ^{g} are the true means, so $\rho\,(\zeta^{\mu}_{k-1} - \zeta^{g}_{k-1}) = \mathbb{E}[D^{-} - D^{+}]$. □

A starting anchor above the true one, which the default may be (Section 6.3), models a weaker repair. For the first repair before any intervention of the horizon, the charged removal then follows the starting anchor and may be below the true removal. From the first intervention on, the reference gap $\zeta^{\mu} - \zeta^{g}$ equals the true gap, the sum of the mean increments since that intervention, so the bound holds again.

Under both laws, then, the reported repair cost $\sum C_R\, z$ is an upper bound on the expected repair cost of the schedule, with the exception above for a starting anchor above the true one. Under ARD_1 the tracked mean and the tracked anchor are both upper bounds, and their difference is still an upper bound on the reference difference: the anchor carries the slack that the mean had

at the last intervention, and the slack of the mean does not decrease until the next one. The bound is exact when the tracked mean has no slack.

Damage Regularisation

The scalar $u \geq 0$ measures the largest damage of one train on one day, over the whole horizon. The objective charges $C_D\, u$ (Section 5). The variable u and its rows are built only when $C_D > 0$. With $C_D = 0$ they are not built, and the input `penalty_type` is ignored. The input check requires $C_D \geq 0$ (Section 3.12): a negative C_D would reward damage.

The input `penalty_type` selects one of two forms. Both have one row per train and day.

- `inf_norm` (default), a linear row:

$$u \;\geq\; \sum_{\ell=1}^{L} \mu_{i\ell k} \qquad \forall\, i, k. \tag{42}$$

  At the optimum, $u = \max_{i,k} \sum_{\ell} \mu_{i\ell k}$.

- `quadratic`, a convex quadratic row:

$$u \;\geq\; \sum_{\ell=1}^{L} \mu_{i\ell k}^2 \qquad \forall\, i, k. \tag{43}$$

  The row is a rotated second-order cone, $u \cdot 1 \geq \|\mu_{i\cdot k}\|_2^2$, which Gurobi solves without the parameter `NonConvex`. It requires `reliability_impl = "exact"` at every cell (Section 3.12): the linear forms are chosen for a model without quadratic rows, and this row is quadratic.

Role. The term prices the worst train-level damage, so the minimisation spreads damage across the fleet. It also removes slack from the tracked means where u is binding (Section 6.4). Since $\mu_{i\ell k} \geq \zeta^{\mu}_{i\ell k} \geq 0$ (Proposition 6.3), both forms of u are upper bounds on the same quantity computed from the reference values.

Loop Constraint

The schedule of the operating phase is meant to repeat without end. One hard row per tracked quantity and per anchor makes every repetition safe:

$$s_{i\ell T} \;\leq\; s_{i\ell H_1} \qquad \text{for every cell } (i,\ell) \text{ and every coordinate } s \text{ of its full tracked state.} \tag{44}$$

The rows cover every tracked quantity of the cell (the mean, and the variance, R or K where the model tracks them) and, under ARD_1, every anchor. There is no soft loop penalty in the objective.

Endless schedule. Repeat the operating phase without end: day $T + nH_2 + h$, for $n \geq 0$ and $1 \leq h \leq H_2$, carries the binaries of day $H_1 + h$. It also carries the increments of day $H_1 + h$, because the operating profile is read cyclically with period H_2 (Section 3.7). So every repetition applies the same day maps $\varphi_{H_1+1}, \ldots, \varphi_T$. Write $\Phi_h = \varphi_{H_1+h} \circ \cdots \circ \varphi_{H_1+1}$ for the first h days of the operating phase and $\Phi = \Phi_{H_2}$ for the whole phase.

Theorem 6.17 (Perpetual safety)**.** *Let a solution satisfy every row of the problem. Then, for every cell and every day $k \geq 1$ of the endless schedule, the true statistics θ_k and the reference values ζ_k are at most a tracked value of the horizon that lies in $\mathcal{C}_{i\ell}$. Therefore $\zeta_k \in \mathcal{C}_{i\ell}$ and $\Pr(D_{i\ell k} > \tau_{i\ell}) \leq \varepsilon_{i\ell}$ on every day $k \geq 1$.*

Proof. Fix a cell. Let ξ_k be its tracked full state, and θ_k and ζ_k the true statistics and the reference trajectory of the endless schedule. By (M2) and Lemma 6.2, the induction of Proposition 6.3 gives $\theta_k \leq \zeta_k$ on every day $k \geq 1$.

- Days $k \le T$: $\zeta_k \le \xi_k$ (Proposition 6.3) and $\xi_k \in \mathcal{C}_{i\ell}$ (consequence on the horizon, Section 6.4).
- By Proposition 6.3 and Lemma 6.2, $\xi_{H_1+h} \ge \varphi_{H_1+h}(\xi_{H_1+h-1}) \ge \cdots \ge \Phi_h(\xi_{H_1})$ for $h = 1, \ldots, H_2$. With Eq. (44), $\Phi(\xi_{H_1}) \le \xi_T \le \xi_{H_1}$.
- Let $\Phi^n(\xi_{H_1}) \le \xi_{H_1}$ (true for $n = 0$). Then $\Phi^{n+1}(\xi_{H_1}) = \Phi(\Phi^n(\xi_{H_1})) \le \Phi(\xi_{H_1}) \le \xi_{H_1}$ by Lemma 6.2. So $\Phi^n(\xi_{H_1}) \le \xi_{H_1}$ for every $n \ge 0$.
- Day $k = H_1 + nH_2 + h$ with $n \ge 0$, $1 \le h \le H_2$:

$$\zeta_k = \Phi_h\big(\Phi^n(\zeta_{H_1})\big) \le \Phi_h\big(\Phi^n(\xi_{H_1})\big) \le \Phi_h(\xi_{H_1}) \le \xi_{H_1+h} \in \mathcal{C}_{i\ell}.$$

Therefore, on every day $k \ge 1$, $\theta_k \le \zeta_k \le s$ for a tracked value $s \in \mathcal{C}_{i\ell}$. Since $\mathcal{C}_{i\ell}$ is a down-set, $\zeta_k \in \mathcal{C}_{i\ell}$, and (M3) gives $\Pr(D_{i\ell k} > \tau_{i\ell}) \le \varepsilon_{i\ell}$. □

The theorem holds for every model and both repair laws, because it uses only the facts (M1)–(M3) and Lemma 6.2. It holds in both encodings and in every reliability form. It needs no relation between H_1 and H_2: the transitory phase may be shorter or longer than the operating phase, because H_1 only marks the day at which the repeated block starts. Slack in the tracked values does not weaken it: the proof compares the reference trajectory with the tracked values, whatever slack they carry.

Remark (why the loop row covers the full state). The third step of the proof needs $\Phi(\xi_{H_1}) \le \xi_{H_1}$ in every coordinate, anchors included. Example: law ARD_1, $\rho = 0.5$, the first phase ends at $(\mu, g) = (0.4, 0.1)$, and the operating phase is a mission (+0.1), a repair, a mission (+0.05). The phase ends at $(0.35, 0.3)$, since the repair gives $0.5 \cdot 0.5 + 0.5 \cdot 0.1 = 0.3$. The row on the mean holds ($0.35 \le 0.4$); the row on the anchor fails ($0.3 > 0.1$). Each repetition raises the state after the repair by 0.075, so the mean grows without bound: the third repetition already peaks at 0.525, above the checked peak 0.5. A loop row on the mean alone would accept this unsafe schedule.

Remark (no loop row on a scalar bound). The loop row must compare the tracked quantities one by one. A single comparison of a scalar function of them, such as the failure-probability bound $U(\mu, v)$, does not chain: U takes the same value at many states, and its level sets are not invariant under the day maps. Example (Cantelli, $U = v/(v + (\tau - \mu)^2)$, $\tau = 10$, $\rho = 0.5$, law ARD_∞; the operating phase is one repair day followed by mission days that add 3 to the mean and 3.99 to the variance in total): from $(\mu, v) = (4, 9)$, $U = 0.2000$ at the end of the first phase and $U = 0.1997$ at the end of the operating phase, so a scalar row would accept. The next repetition gives $U = 0.2151 > 0.2$, and U tends to 0.2495. Eq. (44) rejects the schedule, since the mean rises from 4 to 5. The loop row states an order between tracked quantities; it is not a stochastic order between damage laws.

Proposition 6.18 (Replacement under ARD_1)**.** *Let a cell follow* ARD_1 *with* $\rho_{i\ell} < 1$*, and let its train fly, in the operating phase, at least one mission with a positive mean increment for the cell. Then every feasible solution has at least one replacement of the cell in the operating phase.*

Proof. Let the cell have no replacement in the operating phase. By the anchor rows, $g_k = g_{k-1}$ on a day without intervention and $g_k = \mu_k$ on a repair day. Every day satisfies $\mu_k \ge g_k$: on an intervention day with equality, and on other days because μ does not decrease and g stays constant (with $g_0 \le \mu_0$, Section 6.3). On a repair day k of the operating phase,

$$g_k = \mu_k \ge (1 - \rho)\,\mu_{k-1} + \rho\, g_{k-1} = g_{k-1} + (1 - \rho)\,(\mu_{k-1} - g_{k-1}).$$

So $g_T \ge g_{H_1} + (1 - \rho) \sum (\mu_{k-1} - g_{k-1})$, the sum over the repair days of the operating phase, each term non-negative. The anchor row $g_T \le g_{H_1}$ and $\rho < 1$ force every term to 0. Then every repair day gives $\mu_k \ge \mu_{k-1}$, every other day gives $\mu_k \ge \mu_{k-1} + \delta\mu_k$, and

$$\mu_T \ge \mu_{H_1} + \textstyle\sum_{k=H_1+1}^{T} \delta\mu_k > \mu_{H_1},$$

which violates the row $\mu_T \le \mu_{H_1}$. Therefore the cell is replaced at least once in the operating phase. □

The requirement is not an artefact of the rows. Without a replacement, each repair leaves the share $1-\rho$ of the damage since the last intervention in place, so the reference mean grows without bound over the repetitions, as in the first remark above. With $\rho = 1$ a repair returns the state to the anchor, and no replacement is needed. The Wiener model needs a replacement in the operating phase for another reason, its variance (Section 6.9).

Full Optimization Problem

This section collects the whole problem in one place. Each row is stated once, in the section that the reference names; this section does not repeat the rows.

Objective. Minimise Eq. (4), the mode chosen by `objective_mode`, with the day cost Eq. (2).

Fleet rows. For every train i and day k: one activity, Eq. (5). For every mission $j \ge 1$ and day k: coverage, Eq. (6). For every cell and day: depot-day compatibility, Eq. (7).

Rows of a cell. For every cell (i, ℓ), with the model, repair law, tail bound, encoding and reliability form of the cell:

1. for every tracked quantity s of Table 24: the generic rows Eqs. (12)–(14) (`encoding = "bigm"`) or Eqs. (16)–(17) (`encoding = "indicator"`), with the repair map Eq. (10) and the factor a of the table; the Wiener variance may use the single row Eq. (30);
2. under ARD_1, for every tracked quantity except the Wiener variance: the anchor rows Eq. (15) or Eq. (18);
3. the reliability row of the table, on every day $k = 1, \dots, T$, in the form `exact`, `tangent` (Eq. (20)) or `pwl` (Eqs. (21)–(23)) for a quadratic row, and in its exact form for a linear row;
4. the loop rows Eq. (44), one per tracked quantity and per anchor;
5. where $C_{R,i\ell} > 0$: the repair-cost rows Eq. (41).

Damage regularisation. Where $C_D > 0$: Eq. (42) (`inf_norm`) or Eq. (43) (`quadratic`), for every train and day.

Domains. The variables and their domains are those of Section 4; a variable that no row uses is not built.

Table 24: Tracked quantities and reliability row of each kind of cell. The factor a is the repair factor of Eq. (10); M_s is the natural range of Section 6.4.

Cell	Tracked quantities (factor a)	Natural ranges M_s	Reliability row
Gaussian	μ $(1-\rho)$, v $((1-\rho)^2)$	τ, $V_{\max}$	Eq. (26), quadratic
Inverse Gaussian	μ $(1-\rho)$	τ	Eq. (28), linear
Wiener	μ $(1-\rho)$, v (1, no anchor)	τ, $V_{\max}$	Eq. (26), quadratic
Gamma	μ $(1-\rho)$	τ	Eq. (34), linear
Rainflow, Markov	μ $(1-\rho)$	τ	Eq. (35), linear
Rainflow, Cantelli	μ $(1-\rho)$, v $((1-\rho)^2)$	τ, V^{C}	Eq. (36), quadratic
Rainflow, Hoeffding	μ $(1-\rho)$, R $((1-\rho)^2)$	τ, $2\tau^2/\kappa$	Eq. (37), quadratic
Rainflow, Bernstein	μ $(1-\rho)$, v $((1-\rho)^2)$	τ, V^{B}	Eq. (39), quadratic
Rainflow, Chernoff	μ $(1-\rho)$, K $(1-\rho)$	τ, $s\,\tau - \kappa$	Eq. (40), linear

Problem class. The problem is a mixed-integer linear program when every quadratic reliability row uses the `tangent` or `pwl` form and the damage penalty is `inf_norm` or absent. A quadratic

row in the `exact` form makes it a nonconvex mixed-integer quadratically constrained program, which Gurobi solves with `NonConvex` = 2. The `quadratic` damage penalty alone makes it a convex one (Section 6.13).

Output Specification

The function `solve()` returns one result dictionary. If its argument `results_path` is not `None`, the function also writes the same content to a file. Every call solves one problem, with the horizon H of the input; a solve for several horizons calls `solve()` once per horizon (Section 9).

Result Dictionary

The array values are `numpy.ndarray` objects. The result holds no Gurobi object. In the column *Shape*, a dash marks a scalar or a string.

Key	Shape	Description
Status and quality		
`status`	—	Status of the solve: `optimal`, `time_limit`, `suboptimal`, `interrupted`, `solution_limit`, `node_limit`, `iteration_limit`, `infeasible`, `inf_or_unbounded`, `unbounded`, or `gurobi_status_`n for any other Gurobi status code n
`objective`	— or `null`	Objective value of the best solution found; `null` if there is none
`mip_gap`	— or `null`	Relative MIP gap of the best solution at the end of the solve; `null` if there is no solution. The input key `mip_gap` is the gap that was requested
`bound`	— or `null`	Best bound of Gurobi, if it is available
Problem		
`F`, `M`, `L`	—	Numbers of trains, missions and components
`H`	integer or $[H_1, H_2]$	Horizon as written in the input
`H1`, `H2`, `T`	—	Lengths H_1 and H_2 of the two phases, and $T = H_1 + H_2$
`model`	$F \times L$ list of strings	Degradation model of each cell, as in the input
`component_names`	list of L strings	Names of the components
`tau`, `mu_0`, `v_0`	$F \times L$	Threshold, initial mean and initial variance of each cell, after broadcasting
`bound_method`, `repair_model`, `reliability_impl`, `encoding`	string, or $F \times L$ list of strings	The selector of each cell after broadcasting: one string if all cells agree, else a list. `bound_method` is `null` at a cell that is not rainflow
`objective_mode`, `evaluation_horizon`	—	The objective mode, and E if it was given
Solution (`null` if no solution was found)		
`x`	$F \times (M{+}1) \times T$	Assignment: $x_{ijk} = 1$ if train i flies mission $j \geq 1$ on day k, and $x_{i0k} = 1$ if it is in the depot. A train with $x_{ijk} = 0$ for every $j \in \{0, \ldots, M\}$ is idle
`m`	$F \times L \times T$	Repair: $x^m_{i\ell k}$
`r`	$F \times L \times T$	Replacement: $x^r_{i\ell k}$; zero if `allow_replacement` is `false`
`idle`	$F \times L \times T$	$\text{idle}_{i\ell k} = x_{i0k} - x^m_{i\ell k} - x^r_{i\ell k}$: component ℓ is in the depot without an intervention
`mu`	$F \times L \times T$	Tracked mean degradation

Key	Shape	Description
`v`	$F \times L \times T$	Tracked variance. `NaN` in memory and `null` in a file at cells that track no variance (IG, Ga, RF with Mk, Ho or Ch)
`z`	$F \times L \times T$	Repair-cost variable. `NaN` in memory and `null` in a file at cells with $C_{R,i\ell} = 0$, where the variable is not built. On the transitory days of the mode `operating_average` the value is computed after the solve (below)
`R`, `K`	$F \times L \times T$	Tracked Hoeffding budget R and Chernoff quantity K. `NaN` in memory and `null` in a file at the cells that do not track the quantity: R is tracked at RF (Ho) cells, K at RF (Ch) cells
`g_mu`, `g_v`, `g_R`, `g_K`	$F \times L \times T$	Tracked anchors $g^{\mu}_{i\ell k}$, $g^{v}_{i\ell k}$, $g^{R}_{i\ell k}$ and $g^{K}_{i\ell k}$: the value of the state after the last intervention up to day k. `NaN` in memory and `null` in a file at the cells that do not track the anchor: only cells with `repair_model ard1` track one, `g_mu` for every model, `g_v` at G and RF (Ca, Be) cells, `g_R` at RF (Ho) cells, `g_K` at RF (Ch) cells
`u`	—	Damage regularisation value (below)
Cost report (`null` if no solution was found)		
`step_costs`	T	Cost c_k of each day, Equation (47)
`J_initialization`, `J_op`, `J_op_average`	—	J_{init}, J_{op} and J_{op}/H_2, Equation (48)
`peak_damage`, `damage_penalty`	—	u and $C_D\, u$
`operating_objective`, `J_total`	—	$J_{\text{op}}/H_2 + C_D\, u$ and $J_{\text{init}} + J_{\text{op}} + C_D\, u$
`projected_evaluation_cost`, `evaluation_objective`	— or `null`	$J_{\text{init}} + (E - H_1)\, J_{\text{op}}/H_2$ and the same plus $C_D\, u$; `null` if `evaluation_horizon` is not given
`component_costs`	mapping	Cost coefficients used: `C_M` and `C_D` (scalars), `C_R` and `C_rep` ($F \times L$)
Reports		
`performance`	mapping	Sizes and times of the solve (Section 8.3)
`optimization_progress`	list of mappings	Progress of the incumbent and the bound (Section 8.3)
`warm_start`, `relaxation_warm_start`	mappings	What each warm start did (Section 8.3)

Rules for the Values

Solutions and status. The solution arrays are present whenever Gurobi has found at least one solution, and then `status` tells how the solve ended. For example, a solution found before the time limit is reported with the status `time_limit`. If no solution was found, the solution arrays and the cost report are `null`, and `objective` and `mip_gap` are `null`. The four reports of the last block of the table are present in both cases.

Tracked values. The arrays `mu`, `v`, `R`, `K`, `g_mu`, `g_v`, `g_R` and `g_K` hold the values of the tracked variables of the model, and `z` holds the repair-cost variable. The result carries every tracked state and anchor, so that a validator can check every loop row of Section 6.14. The dynamics rows are one-sided (Section 6.4), so these values, and the costs computed from them, are upper

bounds, not exact values.

Repair cost on transitory days. In the mode `operating_average`, the variable z is not built on the days of the transitory phase, because their cost does not enter the objective (Section 6.12). For these days, at a cell with $C_{R,i\ell} > 0$, the result holds the value computed after the solve from the arrays `mu` and `g_mu`:

$$z_{i\ell k} = \begin{cases} \rho_{i\ell}\left(\mu_{i\ell,k-1} - g^{\mu}_{i\ell,k-1}\right) & \text{if } x^{m}_{i\ell k} = 1, \\ 0 & \text{otherwise,} \end{cases} \qquad k = 1, \dots, H_1, \tag{45}$$

with $\mu_{i\ell 0} = \mu_{0,i\ell}$, $g^{\mu}_{i\ell 0}$ the starting anchor, and $g^{\mu} \equiv 0$ under `ardinf`. It is the amount that the repair removes, in mean units, computed on the tracked values. The cost report uses these values.

Damage regularisation. The value u is recomputed from the arrays `mu`,

$$u = \max_{i,k} \sum_{\ell=1}^{L} \mu_{i\ell k} \quad (\texttt{inf_norm}), \qquad u = \max_{i,k} \sum_{\ell=1}^{L} \mu^{2}_{i\ell k} \quad (\texttt{quadratic}). \tag{46}$$

It is reported also when $C_D = 0$, where the variable u is not built.

Depot, idle and intervention. A row of zeros $x_{i1k} = \cdots = x_{iMk} = x_{i0k} = 0$ is an idle day of train i: no mission and no depot. On a depot day ($x_{i0k} = 1$) each component is repaired, replaced or in the depot without an intervention; the last case is `idle`. Hence $x^{m}_{i\ell k} + x^{r}_{i\ell k} + \text{idle}_{i\ell k} = x_{i0k}$.

Cost report. The cost of day k and the phase costs are

$$c_k = C_M \sum_{i=1}^{F} x_{i0k} + \sum_{i=1}^{F}\sum_{\ell=1}^{L} C_{R,i\ell}\, z_{i\ell k} + \sum_{i=1}^{F}\sum_{\ell=1}^{L} C_{\text{rep},i\ell}\, x^{r}_{i\ell k}, \tag{47}$$

$$J_{\text{init}} = \sum_{k=1}^{H_1} c_k, \qquad J_{\text{op}} = \sum_{k=H_1+1}^{T} c_k. \tag{48}$$

The value `objective` is the value of the model objective. Up to the tolerance of the solver, it equals `J_total` if `objective_mode` is `total`, `operating_objective` if it is `operating_average`, and `evaluation_objective` if it is `evaluation_total`.

Reports

`performance`. The mapping has these entries; an entry that Gurobi does not give is `null`.

Group	Entries
Size of the model	`gurobi_version`, `variables`, `continuous_variables`, `integer_variables`, `binary_variables`, `linear_constraints`, `general_constraints`, `indicator_constraints`, `quadratic_constraints`, `nonzeros`
Work of Gurobi	`solutions_found`, `gurobi_runtime_seconds`, `branch_and_bound_nodes`, `simplex_iterations`, `barrier_iterations`, `work_units`
Wall-clock times, in seconds	`model_construction_seconds`, `optimizer_call_seconds`, `solution_extraction_seconds`, `backend_wall_seconds`
Settings	`requested_time_limit_seconds`, `requested_mip_gap`, `threads`, `seed`

`optimization_progress`. A list of records with the entries `runtime_seconds`, `incumbent`, `best_bound`, `relative_gap`, `nodes` and `solutions`. The incumbent, the bound and the gap are `null` while Gurobi has none. A record is made at the first progress report of Gurobi, then at least

`progress_interval_seconds` after the previous record, and whenever the number of solutions changes; the last record is the final state of the solve.

`warm_start`. A mapping with the entries `applied` (`true` when at least one value was set), `values` (the number of values set), `mode` (`"start"`, `"hint"` or `"both"`, and `null` without a source), `source_horizon` (the pair H_1, H_2 of the source, or `null`) and `completion`. The entry `completion` is a mapping with `requested`, `status`, `fixed_integer_variables`, `values_copied`, `runtime_seconds` and `iis_constraints`. The value of `status` is `"not_requested"`, `"completed"`, `"infeasible"`, `"infeasible_or_unbounded"`, `"time_limit"`, or `"status_`q`"` with the Gurobi status code q. The entry `iis_constraints` is a list of at most 50 names of rows of an irreducible infeasible subset; it is empty unless the status is `"infeasible"`. Section 10 defines the mechanisms.

`relaxation_warm_start`. A mapping with the entries `enabled`; `status` (`"disabled"` when the option is false, `"skipped"` when it is true and a start or a hint was applied, otherwise the Gurobi status of the relaxed copy as a string); `relaxation_objective`; `values_copied`; and `runtime_seconds`.

Result Files

The function `solve` writes the result to the file `results_path`.

- The default value of `results_path` is `"output.yaml"`. The value `None` writes no file and only returns the dictionary.
- A path without extension gets the extension `.yaml`. The extensions `.yaml` and `.yml` give YAML, `.json` gives JSON, and `.h5` and `.hdf5` give HDF5. A path with any other extension raises a `ValueError`.
- The extension, and the directory of the path, are checked before the model is solved, so that a long solve is not lost. The directory must exist and be writable.
- YAML and JSON files hold plain data: lists, numbers, strings and `null`. `NaN` is written as `null`. The files load with `yaml.safe_load` and `json.load`.
- A key whose value is `None` is not written, except `objective`, which is written as `null`.
- The layout of an HDF5 file is that of Section 13.2.

Horizon Sweeps

A single call of `solve()` uses one horizon pair $H = [H_1, H_2]$ (Section 2). A *horizon sweep* solves one scenario for several pairs and compares the solved cases. Two functions implement the sweeps: `sweep_operating_horizons` varies H_2 for one H_1, and `sweep_horizon_grid` varies H_1 and H_2 on a grid. No input key starts a sweep, and `solve()` never runs one. Both functions are exported by the package (Section 11.1).

Cost Measures and the Ranking Criteria

Let x, z, x^r be the solution of a case. The cost of day k is

$$c_k = C_M \sum_{i=1}^{F} x_{i0k} + \sum_{i=1}^{F} \sum_{\ell=1}^{L} \left(C_{R,i\ell}\, z_{i\ell k} + C_{\mathrm{rep},i\ell}\, x^r_{i\ell k} \right), \tag{49}$$

where $z_{i\ell k} = 0$ for a cell with $C_{R,i\ell} = 0$ (no variable z is built, Section 6.12). The cost of the transitory phase and the cost of the operating phase are

$$J_{\text{init}} = \sum_{k=1}^{H_1} c_k, \qquad J_{\text{op}} = \sum_{k=H_1+1}^{T} c_k, \qquad \bar{J}_{\text{op}} = J_{\text{op}}/H_2. \tag{50}$$

The peak damage is $u = \max_{i,k} \sum_{\ell} \mu_{i\ell k}$, and the damage penalty is $C_D\, u$. The result keys are `J_initialization`, `J_op`, `J_op_average` ($= \bar{J}_{\text{op}}$), `peak_damage` and `damage_penalty` (Section 8).

A solve with `objective_mode = "operating_average"` minimises $\bar{J}_{\text{op}} + C_D\, u$ (Section 5). The transitory phase has no cost in this objective, but it is still constrained. With `objective_mode = "evaluation_total"` and an *evaluation horizon* $E > H_1$ (days), the solve minimises $P_E + C_D\, u$, where P_E is the *projected cost*

$$P_E = J_{\text{init}} + (E - H_1)\, \bar{J}_{\text{op}}. \tag{51}$$

P_E is the cost of the transitory phase, then of the operating phase repeated until day E. Two result keys carry it: `projected_evaluation_cost` holds P_E, and `evaluation_objective` holds $P_E + C_D\, u$.

A case is *feasible* when its result holds a solution and its status is not `"infeasible"`, not `"unbounded"` and not `"inf_or_unbounded"`. The *MIP gap* is the relative gap between the objective of the best solution found and the best bound (MIP: mixed-integer program). A case is *proven* when it is feasible and Gurobi reports the status `"optimal"` at the MIP gap used for that case, that is, the run option `mip_gap` of Section 3.10. A proven case is optimal within that gap, not exactly optimal. The sweep report records the requested gap of each case, so a reader sees how strong the proof is.

Ranking criteria. `sweep_operating_horizons` ranks cases by $\bar{J}_{\text{op}}$. The loop constraint (Section 6.14) makes the operating phase repeat, so $\bar{J}_{\text{op}}$ is the cost per day of the repeating regime. It is comparable across values of H_2, whereas a raw objective over T days is not. `sweep_horizon_grid` ranks by $\bar{J}_{\text{op}}$ when no evaluation horizon is given. With an evaluation horizon it ranks by P_E, which also prices the transitory phase, so it is the criterion that compares different values of H_1. In both functions the *best proven case* is the proven case with the smallest ranking value, and the *best feasible case* is the feasible case with the smallest ranking value. A tie goes to the case evaluated first.

Remark (what the proof certifies). The status `"optimal"` certifies the objective that the solve minimised. This objective is $\bar{J}_{\text{op}} + C_D\, u$ in the mode `operating_average` and $P_E + C_D\, u$ in the mode `evaluation_total`, while the ranking value is $\bar{J}_{\text{op}}$ or P_E alone. The sweep therefore ranks by cost and uses the damage penalty only inside each solve.

`sweep_operating_horizons`

```
sweep_operating_horizons(
    input_path, operating_horizons, *, output_path=None,
    stop_on_gradient=False, gradient_tolerance=1e-3,
    flat_gradients_required=2, minimum_cases=3,
    maximum_mip_gap_for_stopping=None,
    warm_start=False, warm_start_mode="start", n_workers=1,
    objective_mode="operating_average", evaluation_horizon=None) -> dict
```

Argument	Meaning

`input_path`	Input file (YAML, JSON or HDF5). H_1 is the first entry of the input key `H`, or its value when `H` is a scalar. The second entry of `H` is ignored.
`operating_horizons`	Candidate values of H_2: positive integers, at least one, no duplicates. The function evaluates them in increasing order.
`output_path`	Report file, with the suffix `.yaml` or `.yml`. The function creates missing parent folders. `None`: no file is written.
`stop_on_gradient`, `gradient_tolerance`, `flat_gradients_required`, `minimum_cases`, `maximum_mip_gap_for_stopping`	Early stop (Section 9.3). Required: `gradient_tolerance` ≥ 0 and finite, `flat_gradients_required` ≥ 1, `minimum_cases` ≥ 2, and `maximum_mip_gap_for_stopping` is `None` or a finite number ≥ 0.
`warm_start`, `warm_start_mode`	Warm start between cases (Section 10).
`n_workers`	Number of cases solved at the same time (Section 9.5).
`objective_mode`, `evaluation_horizon`	Written into every case input, replacing the values of the input file. The mode `evaluation_total` needs $E > H_1$.

Behaviour.

1. The function reads the input file with the reader of `solve()` (Section 11.1). It accepts the same formats and raises the same file errors. The data must be a mapping (`TypeError` otherwise).
2. The function checks every argument of the table above before it solves any case. An invalid argument raises `ValueError`. An argument error is never recorded as a case error. It also raises `ValueError` when `H` is a list of a length other than two, or when `warm_start` is true and `n_workers` > 1, or when `stop_on_gradient` is true and `n_workers` > 1.
3. For each candidate H_2, the case input is the input data with `H` $= [H_1, H_2]$, `objective_mode` and (if given) `evaluation_horizon` replaced. Nothing else changes: the function does not interpolate time-indexed data. A profile whose length differs from H_2 (and does not broadcast) is rejected by the input checks of Section 3.12. This rejection is an error of that case only.
4. The case is solved as `solve()` solves an input file, with `results_path=None`. The sweep keeps the summary of the case (Section 9.4), not its schedule.
5. After each case the function appends the case row, annotates the gradient (Section 9.3), tests the stopping rule, and writes a checkpoint when `output_path` is given (Section 9.4).
6. At the end the function writes the final report and returns it.

Early Stop

Let $\bar{J}_n$ and $H_{2,n}$ be the cost per day and the operating horizon of the n-th evaluated case. When cases $n-1$ and n are both feasible, the *relative gradient* of case n is

$$\hat{g}_n = \frac{\bar{J}_n - \bar{J}_{n-1}}{(H_{2,n} - H_{2,n-1})\,\max\{\,|\bar{J}_{n-1}|,\ \varepsilon_{\text{mach}}\}}, \tag{52}$$

with $\varepsilon_{\text{mach}}$ the machine precision of double-precision numbers. Case n is classified `"increase"` if $\hat{g}_n >$ `gradient_tolerance`, `"decrease"` if $\hat{g}_n <$ −`gradient_tolerance`, and `"flat"` otherwise. If one of the two cases is not feasible, or if $n = 1$, the class is `"unavailable"`. A case *qualifies* when it reports a finite MIP gap that is at most its gap threshold. The threshold of a case is `maximum_mip_gap_for_stopping` if that argument is a number, and otherwise the MIP gap used for that case (the entry `requested_mip_gap` of its row). With the default, a case qualifies when Gurobi has met the gap that was requested for it. The gradient of case n is *eligible to stop* when

both cases $n-1$ and n qualify. The function annotates every case with these values, also when `stop_on_gradient` is false.

With `stop_on_gradient = True` the function stops after case n when $n \geq$ `minimum_cases`, the gradient of case n is eligible to stop, and one of these holds:

- the class of case n is `"increase"` (stopping reason `"cost_increase"`);
- the class of case n is `"flat"`, and the last `flat_gradients_required` cases up to n all have the class `"flat"` and an eligible gradient (stopping reason `"flat_gradient"`).

The last candidate in `operating_horizons` is always a hard limit. A case that does not qualify, for example one that stopped at the time limit above its requested gap, is recorded, but it cannot trigger a stop.

Report, Checkpoint and Errors

The report is a mapping of plain Python types. The file is written with a safe YAML writer, so it contains no arrays and no Python objects.

Key	Content
`input`	Path of the input file, as given.
`complete`	`false` in a checkpoint, `true` in the final report (also after an early stop).
`last_completed_H2`	Largest H_2 among the evaluated cases; `null` before the first case.
`objective`, `varied_parameter`	Text that names the minimised objective, and the string `"H2"`.
`fixed_dimensions`, `H1`	F, M, L, H_1.
`planned_operating_horizons`, `evaluated_operating_horizons`	The sorted candidates, and the H_2 of the cases in `cases`.
`warm_start`, `n_workers`	`enabled`, `mode`; number of workers.
`stopping_rule`	`enabled`, `relative_gradient_tolerance_per_H2`, `flat_gradients_required`, `minimum_cases`, `maximum_mip_gap_for_stopping` (`null` when the default applies), `reason` (`"in_progress"`, `"candidate_range_exhausted"`, `"cost_increase"` or `"flat_gradient"`), `stopped_early` (the run is complete and fewer cases than planned were evaluated).
`cases`	One row per evaluated case, ordered by H_2 (rows below).
`formulation_growth`	Between the first and the last case that carry formulation counts: `from_H2`, `to_H2`, `added_time_steps`, and for each count key q of the row entry `formulation`, the entries `added_`q and q`_per_added_time_step`. `null` when fewer than two cases carry counts or both have the same T.
`best_proven_H2`, `best_proven_J_op_average`	Best proven case (`null` if none).
`best_feasible_H2`, `best_feasible_J_op_average`, `best_feasible_status`	Best feasible case (`null` if none).

Entry of a case row	Content
`H1`, `H2`, `T`, `dimensions`	Horizons and the sizes F, M, L, H_1, H_2, T.
`status`, `objective_mode`, `evaluation_horizon`	As in the result of the case.

`objective`, `J_op`, `J_op_average`, `J_initialization`, `peak_damage`, `damage_penalty`, `operating_objective`, `evaluation_objective`, `J_total`, `projected_evaluation_cost`	Result keys of the case (Section 8).
`objective_bound`, `mip_gap`, `absolute_mip_gap`	Best bound; relative gap; the difference $\max\{0,$ `objective` $-$ `objective_bound`$\}$.
`requested_mip_gap`, `time_limit_seconds`, `optimizer_seconds`, `solutions_found`	The run options used and the optimizer outcome.
`warm_start`	The warm-start report of the case (Section 10).
`formulation`	Model size from the `performance` entry of the result: `variables`, `continuous_variables`, `integer_variables`, `binary_variables`, `linear_constraints`, `general_constraints`, `indicator_constraints`, `quadratic_constraints`, `nonzeros`.
`timing`	`model_construction_seconds`, `optimizer_call_seconds`, `solution_extraction_seconds`, `backend_wall_seconds`, `branch_and_bound_nodes`, `simplex_iterations`, `work_units`.
`cost_gradient_per_H2`, `relative_cost_gradient_per_H2`, `gradient_classification`, `gradient_stopping_eligible`	Equation (52) and its class (Section 9.3).

Checkpoint. When `output_path` is given, the function rewrites the report after every case. Until the sweep ends, the report has `complete = false` and the stopping reason `"in_progress"`. If a job ends before the run is complete, the file is a valid report of the cases evaluated so far. In parallel mode the function recomputes the gradient annotations over the rows ordered by H_2 at each rewrite.

Error capture. The function catches every exception (class `Exception`) that a case raises after the arguments are accepted, for example a failed input check of the case or a solver error such as a missing licence. It logs the error at level ERROR with H_1 and H_2, records a row with the status `"error: "` followed by the message and `null` in every other entry except the sizes, and continues with the next case. Such a row is neither feasible nor proven. An error does not stop the sweep, and a warm start never uses the result of a failed case (Section 10).

Parallel Cases

With `n_workers` > 1 the function solves up to `n_workers` cases at the same time, each in its own process with its own Gurobi environment. Each running solve uses one Gurobi licence token, so a licence with one token allows only `n_workers` $= 1$. The cases do not use each other's results, so the option is refused together with warm starts between cases, and together with the early stop, which needs the preceding cases before it starts the following ones. Rows arrive in the order in which the cases finish; the report orders them by H_2.

`sweep_horizon_grid`

```
sweep_horizon_grid(
    input_path, transitory_horizons, operating_horizons, *,
    output_path=None, warm_start=True, evaluation_horizon=None,
    **sweep_options) -> dict
```

The function evaluates every pair (H_1, H_2) of the two lists. Both lists contain positive integers; the function sorts them and removes repeated values. If `evaluation_horizon` is given, it must exceed every candidate H_1 (`ValueError` otherwise). The keyword options in `sweep_options` are those of Section 9.2 except `objective_mode`, which the grid sets itself. Because `warm_start` is true by default, a call with `n_workers` > 1 must also pass `warm_start=False`.

For each H_1 in increasing order, the function runs `sweep_operating_horizons` on the input data with that H_1 and the list of H_2. It sets `objective_mode` to `"evaluation_total"` when an evaluation horizon is given, and to `"operating_average"` otherwise. The first case of each H_1 starts without warm start; the following cases use the nearest preceding case of the same H_1 that has a solution. The early stop applies within each H_1 and uses $\bar{J}_{\mathrm{op}}$ as in Section 9.3. Case errors are captured as in Section 9.4. The function rewrites the grid report after every case of every H_1 (with `complete = false`) and at the end.

Key of the grid report	Content
`input`, `complete`	As in Section 9.4.
`objective`, `comparison_metric`	Text that names the minimised objective; the ranking key, `"projected_evaluation_cost"` (P_E) or `"J_op_average"`.
`mip_certificate_metric`	The string `"objective"`: the proof rule refers to the solved objective (remark in Section 9.1).
`evaluation_horizon`	E, or `null`.
`varied_parameters`, `fixed_dimensions`	`["H1", "H2"]`; F, M, L.
`planned_transitory_horizons`, `planned_operating_horizons`	The sorted candidates.
`warm_start_enabled`	The value of `warm_start`.
`H1_sweeps`	The report of each operating sweep, one per H_1. Its key `input` names the input file of the grid.
`cases`	All case rows, ordered by H_1 and then H_2.
`best_proven`, `best_feasible`	`null`, or a mapping with `H1`, `H2`, `T`, `J_initialization`, `J_op_average`, `peak_damage`, `damage_penalty`, `objective`, `operating_objective`, `evaluation_objective`, `projected_evaluation_cost`, `status`, `mip_gap` of the selected case.

Warm Starts

A *warm start* gives Gurobi values for the binary variables, so that the search begins from a known schedule. A warm start changes the time of a solve, not the model: it does not change the feasible set or the objective. The package has four mechanisms:

1. a *start* from the schedule of a previous result (Gurobi attribute `Start`), with an optional completion;
2. *hints* from the schedule of a previous result (Gurobi attribute `VarHintVal`);
3. a *relaxation start*, which needs no previous result;
4. a *greedy heuristic*, which constructs a schedule from the input and returns it in the form of item 1.

Interface

`solve()` (Section 11.1) has two keyword arguments for warm starts: `warm_start` (default `None`) and `warm_start_mode` (default `"start"`). The argument `warm_start` is a *source*: a dictionary with the keys of a result, or the dictionary that the greedy heuristic returns (Section 10.7). A source must carry the arrays `x` (shape $F' \times (M'+1) \times T'$) and the integers `H1`, `H2` with

$T' = H'_1 + H'_2$; it may carry `m` and `r` (shape $F' \times L' \times T'$). A source without a solution (`x` is `null`) is accepted and has no effect. A result with a solution found before a time limit is a valid source. The sizes F', M', L' may differ from the problem's sizes: the values are set for the indices that both have.

`warm_start_mode` is `"start"`, `"hint"` or `"both"`; another value raises `ValueError`. The relaxation start is selected by the run options `relaxation_warm_start` and `relaxation_time_limit` (Section 3.10).

Day Mapping

The source has the horizons H'_1, H'_2 and the problem has H_1, H_2. Day $k \in \{1, \dots, T\}$ of the problem takes the values of the source day

$$\sigma(k) = \begin{cases} k, & k \le \min\{H_1, H'_1\}, \\ H'_1 + 1 + \big(\max\{0,\ k - H_1 - 1\} \bmod H'_2\big), & \text{otherwise.} \end{cases} \tag{53}$$

So the transitory days keep their index, and the operating days repeat the operating phase of the source cyclically. A shorter operating phase can seed a longer one. When $H_1 > H'_1$, the days $H'_1 + 1, \dots, H_1$ all take the first operating day of the source. When the source has the same sizes F, M, L, the mapped schedule copies whole days of the source, so it keeps the one-activity rows, the mission coverage and the depot-day rows of every day. It need not satisfy the dynamics or the loop rows of the horizons of the problem. For each day k, train i and cell (i, ℓ) the mapped values are $x_{ijk} \leftarrow x'_{ij\sigma(k)}$ for $j = 0, \dots, M$, $x^m_{i\ell k} \leftarrow m'_{i\ell\sigma(k)}$ (value 0 if the source has no `m`), and $x^r_{i\ell k} \leftarrow r'_{i\ell\sigma(k)}$ when replacement is allowed (`allow_replacement`, Section 6.5) and the source has `r`. Every value is rounded to the nearest integer. Each other binary variable that these values fix through its defining rows (the selector $n_{i\ell k}$ of the indicator form, Eq. (16)) receives that value.

Start, Hints and Completion

Mode `"start"`. The solve sets the attribute `Start` of every mapped variable (Section 10.2). The continuous variables get no value. Gurobi tries to complete the start by itself, with limited effort, and discards a start that it cannot make feasible.

Completion. If the source has the key `complete_binary_start` with the value `true`, the solve completes the start before it optimises. The time limit of the completion is the source key `completion_time_limit` (seconds, default 60). The steps are:

1. Copy the model. In the copy, fix every binary variable that received a `Start` value to that value.
2. Optimise the copy. All binary values are fixed, so the copy has only continuous variables.
3. If the copy has a solution, set the attribute `Start` of every variable of the original model to the value in the copy. The start is then a complete, feasible point of the original model.
4. Otherwise record the status of the copy. If the copy is infeasible, compute an *irreducible infeasible subset* (IIS) of the copy: a set of rows that is infeasible, and that becomes feasible when any one row is removed (Gurobi function `computeIIS`). Record the names of its linear rows and general constraints, at most 50 names.

The solve never relaxes or changes the original model, and it never stops because a completion failed: the optimisation starts with the binary values only. A completion is requested by the source, not by an argument of `solve()`. The greedy heuristic requests it.

Mode `"hint"`. The solve sets the attribute `VarHintVal` of every mapped variable to its mapped value, and sets no `Start` and no completion. Gurobi uses hints to guide its heuristics and its branching during the whole solve. A hint needs no feasible point, so it also helps when the

mapped schedule does not fit the horizons of the problem and Gurobi discards it as a start. The hint priority `VarHintPri` is not set.

Mode `"both"`. The solve sets `Start` and `VarHintVal` on the same variables, and performs the completion if the source requests it.

Relaxation Start

If `relaxation_warm_start` is true and no start or hint was applied (the entry `applied` of `warm_start` is false), the solve computes a start without a source:

1. Copy the model, and relax its linear rows with Gurobi's function `feasRelaxS`, which minimises the sum of the absolute violations. The relaxation keeps integrality, variable bounds and general constraints.
2. Optimise the relaxed copy with the time limit max{1, `relaxation_time_limit`} seconds and the MIP gap 0.05.
3. If the copy has a solution, set `Start` of every integer variable of the original model to the rounded value of the variable of the same name in the copy.

The relaxed solution is not a feasible point of the original model. It gives integer values only, and the original model must find its own first solution.

Warm-Start Reports

The result of a solve carries two dictionaries (Section 8).

Key	Content
`warm_start.applied`	`true` when at least one value was set.
`warm_start.values`	Number of values set.
`warm_start.mode`	The mode used, or `null` without a source.
`warm_start.source_horizon`	`H1` and `H2` of the source, or `null`.
`warm_start.completion`	`requested`; `status` (`"not_requested"`, `"completed"`, `"infeasible"`, `"infeasible_or_unbounded"`, `"time_limit"` or `"status_`q`"` with the Gurobi status code q); `fixed_integer_variables`; `values_copied`; `runtime_seconds`; `iis_constraints` (list of names, empty unless the status is `"infeasible"`).
`relaxation_warm_start`	`enabled`; `status` (`"disabled"` when the option is false, `"skipped"` when it is true and a start was applied, otherwise the Gurobi status of the relaxed copy as a string); `relaxation_objective`; `values_copied`; `runtime_seconds`.

Warm Starts in Sweeps

The sweep functions (Section 9) with `warm_start=True` use, for each case, the result of the nearest preceding case of the same H_1 that has a solution as the source of the solve, with the mode `warm_start_mode`. A case that raised an error or has no solution is skipped. The first case of a sequence has no source. Warm starts between cases need the preceding results, so they are refused together with parallel cases (Section 9.5).

Greedy Heuristic

```
build_greedy_warm_start(cfg, *, repair_trigger=0.70)
    -> (dict, GreedyWarmStartReport)
```

The function is in the module `fleet_management.greedy_warm_start`; the package does not export it. The argument `cfg` is the normalised configuration of an input (Section 3). The trigger `repair_trigger` must lie in $[0, 1]$ (`ValueError` otherwise). The function works for every model of this specification. It builds a schedule that tries to satisfy the rows of the model. It does not try to give a low cost. It returns a source for `solve()`: the dictionary with the keys `H1`, `H2`, `x`, `m`, `r`, `complete_binary_start` (`true`) and `completion_time_limit` (60), and a report.

Construction pass. Keep a proxy state $\hat{s}_{i\ell}$ for each cell, with the initial value $\mu_{0,i\ell}$. Treat the days $k = 1, \ldots, T$ in order.

1. Order the missions by decreasing difficulty $\max_{i,\ell} \mu^{\mathrm{inc}}_{i\ell jk}/\tau_{i\ell}$. A tie goes to the mission with the larger index.
2. Take the missions in this order. Give each one to the train i that has no mission yet on day k and minimises the pair $\big(\max_\ell(\hat{s}_{i\ell} + \mu^{\mathrm{inc}}_{i\ell jk})/\tau_{i\ell},\ \sum_\ell(\hat{s}_{i\ell} + \mu^{\mathrm{inc}}_{i\ell jk})/\tau_{i\ell}\big)$ in lexicographic order. A tie goes to the smaller index i. Increase $\hat{s}_{i\ell}$ by $\mu^{\mathrm{inc}}_{i\ell jk}$ for the train and day. The input check $F > M$ (Section 3.12) guarantees a train for every mission.
3. Let r be the value of `repair_trigger`. A train without a mission stays idle, except when some component has $\hat{s}_{i\ell} > r\,\tau_{i\ell}$. Then the train goes to the depot ($x_{i0k} = 1$), every such component is repaired ($x^m_{i\ell k} = 1$), and $\hat{s}_{i\ell} \leftarrow (1 - \rho_{i\ell})\,\hat{s}_{i\ell}$ for it.

The pass schedules no replacement. The proxy uses the whole-state repair map for every cell, whatever its repair law.

Loop pass. After the construction pass the missions are fixed, so the cells are independent except through the depot days of a train. For each cell, compute its reference trajectory ζ (Section 6.4) for the current schedule, with the starting values, the starting anchors and the replacement values of the inputs. The *excess* e of the cell is the largest of

- $\max\{0,\ \zeta^q_T - \zeta^q_{H_1}\}/M_q$ for every tracked quantity and anchor q with natural range M_q (the loop row, Section 6.14);
- the violation of the reliability row of the cell in its exact form, on every day, divided by the natural range of the quantity that the row bounds (Sections 6.7–6.11).

While $e > 10^{-10}$, the pass tries every *free day* $k \in \{H_1 + 1, \ldots, T\}$, a day on which the train has no mission and the cell has no intervention. It puts a repair of the cell on the free day (the train goes to the depot on that day) and recomputes the excess. It keeps the repair that gives the smallest excess; a tie goes to the later day. If no repair lowers e by more than 10^{-10} and replacement is allowed, the same trial is made with a replacement. If no trial lowers e, the pass counts one failure for the cell and goes to the next cell. Repairs and replacements of a cell never use a day on which the train flies a mission.

Report. `GreedyWarmStartReport` is a record; its method `as_dict()` returns the fields below as plain Python types.

Field	Content
`repairs`, `replacements`	Number of ones in `m` and in `r`.
`repeatability_feasible`	`true` when the loop pass counted no failure.
`repeatability_failures`	Number of cells with a failure.
`maximum_repeatability_excess`	Largest raw excess (in the units of its quantity) over all cells after the loop pass.
`maximum_proxy_ratio`	Largest value of $\hat{s}_{i\ell}/\tau_{i\ell}$ in the construction pass and of the reference mean over $\tau_{i\ell}$ in the loop pass.
`assignment_order`	For each day, the list of missions (indices $1, \ldots, M$) in the order that the pass treated them.

A start with failures is still returned. Its completion can then end with the status `"infeasible"` with an IIS.

File & Folder Structure

The package `fleet_management` has one module per task. The shared Gurobi model and the registry of cell builders are in `base.py`; each degradation model has its own module.

```
Fleet_Management/
    pyproject.toml                Python >= 3.10; extra "dashboard" (
       streamlit)
    README.md
    spec/spec.tex, references.bib this specification
    examples/                     example inputs and regression scripts
    tests/                        pytest suite
    euler/                        two Slurm job scripts of the ETH Euler
        cluster
    src/fleet_management/
        __init__.py               public API (Section "Public API")
        config.py                 input reading, checks and broadcasting
           : load_config -> FleetConfig
        solver.py                 solve(): file checks, dispatch,
           serialisation (YAML, JSON, HDF5)
        horizon_sweep.py          sweep_operating_horizons,
           sweep_horizon_grid
        greedy_warm_start.py      build_greedy_warm_start
        validator_dashboard.py    Streamlit entry point of the
           dashboards
        degradation_model/
            base.py               shared model, registry, objective,
               warm starts, result extraction,
                                  performance counters
            gaussian.py           cell builder "gaussian"
            inverse_gaussian.py   cell builder "inverse_gaussian"
            wiener.py             cell builder "wiener"
            gamma.py              cell builder "gamma"; computation of
               the cap A_1
            rainflow.py           cell builder "rainflow" (five bounds)
            gamma_utils/
                gamma_replay_validator.py       replay validator
                gamma_stochastic_validator.py   Monte Carlo validator
                gamma_tail_validator.py         exact validator
                gamma_tail_bound.py
                   moschopoulos_tail_probability
        utils/
            plotter.py            plot_management, plot_horizon_sweep
        validation/
            validator.py          validate
            dashboard_inspector.py, dashboard_gamma.py
```

Cluster jobs. The folder `euler/` holds two Slurm job scripts (Slurm is the job scheduler of the Euler cluster of ETH Zürich). Each runs one script of `examples/regression/`:

- `gamma_horizon_convergence.sbatch` runs `run_horizon_sweep.py`;
- `vbz_progressive_feasibility.sbatch` runs `run_progressive_vbz_feasibility.py`.

Both scripts call only the public API of Section 11.1. The behaviour that this specification defines does not depend on them.

Public API

The package exports the functions of the table.

Function	Defined in
`solve`	this section
`plot_management`	Section 11.3
`plot_horizon_sweep`	Section 11.3
`sweep_operating_horizons`	Section 9
`sweep_horizon_grid`	Section 9
`validate`	Section 12
`validate_gamma_replay_schedule`	Section 12.3
`validate_gamma_replay_files`	Section 12.3
`validate_gamma_stochastic_schedule`	Section 12.3
`validate_gamma_schedule`	Section 12.3
`validate_gamma_schedule_files`	Section 12.3

```
solve(input_path, results_path="output.yaml", *,
      warm_start=None, warm_start_mode="start") -> dict
```

`solve()` reads one input file, builds one Gurobi model for the whole fleet, solves it, and returns the result dictionary of Section 8. Arrays in the dictionary are NumPy arrays; Gurobi objects are not stored in it.

1. The input file must exist (`FileNotFoundError`) and have the extension `.yaml`, `.yml`, `.json`, `.h5` or `.hdf5` (`ValueError`).
2. If `results_path` is not `None`, the solve resolves it before it reads the input. A path without an extension gets the extension `.yaml`. The extension selects the format of the result file: YAML, JSON or HDF5 (as for inputs); another extension raises `ValueError`. The folder of the result file must exist (`FileNotFoundError`) and be writable (`PermissionError`). With `results_path=None` no file is written.
3. The *input reader* reads the file by its extension into a mapping of the keys of Section 13. The horizon sweeps and `validate` use the same reader.
4. `load_config` normalises the mapping and runs the checks of Section 3.12. A failed check raises `ValueError`, and no model is built.
5. The solve builds the model (Section 11.2), applies the warm start (Section 10), optimises, extracts the solution, writes the result file if requested, and returns the dictionary.

Every fleet, whatever its models, goes through the same route: no model has a separate solver. Gurobi errors, such as a missing licence, are not caught.

Cell-Builder Interface

The shared model of `base.py` is built once for the whole fleet. Each cell (i, ℓ) is then filled in by the *cell builder* of its model (the entry of `model` in the input). Builders are registered in a dictionary by model name.

```
register_cell_builder(name, builder)    # CELL_BUILDERS[name] = builder
get_cell_builder(name)                  # NotImplementedError if name is
    not registered
dispatch_cell(ctx, i, l)                # get_cell_builder(ctx.model_of[i
    , l]).add_cell(ctx, i, l)
```

The names are `gaussian`, `inverse_gaussian`, `wiener`, `gamma` and `rainflow`. The error of `get_cell_builder` lists the registered names. A builder is an object with the attribute `name` and the hooks below.

Hook	Duty
`prepare(ctx, cfg, cells, opts)`	Called once for each model that occurs in the fleet, with the list `cells` of the cells of that model. It stores the per-cell arrays of the model in the context and creates the variables of the model's tracked quantities, anchors and selectors, for these cells only (no variable that no row uses). It may tighten variable bounds, and it sets Gurobi's parameter `NonConvex` to 2 when some cell of the model keeps an exact quadratic row.
`add_cell(ctx, i, l)`	Adds every row of one cell that Section 6 assigns to it: the dynamics in the form selected by `encoding` (Section 6.4), the replacement and anchor rows, the repair-cost rows (Section 6.12), the reliability rows in the form selected by `reliability_impl` (Section 6.6), and the loop rows of the full tracked state (Section 6.14).
`extract(ctx, cfg, out)`	Optional. Adds model-specific entries to the result dictionary `out`, after the shared entries are extracted.

The *context* `ctx` carries the sizes (F, L, M, H_1, H_2, T), the Gurobi model, the shared variables (x of shape $F \times (M+1) \times T$; x^m and x^r of shape $F \times L \times T$, with x^r absent when replacement is not allowed; the mean μ of shape $F \times L \times T$; z at the cells with $C_R > 0$; u when $C_D > 0$), the per-cell arrays (model, τ, ε, ρ, initial and replacement values, starting anchors), the increment accessor, the resolved run options and a dictionary `extras` for storage of a model. The increment accessor returns, for train i, mission j (index from 0 for the depot), cell ℓ and day k, the increment after the phase lookup of Section 3.7.

Building order.

1. Create the model, apply the run options, and create the shared variables.
2. Call `prepare` once for each model present, in the alphabetical order of the names.
3. Build the objective (Section 5) and the rows that do not depend on the model: the assignment rows (Section 6.1), the depot-day rows (Section 6.2) and the damage-regularisation rows (Section 6.13).
4. Call `add_cell` for every cell, with i in the outer loop and ℓ in the inner loop.

Every builder must define the shared variable $\mu_{i\ell k}$ for all $k = 1, \ldots, T$ of each of its cells, because the damage regularisation and the output read it. After the optimisation, the shared extraction reads the variables, and then each model's `extract` hook runs. A model is added by writing a module with its builder and registering it, by adding its input checks to `config.py` (Section 3.12), and by adding its section to this specification.

Plotting API and Layout

The package has one schedule plot and one sweep plot, both in `utils/plotter.py`.

```
plot_management(input_file_path, plot_file_path=None) -> None
```

Parameter	Description
`input_file_path`	Result file of `solve()`, in YAML, JSON or HDF5 format. A missing file raises `FileNotFoundError`; another extension raises `ValueError`.
`plot_file_path`	Image file, `.png` or `.pdf`. Default `"output.png"`. A path without an extension gets `.png`. Another extension raises `ValueError`; a missing folder, `FileNotFoundError`; a folder without write permission, `PermissionError`.

The function plots any result that holds a solution, whatever its status: an incumbent found before a time limit is plotted, and the status is shown in the summary panel. It raises `ValueError` when the result lacks one of the keys `F`, `M`, `L`, `H1`, `H2`, `tau`, `mu_0`, `mu`, `x` (names the missing keys), when a shape differs from Section 8, or when $H_1 + H_2 \neq T$. The keys `m` and `r` are read when present and taken as all zero otherwise. The names of the components come from `component_names` (default "Component ℓ" with ℓ from 1); a list of a length other than L raises `ValueError`. The model of each cell comes from the key `model` (Section 8).

Plot Layout

The figure has a grid on the left and a summary panel on the right.

- **Rows.** One row for each cell (i, ℓ), ordered by train and then by component: $F\,L$ rows. The label of a row has two lines: "Train i — *component name*" and the model name of the cell (`gaussian`, `inverse_gaussian`, `wiener`, `gamma` or `rainflow`). A heavier horizontal line separates the trains.
- **Columns.** $T + 1$ columns: "initial" (the values $\mu_{0,i\ell}$) and the days $1, \ldots, T$. A vertical dashed line after day H_1 separates the phases. Above the grid, the labels "transitory" and "operating" name the two parts.
- **Colour.** The background of a field is a green-to-red scale of $\mu_{i\ell k}/\tau_{i\ell}$, clipped to $[0, 1]$ (green 0, red 1). A colour bar is labelled "tracked mean / failure threshold". The values are the tracked means, which are upper bounds of the expected degradation (Section 6.4).
- **Marks.** Each field of a day carries one mark. The first rule that applies decides:

Mark	Rule for cell (i, ℓ) on day k
`P`	replacement: $x^r_{i\ell k} = 1$
`R`	repair: $x^m_{i\ell k} = 1$
`M`$_j$	train i flies mission j: $x_{ijk} = 1$ with $j \geq 1$ (the mark shows j)
`D`	depot day without intervention: $x_{i0k} = 1$, $x^m_{i\ell k} = 0$, $x^r_{i\ell k} = 0$
`I`	idle: the train has no activity on day k (all $x_{ijk} = 0$)

A legend below the grid names the five marks, with a distinct colour for each. A value is 1 when it is above 0.5. The column "initial" has no mark.

Summary panel (title "Run summary"). Each line is a label and a value; a line is omitted when its value is missing from the result. Status (coloured by outcome); Objective; Dimensions F, M, L; Horizon H_1, H_2, T; Models (the distinct model names); Repair (the distinct values of `repair_model`); Objective mode and J_{op}/H_2, if `objective_mode` is in the result; MIP gap in percent; the numbers of variables, continuous variables, integer variables and linear rows; optimizer time; branch-and-bound nodes; repaired components; replaced components. The model counts are those of Gurobi's presolved model when the result carries them (`presolved_performance`), and those of the original model otherwise; the label says which. When the result carries no presolved counts and a file `runner.out` (a Gurobi log) lies in the folder of the result file, the plot reads the counts from the lines "Presolved: r rows, c columns" and "Variable types: a continuous, b integer". The numbers of repaired and replaced components are the numbers of ones in `m` and `r` (components, not depot visits).

```
plot_horizon_sweep(sweep_report_path, plot_file_path=None) -> None
```

`sweep_report_path` is one path, or a list of paths, of reports of `sweep_operating_horizons`. A report is a YAML or JSON file. Another file type raises `ValueError`, a missing file raises `FileNotFoundError`, and an empty list raises `ValueError`. The default image file is `"horizon_sweep.png"`; the rules for `plot_file_path` are those of `plot_management`. Reports in a list must

agree in F, M, L and H_1 (`ValueError` otherwise), and each needs a list `cases`. When several cases have the same H_2, the plot keeps the one with a solution, then the smaller J_{op}/H_2, then the larger objective bound. A report without a case raises `ValueError`. The figure has three panels over H_2 (the cases are joined by lines that are not extrapolations):

(a) J_{op}/H_2 for each case, with a different mark for a proven case (status `"optimal"`), a feasible case that is not proven, and a case without a feasible solution (including an error row); the best feasible case is marked and the values of the proven cases are written;
(b) the relative MIP gap in percent. When the entry `maximum_mip_gap_for_stopping` of `stopping_rule` is a number, a horizontal line shows it; otherwise a mark shows the requested MIP gap of each case;
(c) the numbers of continuous variables, integer variables and linear constraints.

The title names the sweep and the sizes F, M, L, H_1.

Validation Tools

The validation tools check a saved result against its input. They do not call the optimiser and they read no Gurobi object. There are three groups: the structural validator for every model (Section 12.1), the dashboards (Section 12.2), and the validators of the Gamma model (Section 12.3). All tools read the input with the input reader of `solve()` and normalise it with `load_config` (Section 11.1), and read a result file in the formats that `solve()` writes. They read the model of each cell from the result key `model`.

Structural Validator

```
validate(input_path, results_path, validation_path="validation.yaml",
         tol=1e-6) -> dict
```

`validate` checks that the saved schedule and states satisfy the rows of the formulation that do not need the solver, and writes and returns a report. The report file has the extension `.yaml`, `.yml` or `.json` (a path without an extension gets `.yaml`; another extension raises `ValueError`). The function creates the folder of the report; it raises `PermissionError` if the folder is not writable. It raises `FileNotFoundError` for a missing input or result file, and `ValueError` for the following: an unsupported extension, a result without a solution (`x` is `null`), an input that fails the checks of Section 3.12, and a result whose sizes F, M, L, H_1, H_2 or whose array shapes differ from those of the input (the message names the quantity). The solver status is not a check: a feasible incumbent found before a time limit is validated like an optimal solution, and the report echoes the status.

A check passes when its *violation* is at most `tol`. The checks are:

Name	Statement	Violation
`binary`	x, x^m, x^r take only the values 0 and 1	largest distance of an entry from $\{0,1\}$
`one_activity`	$\sum_{j=0}^{M} x_{ijk} \le 1$ for all i, k (Section 6.1)	$\max\{0,\ \max_{i,k}(\sum_j x_{ijk} - 1)\}$
`mission_coverage`	$\sum_i x_{ijk} = 1$ for $j = 1,\dots,M$ and all k; the depot $j = 0$ is not summed	$\max_{j\ge1,k} \lvert \sum_i x_{ijk} - 1 \rvert$
`depot_gating`	$x^m_{i\ell k} + x^r_{i\ell k} \le x_{i0k}$ (Section 6.2); $x^r = 0$ if `allow_replacement` is false	largest excess of the left over the right side
`idle`	if the result has `idle`: $\texttt{idle}_{i\ell k} = x_{i0k} - x^m_{i\ell k} - x^r_{i\ell k}$	$\max \lvert \texttt{idle} - (x_{i0k} - x^m - x^r) \rvert$
`damage_rows`	if $C_D > 0$: the rows of Section 6.13 hold for the saved u and μ	largest violation of a row

`loop_`q	the loop row (Section 6.14) $q_{i\ell T} \le q_{i\ell H_1}$ for each tracked quantity or anchor $q \in \{$`mu`, `v`, `R`, `K`, `g_mu`, `g_v`, `g_R`, `g_K`$\}$. The result carries every one of them, with `NaN` (in a file `null`) at the cells that do not track it; these entries are skipped	$\max\{0,\ \max_{i,\ell}(q_{i\ell T} - q_{i\ell H_1})\}$
`objective`	the objective recomputed from the saved arrays equals the saved `objective`	relative difference of the two values, with the divisor $\max\{1, \lvert$`objective`$\rvert\}$

The recomputed objective uses the day cost c_k of Eq. (49) with the saved x, z, x^r and the costs of the input, and the saved u. For `objective_mode` equal to `total` it is $\sum_{k=1}^{T} c_k + C_D\, u$; for `operating_average` it is $J_{\text{op}}/H_2 + C_D\, u$; for `evaluation_total` it is the projected cost P_E of Eq. (51) (Section 5). The check does not test the optimality of the schedule. It tests that the reported objective is the cost of the reported solution.

The loop checks use the saved tracked states, which satisfy the loop row, and not the reference trajectory: the reference trajectory of a feasible schedule can violate the loop row (Section 6.4).

The report is a mapping with the keys `status`, `solver_objective`, `recomputed_objective`, `tolerance`, `checks` (a list of mappings with `name`, `passed`, `violation`, `tolerance`), `passed` (all checks passed) and `max_violation` (the largest violation).

Dashboards

The dashboards are Streamlit pages (Streamlit is a Python library for web dashboards; it is in the extra `dashboard`). The entry point is `validator_dashboard.py`; the command is `python -m streamlit run src/fleet_management/validator_dashboard.py`. A sidebar selects the mode, the input file, the result file and the tolerance (default 10^{-6}), and a button loads the pair. The dashboards read only the input and the result: they need no log file. A failed load shows the error on the page; no page is drawn from partial data.

Mode "Result inspector" (every model). Tabs:

1. *Overview*: status, objective, the models present, H_1, H_2, T, the number of reliability violations, the largest μ/τ, and the cells with the smallest margin.
2. *Reliability*: a table of the margin of every (train, component, day), with filters for train, component and violations only.
3. *Repeatability*: for each cell and each saved tracked state, the value of the loop row (Section 6.14).
4. *Train timeline*: the mean of each component of one train over the days, with the threshold τ and the marks of Section 11.3.
5. *Heatmap*: the margins on the grid of cells and days.
6. *Component comparison*: one component across the trains.
7. *Validation*: the checks of `validate` (Section 12.1).
8. *Raw data*: the table behind the tabs, with an export.

The *margin* of a cell on a day is the right side minus the left side of the reliability row of its model and bound, in its exact form, with the τ, ε and bound of that cell and the saved tracked states (Sections 6.7–6.11). A margin of at least $-$`tolerance` means that the row holds. The exact form is used also when the solve used the tangent or piecewise-linear form, so the margin shows the true slack. The result carries every tracked quantity; the dashboard leaves out the entries that are `NaN`. The margins of cells with different models and bounds are not comparable in size; only the sign is.

Mode "Gamma diagnostic" (Gamma cells). Tabs: an overview with the counts of repairs and replacements and the outcome of the replay validator; the tracked mean of each Gamma cell with its cap $A_1(\beta^*\tau, \varepsilon)/\beta^*$ (Section 6.10); the exact tail probability of the true damage against ε (Section 12.3); a heatmap of this tail over the cells and days; and the raw data.

Validators of the Gamma Model

The three validators of this section check the Gamma cells of a result. Each takes the normalised configuration `cfg` (Section 3) and the result dictionary, or an input path and a result path, and returns a report (a mapping of plain Python types). Each raises `ValueError` when `cfg` has no Gamma cell, when a required array is missing, has a wrong shape or has a non-finite entry, and when an array that must be binary is not within `tolerance` of $\{0, 1\}$. The argument `tolerance` (default 10^{-8}) must be positive. With `raise_on_failure=True` a report with `valid = false` raises `AssertionError`. The file versions take also `report_path` (default `None`): the report is written there as YAML or JSON (`ValueError` for another suffix), and the folder is created if it is missing. The tracked state of a Gamma cell is the mean μ and, under ARD_1, its anchor g^μ (Section 6.10).

Replay validator.

```
validate_gamma_replay_schedule(cfg, result, *, tolerance=1e-8,
                               raise_on_failure=False) -> dict
validate_gamma_replay_files(input_path, result_path, report_path=None,
                            **kwargs) -> dict
```

It replays the bookkeeping of the saved schedule. It reads the keys `x`, `m`, `r` (optional), `mu`, `z`, and the anchor `g_mu` of the mean for the cells that use ARD_1. Global violations (strings in `global_violations`): a result horizon different from T; a train with more than one activity; a mission not covered exactly once; repair and replacement of a cell on one day; an intervention on a day with a mission or without a depot day. For each Gamma cell and day k the validator computes the reference trajectory of the mean and of its anchor (Section 6.4) from the saved schedule, with the initial values, the starting anchor and the replacement value of the cell. The *event* of the day is `"replacement"`, `"repair"`, `"mission"` or `"idle"`. The errors, by field, are:

Field	Error
`tracked_mean`	$\max\{0,$ reference mean $-\,\texttt{mu}_{i\ell k}\}$: a tracked mean below the reference breaks the upper-bound property of the one-sided rows (Section 6.4). A tracked mean above the reference is slack and is not an error.
`mean_anchor`	the same for the anchor `g_mu`, under ARD_1
`removed_mean`	$\lvert z_{i\ell k} - x^m_{i\ell k}\,\rho_{i\ell}(\mu_{i\ell,k-1} - g_{i\ell,k-1})\rvert$ computed from the saved states, with $g = 0$ under ARD_∞ (Section 6.12); $z = 0$ on a day without repair
`cap`	$\max\{0,\ \texttt{mu}_{i\ell k} - A_1(\beta^*\tau, \varepsilon)/\beta^*\}$ (Section 6.10)
`repeatability`	$\max\{0,\ \texttt{mu}_{i\ell T} - \texttt{mu}_{i\ell H_1}\}$, and the same for the saved anchor

An error above `tolerance` adds a mapping `i`, `l`, `k`, `event`, `field`, `expected`, `saved`, `error` to `violations`. The report has the keys `validator`, `valid` (no global violation and no violation), `solver_status`, `dimensions`, `gamma_cells`, `gamma_ard1_cells`, `transitions_checked`, `repairs`, `replacements`, `maximum_errors` (the largest error of each field), `global_violations`, `violations` and `timing`.

Exact validator (module `degradation_model.gamma_utils.gamma_tail_validator`; the package does not export it).

```
validate_gamma_tail_bound_schedule(cfg, result, *, tolerance=1e-8,
```

```
    convolution_tolerance=1e-12, max_series_terms=100000,
    include_steps=False, raise_on_failure=False) -> dict
validate_gamma_tail_bound_files(input_path, result_path, report_path=
   None,
                                **kwargs) -> dict
```

It checks the safety claim itself, for the saved schedule and without the tracked states. For each Gamma cell it rebuilds the true damage on each day $k = 1, \dots, T$ as a list of independent Gamma terms, as in the proof of the class of true states (Section 6.10): every mission adds a term with the exact rate of its increment; a repair multiplies the rate of the repairable terms by $1/(1-\rho)$ (the shape is unchanged; for $\rho = 1$ the repairable terms disappear); under ARD_1 the repaired terms join the frozen part of the anchor; a replacement starts a new list with the replacement term. An initial or replacement damage without a rate (the inputs `gamma_beta_0` or `gamma_beta_new` are omitted) is a known constant c and is not a term: the tail is then $\Pr(\text{sum of the terms} > \tau - c)$, which is 1 if $c > \tau$. It computes the tail $\Pr(D_k > \tau)$ of the sum of the terms with the Moschopoulos series. The series writes the law of a sum of independent Gamma variables with different rates as a mixture of Gamma laws with non-negative weights. When all rates are equal, the tail is the Gamma tail. The partial sum of the series is a lower bound of the tail. The validator adds the mixture weight that remains, which is at most `convolution_tolerance`, and so obtains a certified upper bound of the tail. If the series does not reach this tolerance within `max_series_terms` terms, the validator raises `RuntimeError`. The argument `convolution_tolerance` must lie in $(0, 1)$ and `max_series_terms` must be positive. A day passes when the upper bound is at most $\varepsilon_{i\ell} +$ `tolerance` and the saved mean is at least the exact mean $\mathbb{E}D_k$ minus `tolerance`. Under ARD_1 the saved anchor must also be at least the exact mean of the frozen part, minus `tolerance`. (The saved values may be larger: they are upper bounds.) The report adds, to the keys of the replay report, `minimum_reliability_slack` ($\min(\varepsilon -$ tail upper bound)), `worst_reliability` (the step that attains it), `maximum_remaining_mass`, the numbers of series terms, and the timing of the tail computations. With `include_steps=True` the report has the list `steps` with the terms and values of every cell and day.

Monte Carlo validator.

```
validate_gamma_stochastic_schedule(cfg, result, *, repetitions=100000,
    random_seed=20260908, maximum_schedule_failure_rate=None,
    confidence_level=0.95, batch_size=20000, tolerance=1e-8,
    raise_on_failure=False) -> dict
validate_gamma_schedule(cfg, result, *, mode=None, repetitions=None,
    random_seed=None, maximum_schedule_failure_rate=None,
    confidence_level=None, batch_size=None, tolerance=1e-8,
    raise_on_failure=False) -> dict
validate_gamma_schedule_files(input_path, result_path, report_path=None,
                              **kwargs) -> dict
```

The schedule stays fixed. In each of `repetitions` runs, the validator draws the true damage of every Gamma cell: the initial damage, each mission increment and each replacement draw are independent Gamma variables with the inputs' means and rates (the initial damage under ARD_1 is the sum of two independent terms with the means g_0^{μ} and $\mu_0 - g_0^{\mu}$, the first one being the anchor). An initial or replacement damage without a rate (the inputs `gamma_beta_0` or `gamma_beta_new` are omitted) is the known constant of its mean. Each day it applies the repair law of the cell to the sampled path: $D \leftarrow (1-\rho)\,D$ under ARD_∞; $D \leftarrow g + (1-\rho)(D-g)$ then $g \leftarrow D$ under ARD_1; a replacement draws the damage of the replacement part and, under ARD_1, sets g to it. After each day it records whether $D > \tau$. A *schedule failure* is a run in which some Gamma cell exceeds its threshold on some day. The draws use a generator seeded with `random_seed` and are made in batches of `batch_size` runs; the same arguments give the same

report. `confidence_level` must lie in $(0, 1)$, `repetitions` and `batch_size` must be positive, and `maximum_schedule_failure_rate` (if given) must lie in $[0, 1]$.

The report counts the failures of the fleet, of each cell and of the worst day of each cell. Each rate comes with its one-sided Clopper–Pearson upper confidence bound at `confidence_level`. For f failures in n runs the bound is the `confidence_level`-quantile of the Beta$(f + 1,\ n - f)$ law, and 1 if $f = n$. The keys are:

Key	Content
`validator`, `mode`, `valid`, `timing`	Name of the validator, the mode, the verdict, the time used.
`scope`, `complete_fleet_validation`	"Gamma cells only"; false if the fleet has cells of other models.
`excluded_cells`	For each cell that is not a Gamma cell: i, l, component, model, reason.
`repetitions`, `random_seed`, `batch_size`, `confidence_level`	The arguments used.
`schedule`	failed_replays, failure_rate, failure_rate_upper_confidence_bound, maximum_failure_rate, observed_target_met, confidence_qualified_target_met.
`cells`	For each Gamma cell: i, l, component, repair_model, threshold, analytical_epsilon, failed_replays, failure_rate and its upper confidence bound, worst_time_step with the failures, the rate and the bound of that day, maximum_sampled_damage, maximum_threshold_excess.

The report is valid when the observed schedule failure rate is at most the target `maximum_schedule_failure_rate`, or when no target is given. The result is an empirical stress test and not a proof: zero failures in n runs does not show that the true rate is below $1/n$.

`validate_gamma_schedule` selects the validators by `mode`: `"deterministic"` (the replay validator; the default), `"stochastic"` (the Monte Carlo validator), or `"both"` (a report with the keys `deterministic` and `stochastic` and `valid` true only if both are valid). Another value raises `ValueError`. An argument that is `None` takes the default of the Monte Carlo validator. The Monte Carlo validator samples Gamma cells only. It lists every other cell in `excluded_cells` with the reason "not a Gamma cell".

Input File Schema

Format Rules

The supported formats are **YAML** (`.yaml`, `.yml`), **JSON** (`.json`) and **HDF5** (`.h5`, `.hdf5`). The format follows the extension of the path, in lower case. Any other extension raises an error. One reader serves every function that takes an input path.

- A file holds one mapping from the keys of Section 3 to values. The three formats hold the same keys with the same meaning.
- An array is a nested list in YAML and JSON. The outermost list runs over the first axis of the shape, in the order of Section 3: train, component, mission, day.
- A key that some cell reads is required if it has no default. A key that no cell reads may be omitted. For a key that only some cells read, the entries of the other cells are `null`.
- YAML is read in the safe mode: a file cannot create program objects.

HDF5 Layout

An HDF5 file has a flat layout: every key is stored at the root of the file, as an attribute or as a dataset. The reader makes one entry of every root attribute and of every root dataset, so every key of Section 3 can be read from HDF5. A byte string is decoded as UTF-8. A text attribute that begins with `{` or `[` is parsed as JSON.

Input file.

- A scalar number, a flag or a single string is an attribute, or a dataset of shape ().
- `H` is an integer, or two integers, as an attribute or as a dataset.
- An array of numbers is a dataset of type `float64` with the shape of the key. A `null` entry is `NaN`.
- A selector array and `component_names` are datasets of strings, or JSON text. A `null` entry of a selector is the empty string.
- `gurobi_params` is JSON text.

Output file. The result keys of Section 8 are stored by this rule: a number or a string is an attribute; an array of numbers is a dataset of type `float64`; a list of strings, a mapping or a list of records is JSON text in an attribute. A key whose value is absent is not stored. The table below lists the placement of every key. The arrays `v`, `R`, `K`, `g_mu`, `g_v`, `g_R`, `g_K` and `z` hold `NaN` where the quantity is not tracked, because a `float64` dataset has no `null`.

Placement	Keys
Attribute: number or string	`status`, `objective`, `mip_gap`, `bound`, `F`, `M`, `L`, `H` (one or two integers), `H1`, `H2`, `T`, `objective_mode`, `evaluation_horizon`, `u`, `J_initialization`, `J_op`, `J_op_average`, `peak_damage`, `damage_penalty`, `operating_objective`, `J_total`, `projected_evaluation_cost`, `evaluation_objective`; the selectors `bound_method`, `repair_model`, `reliability_impl` and `encoding` when they are one string
Dataset (`float64`)	`x`, `mu`, `v`, `R`, `K`, `g_mu`, `g_v`, `g_R`, `g_K`, `z`, `m`, `r`, `idle`, `step_costs`, `tau`, `mu_0`, `v_0`
JSON text in an attribute	`model`, `component_names`, `component_costs`, `performance`, `optimization_progress`, `warm_start`, `relaxation_warm_start`; the selectors `bound_method`, `repair_model`, `reliability_impl` and `encoding` when they differ between cells

Keys by Cell Type

The table below lists, for each cell type, the keys that the cell requires and the optional keys that it reads in addition to those of the first row. A transitory profile (`*_trans`) is optional for every profile key. The optional keys of the first row are read at every cell; the others only at cells of the type.

Cell type	Required	Optional (default in Section 3)
Every cell	`F`, `M`, `H`, `model`, `tau`, `epsilon`, `rho`, `mu_0`, `mu`, `C_M`, `C_R`, `C_rep`, `C_D`	`L`, `repair_model`, `reliability_impl`, `encoding`, `tangent_ref`, `pwl_points`, `anchor_mu_0`, `replacement_mu`, `replacement_as_new`, `allow_replacement`, `penalty_type`, `component_names`, `mu_trans`, the run options

Cell type	Required	Optional (default in Section 3)
G	`v`	`v_0`, `replacement_v`, `anchor_v_0`, `v_max_user`, `v_trans`
IG	`eta`	none
W	`sigma`	`v_0`, `replacement_v`, `v_max_user`
Ga	`gamma_beta`	`gamma_beta_0`, `gamma_beta_new`, `gamma_beta_trans`
RF, Mk	`bound_method` with the value `markov`	none
RF, Ca	`v`	`v_0`, `replacement_v`, `anchor_v_0`, `v_max_user`, `v_trans`
RF, Ho	`support`	`R_0`, `R_new`, `anchor_R_0`, `support_trans`
RF, Be	`v`, `support`	`v_0`, `support_0`, `replacement_v`, `support_new`, `anchor_v_0`, `v_max_user`, `v_trans`, `support_trans`
RF, Ch	`cgf`, `s_chernoff`	`cgf_0`, `cgf_new`, `anchor_K_0`, `cgf_trans`

Every rainflow cell also needs the key `bound_method` when its bound is not Cantelli. `anchor_mu_0` and the other starting anchors are read only at cells with `repair_model` equal to `ard1`.

YAML Example (Mixed Models)

The example has $F = 3$ trains, $M = 2$ missions and $L = 2$ components. The six cells use Gaussian, inverse Gaussian, Wiener, Gamma, rainflow with Bernstein, and rainflow with Chernoff. It uses an array, a list and a scalar for cell values, a profile of shape (L, M) and a transitory profile of shape (L, M, H_1). It passes every check of Section 3.12.

```
F: 3
M: 2
L: 2
H: [2, 3]                # H1 = 2, H2 = 3, so T = 5
component_names: [Wheelset, Brake pad]

model:
  - [gaussian, inverse_gaussian]
  - [wiener,   gamma]
  - [rainflow, rainflow]
repair_model:
  - [ard1,   ard1]
  - [ardinf, ard1]       # the default is ardinf
  - [ard1,   ard1]
bound_method:            # read at rainflow cells only
  - [null,      null]
  - [null,      null]
  - [bernstein, chernoff]
reliability_impl:        # linear-row cells (IG, Gamma, Chernoff): null
  - [tangent, null]
  - [pwl,     null]
  - [exact,   null]
encoding:
  - [bigm, bigm]
  - [bigm, bigm]
  - [bigm, indicator]

tau: 1.0
epsilon:
```

```
  - [0.05, 0.05]
  - [0.05, 0.05]
  - [0.10, 0.10]
tangent_ref: 0.4        # read at the tangent cell only
pwl_points: 6           # read at the pwl cell only
rho: [0.8, 0.7]         # per component, the same for every train

mu_0:
  - [0.12, 0.10]
  - [0.15, 0.08]
  - [0.09, 0.14]
v_0:
  - [0.0004, null]
  - [0.0005, null]
  - [0.0004, null]
support_0:              # Bernstein: width of the initial range
  - [null, null]
  - [null, null]
  - [0.05, null]
anchor_mu_0:            # ard1 cells only; at most mu_0
  - [0.10, 0.06]
  - [null, 0.05]
  - [0.00, 0.10]
anchor_v_0:             # ard1 cells with a variance state
  - [0.0002, null]
  - [null,   null]
  - [0.0002, null]
anchor_K_0:             # ard1 cell with Chernoff; K_0 = s * mu_0 = 0.70
  - [null, null]
  - [null, null]
  - [null, 0.50]

replacement_mu: 0.02
replacement_v:
  - [0.00004, null]
  - [0.00004, null]
  - [0.00004, null]
support_new:
  - [null, null]
  - [null, null]
  - [0.03, null]
replacement_as_new: false
allow_replacement: true

# profiles, axes (L, M): one row per component, one column per mission
mu:
  - [0.06, 0.05]
  - [0.04, 0.07]
mu_trans:               # axes (L, M, H1): days 1 and 2
  - [[0.07, 0.06], [0.06, 0.05]]
  - [[0.04, 0.04], [0.07, 0.07]]
v:
  - [0.0001, 0.0001]
  - [null,   null]
support:
  - [0.05, 0.04]
  - [null, null]
cgf:                    # at s_chernoff = 5; at least s * mu
```

```
  - [null, null]
  - [0.22, 0.38]
s_chernoff:
  - [null, null]
  - [null, null]
  - [null, 5.0]

sigma:
  - [null, null]
  - [0.03, null]
  - [null, null]
eta:
  - [null, 0.15]
  - [null, null]
  - [null, null]
v_max_user:
  - [null, null]
  - [0.05, null]
  - [null, null]
gamma_beta:
  - [null, null]
  - [null, 20.0]
  - [null, null]
gamma_beta_new:          # gamma_beta_0 is omitted: the initial damage is
    known
  - [null, null]
  - [null, 20.0]
  - [null, null]

C_M: 1.0
C_R: [2.0, 1.5]
C_rep: [10.0, 8.0]
C_D: 1.5
penalty_type: inf_norm
objective_mode: evaluation_total
evaluation_horizon: 30

verbose: 0
mip_gap: 0.05
time_limit: 600
fast: false
gurobi_params: {Threads: 4}
progress_interval_seconds: 10
relaxation_warm_start: false
```

In the example, the inverse Gaussian cell uses the law `ard1` with the starting anchor $0.06 \le \mu_0 = 0.10$. The Gamma cell has the starting anchor $0.05 \le \mu_0 = 0.08$ and no `gamma_beta_0`, so that its initial damage is known. The Chernoff cell has $K_0 = 5 \cdot 0.14 = 0.70$ by default, so that its starting anchor 0.50 is allowed. The Bernstein cell has the support constant $b = \max(0.05, 0.03, 0.05) = 0.05$, so that $2\kappa b/3 = 0.077 < \tau = 1$ with $\kappa = \ln 10$.

Open Questions

This section lists the design questions that this specification leaves open. The planned work of the project is listed in its to-do file, `Things to change.txt`.

- **Hierarchical failure-probability bounds.** Every reliability row bounds the failure prob-

ability of one component: $\Pr(D_{i\ell k} > \tau_{i\ell}) \le \varepsilon_{i\ell}$ for all i, ℓ, k. A user may want to state the requirement at a coarser level:

  - **Train level:** $\Pr(\exists\, \ell : D_{i\ell k} > \tau_{i\ell}) \le \varepsilon_i$ for all i, k, the probability that *any* component of train i fails. Under independence of the components this reads $1 - \prod_\ell (1 - \Pr(D_{i\ell k} > \tau_{i\ell})) \le \varepsilon_i$, a nonlinear row that couples the components of a train.
  - **Fleet level:** $\Pr(\exists\, i, \ell : D_{i\ell k} > \tau_{i\ell}) \le \varepsilon$ for all k, the probability that *any* component of the fleet fails. Under independence this row couples all trains.

  Both require either union-bound approximations (conservative and linear) or product-form rows (exact and nonlinear). The per-component levels give both through the union bound: $\varepsilon_{i\ell} \le \varepsilon_i / L$ for every ℓ gives the train level, and $\varepsilon_{i\ell} \le \varepsilon/(FL)$ for every cell gives the fleet level.

- **Sampling error of the empirical inputs.** Rainflow counting yields *estimates* of the per-mission mean, variance, support width and cumulant generating function, not exact values. A guaranteed reliability bound needs one-sided confidence margins on these inputs.

- **Gamma cap over the reachable states.** The cap of Section 6.10 is the largest cap that is safe for every member of the class of true states defined there. The states that a schedule can reach form a smaller set, because the tracked mean also satisfies the cap on every earlier day. For a cell with one rate and $\beta^*\tau$ below the threshold $t_1(\varepsilon)$ of Section 6.10, a larger safe cap may exist. Its value is not known.

Changelog

Change log (v0.3, panel review). ARA1 maintenance removed from the Gaussian model (no literature basis for the variance extension). Output `v` shape changed from $F \times V \times 2H$ to $F \times L \times 2H$ with NaN for non-variance components. ε bound tightened from $(0, 0.5)$ to $(0, 0.01]$. IG FSD TODO replaced with verified Proposition (MLR proof). Tightness remark refined to three-mechanism argument.

Change log (v0.4, rainflow extension). Rainflow degradation model added: distribution-free, with per-mission empirical mean/variance/support inputs from rainflow counting; reliability certified by a Cantelli or Bernstein tail bound (new `tail_bound` option, `"cantelli"` default); exact nonconvex-quadratic and conservative LP tangent formulations both specified; loop constraint imposed on both tracked moments, with the argument that moment domination chains across repetitions while the scalar bound comparison $U_{2H} \le U_H$ does not; ARD1 only, mirroring the Gaussian precedent. `formulation` token renamed `"socp"` → `"exact"` (`"socp"` retained as a deprecated alias with a `UserWarning`); the previous identification of the Gaussian/Wiener exact reliability constraint as a rotated SOC was corrected—with the variance affine in the decisions, the squared form is a nonconvex quadratic, handled by Gurobi's `NonConvex=2` support (the `"quadratic"` damage penalty remains genuinely convex SOC). Wiener loop-constraint wording corrected per accepted review item A-02 (moment bounds, not FSD). Version reconciliation: the v0.3 change log above corresponds to the revision previously titled "v0.2 (revised)" on the title page. A post-draft panel pass (markers RF-042–RF-045 and in-place corrections) additionally fixed the loop-induction base case ($m \ge 2$ with the loop row as the $m = 1$ case), the boundary wording of the $\tau > 2\kappa b/3$ precondition, the a-fortiori centering step in the Bernstein application, and the problem-class qualifier for IG/Gamma-only fleets; extended the Bhatia–Davis consistency checks to the initial and replacement states and the shape-validation list to $b^{\text{inc}}, b^0, b^{\text{new}}$; relocated `tail_bound` to the model-specific schema tables; added the fifth plot border style; and corrected the YAML example's ε to respect its own $(0, 0.01]$ bound.

Change log (v0.5, loop-penalty removal). The soft loop-constraint penalty $\mathcal{L}_{\text{loop}}$ and its coefficient C_P were removed throughout (objective, per-model penalty formulas, total penalty sum, input

tables, YAML example, full-problem statement); the hard loop constraints are unchanged for all five models. The former three-mechanism big-M tightness remark was revised to its two remaining cost mechanisms ($C_D\,u$ and $C_R\,z$), and a slack-robustness argument was added: the tracked states are componentwise upper bounds on the true moments, each one-sided dynamics row dominates the exact monotone one-step map, so the hard loop constraint yields $\Phi(\cdot_H) \le \cdot_H$ for the exact second-half period map Φ — the induction premise of the repeatability argument — and monotonicity of the reliability certificates keeps every future repetition certified even when the tracked states carry slack; the penalty's only role was a tie-break toward larger sustainability margins. A subsequent review pass (markers REV-001–REV-022) closed the ARA1 loop-constraint anchor gap by adding the hard rows $\mu_{2H}^{\text{last}} \le \mu_H^{\text{last}}$ (Wiener–ARA1) and $\alpha_{2H}^{\text{last}} \le \alpha_H^{\text{last}}$ (Gamma–ARA1), with a full-state monotonicity derivation showing they restore the chaining premise $\Phi(s_H) \le s_H$, and extended the slack-robustness remark accordingly. The same pass made the idle-as-all-zeros assignment semantics explicit (the C_M term is a genuine decision cost, not a constant), corrected the ARA1 repair-cost anchor index ($k \to k-1$), removed the stale duplicate parameter `alpha_target` (legacy name of τ), reconciled the optionality of `tail_bound` and the defaulted schema fields, added the derived constants to the notation table, expanded the two rainflow FUTURE bullets with pointers to the scoping document's Route-I refinements, and recorded the ARA1 anchor-convention questions (pre- vs. post-repair storage; no reset at replacement) as open items.

Change log (v1.0, merged package). The specification describes the package that merges the branches `lore` and `main`, on `main`'s code base, and it is authoritative for the formulation and for the tools. The five degradation models are cell builders of one registry. The repair laws carry the names of the literature: ARD_1, the reduction of the degradation accumulated since the last intervention [2, §3.1, eq. (4)], and ARD_∞, the reduction of the whole degradation, named by analogy with ARA_∞ [1, eq. (20), p. 11]. The horizon has a transitory and an operating phase, $H = [H_1, H_2]$, and the loop row bounds the full tracked state, anchors included, at $k = T$ by its value at $k = H_1$. Every dynamics row is one-sided, in big-M or indicator form. The rainflow model offers five tail bounds (Markov, Cantelli, Hoeffding, Bernstein, Chernoff); the quadratic reliability rows have an exact, a tangent and a piecewise-linear form. The inverse Gaussian row is a cap from a conditional tail mean, safe for any dependence between missions; the Gamma row is the sharpest safe cap on the tracked mean over the class of true states, for any mix of rates. The reliability level, the tangent point and the costs are set per cell. The horizon sweeps, the warm starts, the schedule plot and the validators are specified. The input keys follow one table, and unknown or old keys raise an error. The change markers of earlier versions are removed; git history keeps them.

References

References

[1] L. Doyen and O. Gaudoin. Classes of imperfect repair models based on reduction of failure intensity or virtual age. *Reliability Engineering & System Safety*, 84(1):45–56, 2004. doi:10.1016/S0951-8320(03)00173-X. Read in the authors' preprint submitted on 25 October 2002, whose page numbers are cited.

[2] S. Mercier and I. T. Castro. Stochastic comparisons of imperfect maintenance models for a gamma deteriorating system. *European Journal of Operational Research*, 273(1):237–248, 2019. doi:10.1016/j.ejor.2018.06.020.

[3] F. Roosta-Khorasani and G. J. Székely. Schur properties of convolutions of gamma random variables. *Metrika*, 78:997–1014, 2015. doi:10.1007/s00184-015-0537-9. Read in arXiv:1601.04731v1, whose page numbers are cited.

[4] G. J. Székely and N. K. Bakirov. Extremal probabilities for Gaussian quadratic forms. *Probability Theory and Related Fields*, 126:184–202, 2003. doi:10.1007/s00440-003-0262-6.